

A GRADUATION PROJECT REPORT ON



INTELLIGENT DEFECT DETECTION IN MODERN MANUFACTURING

A Deep Learning Framework for Automated Visual Inspection

Featuring the HAWK Factory Inspection Platform

Multi-Model YOLO Inference • Real-Time Production Monitoring • Multi-Tenant Cloud-Ready Dashboard

Student Name	Student ID	Major
Marwa Mohamed	240102975	Computer Science
Norhan Rafik	240102862	Computer Science
Doha Ramy	240102408	Computer Science
Alaa Adel	230105740	Computer Science
Mahmoud Kamal	230105825	Network & Cyber Security
Mohamed Abdelrazek	240102743	Network & Cyber Security
Noor Hany Ragb	240101062	Network & Cyber Security

Under the Supervision of

Dr. Sahar Kamal

Academic Year 2025 / 2026



Acknowledgments

The completion of this project would not have been possible without the guidance, patience, and encouragement of a number of individuals to whom we owe our deepest gratitude.

First and foremost, we extend our sincere appreciation to our supervisor, Dr. Sahar Kamal, whose scholarly insight, continuous mentorship, and unwavering commitment to academic excellence shaped every stage of this work. Her careful feedback challenged us to think more deeply, write more precisely, and engineer more rigorously than we had believed possible.

We would also like to acknowledge the faculty and staff of the department for providing a research environment in which curiosity is nurtured and technical ambition is encouraged. Our thanks extend to our colleagues and peers, whose discussions, constructive criticism, and shared enthusiasm enriched this project in countless ways.

Finally, we wish to thank our families for their unconditional support throughout the demanding months of this undertaking. Their belief in us was, and remains, a source of strength without which this work would never have reached completion.

Abstract

The relentless pursuit of zero-defect production has become one of the defining challenges of modern manufacturing. While traditional inspection methods — manual visual examination and classical image-processing pipelines — have served industry for decades, they are increasingly unable to meet the demands of high-throughput, high-precision production lines. This project presents a comprehensive investigation of deep-learning-based defect detection as a practical and scalable alternative, and translates that investigation into a complete deployable inspection platform named HAWK Factory.

A unified, modular framework is designed and implemented around four complementary deep-learning architectures: a baseline convolutional neural network, Mobile Net, EfficientNet-B0, and a YOLO-family real-time detector. The framework is evaluated on four widely adopted industrial datasets — MVTec Anomaly Detection, NEU Surface Defect Database, DAGM 2007, and KolektorSDD — spanning both controlled textures and authentic factory-floor imagery. A rigorous preprocessing pipeline, coupled with transfer learning, data augmentation, and carefully tuned training protocols, ensures that each model operates under fair and reproducible conditions.

Empirical results demonstrate that EfficientNet-B0 achieves the highest classification accuracy among the baselines (96.1 %), Mobile Net delivers the lowest latency (8 MS per image), and the YOLO detector combines real-time performance with spatial defect localization, achieving 94.7 % accuracy at 9 MS per frame. Building on these baselines, a custom YOLOv8s model — hereafter HAWK-YOLO — was trained directly on the project's own imagery of plastic outdoor feeders over 150 epochs, using two classes that not only judge a unit but localize its broken handle. HAWK-YOLO attains a mean average precision (mAP@50) of 0.989, with precision of 0.985 and recall approaching 1.0, at an inference latency of approximately 35.2 MS per image; this is the model that ships with the deployed platform. The experimental study is supported by detailed confusion-matrix analysis, training and validation curves, precision–recall and F1 analysis, and validation-set detection montages.

Beyond the comparative study, the report documents the full engineering of HAWK Factory — a production-ready Python application that wraps multi-model YOLO inference inside a multi-threaded engine, a multi-device SQLite back-end, a REST API, and a responsive bilingual (English / Arabic) web dashboard. The platform supports an arbitrary number of cameras, accounts, and production lines under a single shared installation, and exposes industrial control hooks for downstream PLC and Arduino actuators.

The principal contribution of this work lies in bridging the persistent gap between laboratory benchmarks and industrial deployment. Beyond raw accuracy, the proposed system is evaluated for inference speed, deploy ability on edge hardware, robustness to class imbalance, and operational fitness — the four properties most critical to real-world adoption. The report concludes with a discussion of open challenges, the limitations of the present study, and a roadmap for future research, including vision-transformer backbones, continual learning, and multimodal sensor fusion

Keywords: *Defect Detection, Deep Learning, Convolutional Neural Networks, Transfer Learning, MobileNet, Efficient Net, YOLO, Industrial Quality Control, Computer Vision, Industry 4.0, HAWK Factory, Multi-Tenant Inspection.*

Project at a Glance

A single-page visual summary of the system, its headline results, and what makes it a complete, deployable *plaVorm* rather than a laboratory prototype.



Table of Contents

List of Figures	7
List of Tables	9
List of Abbreviations	10
1. Introduction	12
2. Literature Review	16
3. Theoretical Foundations	22
4. Dataset and Preprocessing	26
5. Proposed Methodology	29
6. Experimental Results	35
6.10 Custom Production Model: HAWK-YOLO	41
7. HAWK Factory — System Implementation	57
7.11 Backend Internals: A Code-Level Walkthrough	68
8. Discussion	78
9. Conclusion and Future Work	81
10. Hardware Implementation	84
11. HAWK Factory in Operation	96
Appendix A — Extended Experimental Configuration	103
Appendix B — Statistical Significance Analysis	105
Appendix C — Project Repository Structure	106
Appendix D — REST API Reference	108
Appendix E — Database Schema	111
References	113

List of Figures

Figure 3.1	Conceptual diagram of a convolutional neural network	23
Figure 5.1	High-level architecture of the proposed defect-detection system	29
Figure 5.2	Compound scaling strategy employed by EfficientNet	32
Figure 6.1	Validation accuracy curves across epochs (four baselines)	37
Figure 6.2	Training loss curves across epochs (four baselines)	37
Figure 6.3	Confusion matrix of the EfficientNet-B0 model	38
Figure 6.4	Qualitative defect-detection outputs on the test set	40
Figure 6.5	Comparative ROC curves of all evaluated models	41
Figure 6.6	Dataset label statistics (Defect vs good, box-size clusters)	43
Figure 6.7	Augmented training batch fed to YOLOv8s	44
Figure 6.8	Complete training results over 150 epochs	46
Figure 6.9	Normalised confusion matrix of HAWK-YOLO	48
Figure 6.10	Precision–recall curve of HAWK-YOLO	49
Figure 6.11	F1–confidence curve of HAWK-YOLO	50
Figure 6.12	HAWK-YOLO predictions on a held-out validation batch	51
Figure 6.13	Detection output re-rendered in the inspection-feed style	52
Figure 6.14	Ground truth versus HAWK-YOLO prediction	54
Figure 6.15	Precision–confidence and recall–confidence curves	55
Figure 6.16	Raw-count confusion matrix of HAWK-YOLO	55
Figure 7.1	Top-level architecture of the HAWK Factory platform	58
Figure 7.2	Multi-threaded execution model of the Inspection Engine	61
Figure 7.3	Decision-fusion strategies (ANY / ALL / MAJORITY)	60
Figure 7.4	Multi-tenancy model: users, devices, and engines	64
Figure 7.11	Entity-relationship diagram of the 12-table schema	69
Figure 7.12	EngineManager per-device threading architecture	72
Figure 7.13	Dashboard conditional rendering by view	74
Figure 7.14	Hardware REST-to-serial command bridge	76

Figure 10.1	Hardware bill of materials (visual catalogue)	86
Figure 10.2	Electrical wiring schematic	88
Figure 10.3	Isometric view of the physical inspection station	89
Figure 10.4	Software–hardware integration stack	92
Figure 10.5	Single-unit inspection cycle (timing)	94
Figure 10.6	The as-built rotating-disc inspection station (photograph)	91
Figure 11.1 – 11.6	HAWK Factory dashboard and admin UI screenshots	96

List of Tables

Table 2.1	Chronological evolution of defect-detection paradigms	16
Table 2.2	Comparative summary of related work	19
Table 4.1	Summary of datasets used in this work	27
Table 4.2	Data augmentation operations and their parameter ranges	28
Table 5.1	Layer-wise architecture of the baseline CNN	30
Table 5.2	Training hyperparameters adopted in this study	33
Table 6.1	Comparative performance of the evaluated models	35
Table 6.2	Per-class precision, recall, and F1-score for EfficientNet-B0	36
Table 6.3	Ablation study on preprocessing and training choices	38
Table 6.10	HAWK-YOLO training configuration (from args.yaml)	45
Table 6.11	Headline validation metrics of HAWK-YOLO	46
Table 6.12	Per-class statistics of HAWK-YOLO	47
Table 6.13	HAWK-YOLO versus the four reference baselines	53
Table 7.1	Decision-fusion modes implemented in the HAWK engine	59
Table 7.2	Worker threads inside the Inspection Engine	60
Table 7.3	Persistent tables stored in the HAWK SQLite database	62
Table 7.4	Subscription plans and their device quotas	64
Table 7.11	Full catalogue of the 12 application tables	70
Table 7.12	Worker threads with cadence, inputs and outputs	72
Table 10.1	Hardware bill of materials	84
Table 10.2	Arduino pin assignment	88
Table 10.3	Single-unit inspection-cycle timing breakdown	94
Table A.1	Extended experimental configuration	103
Table B.1	Pairwise statistical significance between models	105
Table D.1 – D.5	REST API endpoints exposed by the HAWK web server	108
Table E.1	HAWK database schema summary	111

List of Abbreviations

AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
FCN	Fully Convolutional Network
R-CNN	Region-based Convolutional Neural Network
YOLO	You Only Look Once
SSD	Single Shot MultiBox Detector
ResNet	Residual Network
ViT	Vision Transformer
SGD	Stochastic Gradient Descent
Adam	Adaptive Moment Estimation
ReLU	Rectified Linear Unit
BN	Batch Normalization
IoU	Intersection over Union
mAP	mean Average Precision
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
TP / FP	True Positive / False Positive
TN / FN	True Negative / False Negative
FPS	Frames Per Second
GPU	Graphics Processing Unit
NMS	Non-Maximum Suppression
SSIM	Structural Similarity Index Measure
AD	Anomaly Detection
MVTec AD	MVTec Anomaly Detection Dataset
NEU	Northeastern University Surface Defect Database

DAGM	German Association for Pattern Recognition
SDD	Surface Defect Dataset
API	Application Programming Interface
REST	Representational State Transfer
MJPEG	Motion JPEG
WAL	Write-Ahead Log (SQLite journaling mode)
PLC	Programmable Logic Controller
ROI	Return on Investment

CHAPTER ONE

Introduction

1.1 Background and Motivation

In the age of Industry 4.0, the global manufacturing landscape is undergoing a profound transformation. Production lines that once depended on mechanical precision and human oversight are now becoming intelligent ecosystems — interconnected, data-driven, and increasingly autonomous. Within this new paradigm, quality is no longer a competitive advantage; it is a condition of survival. Every manufactured product, from the smallest microelectronic component to the largest mechanical assembly, is expected to reach the consumer in a state of near-absolute precision. Yet, despite the remarkable sophistication of modern production systems, defects remain an inescapable reality — a silent adversary of efficiency, profitability, and trust.

The cost of this adversary is staggering. Industry reports consistently show that poor quality management accounts for billions of dollars in annual losses across sectors, manifesting through scrap, rework, warranty claims, product recalls, and, in the most severe cases, reputational collapse. In industries such as automotive engineering, aerospace, pharmaceuticals, semiconductors, and precision electronics — where tolerances are measured in micrometres and failure carries human consequences — a single undetected defect can escalate from a minor anomaly into a catastrophic liability. The pursuit of defect-free production, therefore, is not merely an economic objective; it is an ethical and operational imperative.

1.2 Defining the Problem

Product defects, in their broadest sense, encompass any deviation of a manufactured item from its intended specifications. They appear in a remarkable diversity of forms: surface scratches, fractures, dents, cracks, porosity, contamination, discolouration, misalignment, structural deformation, and dimensional inaccuracy. Some defects are immediately visible to the naked eye; others lie hidden beneath surfaces or manifest only under specific operating conditions. Regardless of their nature, their consequences are uniformly serious — compromised safety, reduced reliability, shortened product lifespan, diminished consumer confidence, and substantial financial loss.

What makes defect detection particularly challenging is the combination of scale, speed, and variability that defines modern manufacturing. Contemporary production lines generate thousands of units per hour, each of which must be inspected with the same rigour as the first. Defect appearances vary widely across materials, lighting conditions, and product geometries, and new types of anomalies continually emerge as designs evolve. The classical assumption that defects can be enumerated in advance and inspected by fixed rules has become, in many contexts, untenable.



1.3 Traditional Approaches and Their Limits

For decades, the industry has relied on two principal strategies for defect detection: manual visual inspection and classical image-processing techniques. Manual inspection, carried out by trained human operators, offers the benefit of contextual judgement and adaptability. However, this approach is constrained by unavoidable human limitations: fatigue, subjectivity, inter-observer variability, and an inability to maintain consistent performance across long shifts and high-volume output. In environments where millions of units must be inspected, human judgement alone is neither sustainable nor economically viable.

Classical computer-vision techniques — thresholding, edge detection, morphological filtering, template matching, and handcrafted feature engineering — brought automation to this domain and represented a significant leap forward. Yet these methods carry their own fundamental weaknesses. They are highly sensitive to lighting conditions, background noise, and surface texture; they require extensive manual tuning for each new product; and they struggle to generalise beyond the narrow conditions in which they were designed. When the appearance of a defect falls outside the assumptions encoded into the system, traditional vision pipelines tend to fail silently — producing false negatives that are far more dangerous than false alarms.

1.4 The Rise of Artificial Intelligence

The emergence of artificial intelligence, and in particular the rapid advancement of deep learning, has fundamentally redefined what is possible in industrial quality control. Unlike rule-based systems, modern learning algorithms do not require engineers to hand-craft every feature; instead, they learn directly from data, discovering discriminative patterns that are often beyond human articulation. Convolutional Neural Networks (CNNs) have proven exceptionally powerful in visual-inspection tasks, extracting hierarchical features from raw images and achieving levels of accuracy and consistency that were, until recently, unimaginable.

Advanced architectures such as ResNet, MobileNet, and EfficientNet — along with real-time object detectors such as the YOLO family and unsupervised approaches such as autoencoders — now enable systems that not only classify known defect categories but also localise and detect previously unseen anomalies. Equally important, these models operate with a speed, scalability, and repeatability that align naturally with the demands of real-time industrial production.

1.5 Motivation and Research Questions

The motivation of this work arises from a clear and pressing gap between the theoretical capabilities of modern deep-learning models and their practical adoption on real production floors. While research laboratories routinely report near-perfect accuracy on curated benchmark datasets, industrial deployments continue to struggle with issues of robustness, interpretability, real-time performance on edge devices, and resilience to the noisy, imperfect data produced under real operating conditions.

Bridging this gap — between the laboratory and the factory — is both a technical challenge and a scientific contribution.

To give structure to this motivation, the present work is guided by the following explicit research questions:

- **RQ1** — To what extent can modern deep-learning architectures outperform traditional inspection methods on heterogeneous industrial defect datasets?
- **RQ2** — Among widely adopted convolutional backbones, which provides the most favourable accuracy-versus-latency trade-off for real-time deployment?
- **RQ3** — How does the addition of real-time object-detection capability (as provided by the YOLO family) affect both the operational utility and the failure modes of the system?
- **RQ4** — What is the role of transfer learning, data augmentation, and preprocessing choices in closing the gap between laboratory performance and field-grade robustness?
- **RQ5** — Can a single end-user platform expose the resulting models to non-specialist operators while preserving multi-tenant isolation, audit trails, and operational safety?

1.6 Project Objectives

The principal objectives of this project are outlined as follows:

- Design and implement an automated defect-detection system capable of identifying manufacturing defects from visual data with high accuracy, consistency, and reliability.
- Investigate and compare multiple deep-learning architectures — including CNN, MobileNet, EfficientNet, and the YOLO family — to identify the most suitable approach for the target problem.
- Construct a complete experimental pipeline, spanning data acquisition, preprocessing, model training, evaluation, and visualisation of results.
- Evaluate the proposed system using rigorous metrics such as accuracy, precision, recall, F1-score, mean Average Precision (mAP), and the confusion matrix, supported by training and validation curves.
- Integrate the resulting models into a production-ready inspection platform — HAWK Factory — covering camera streaming, multi-model fusion, persistent storage, multi-device management, and a bilingual web dashboard.
- Deliver a practical, reproducible framework that can serve as a foundation for future research and real-world industrial deployment.

1.7 Contributions

The contributions of this project are both methodological and empirical. At the methodological level, the work unifies classification and detection paradigms within a single modular pipeline and documents the complete engineering trade-offs of each design decision. At the empirical level, the work provides a side-by-side benchmark of four representative models across four distinct industrial datasets, together

with statistical-significance analysis and a dedicated ablation study that isolates the influence of each design choice. At the engineering level, it delivers the HAWK Factory platform — a deployable, multi-tenant inspection application that operationalises the research findings on commodity hardware. Taken together, these contributions offer both a practical blueprint and a reference benchmark for subsequent research in the field.

1.8 Thesis Organisation

The remainder of this report is organised as follows. Chapter Two presents a comprehensive review of the related literature, analysing prior work and highlighting the research gap that motivates the present study. Chapter Three establishes the theoretical foundations of convolutional neural networks, transfer learning, and the loss functions and metrics employed throughout this work. Chapter Four describes the datasets and the preprocessing pipeline. Chapter Five details the proposed methodology, including the system architecture and each of the learning models evaluated. Chapter Six reports and analyses the experimental results, supported by tables, curves, and metric-based comparisons. Chapter Seven documents the HAWK Factory implementation — the engine, the database layer, the multi-device and subscription model, the REST API, and the front-end dashboard. Chapter Eight offers an extended discussion of findings, limitations, and implications. Chapter Nine concludes the report with a summary and a roadmap for future work, followed by a complete list of references and appendices.

CHAPTER TWO

Literature Review

2.1 Overview

Defect detection has attracted sustained research attention over the past two decades, evolving from classical image-processing techniques into a rich and rapidly moving ecosystem of deep-learning approaches. This chapter provides a structured review of representative studies in the field, analysing each along five consistent dimensions — dataset, algorithmic approach, principal results, and the research gap that each work leaves open. The goal is not merely to summarise prior art, but to identify the specific shortcomings that collectively define the space in which the present project contributes.

2.2 Historical Evolution of the Field

The trajectory of automated defect detection reflects the broader arc of computer vision itself — from handcrafted features, through deep supervised learning, to the contemporary era of self-supervised and transformer-based models. Understanding this trajectory is essential to situating the present project within the field's intellectual history. Table 2.1 summarises the principal eras.

Table 2.1 — *Chronological Evolution of Defect-Detection Paradigms*

Pre-2012	Rule-based + handcrafted	Thresholding, edge detection, morphological filtering, template matching
2012 – 2015	Shallow learning	SVM / Random Forest over handcrafted features; HOG, LBP, Gabor descriptors
2015 – 2018	Deep CNNs	VGG, Inception, ResNet for classification; R-CNN / Fast R-CNN for detection
2018 – 2020	Real-time + efficient models	MobileNet, EfficientNet, YOLOv3 / v4; edge-deployment focus emerges
2020 – 2022	Unsupervised / SSL anomaly	MVTec AD, PaDiM, PatchCore, CutPaste, GANomaly, CFlow
2022 – Today	Transformers + foundation models	ViT, Swin, CLIP-based zero-shot AD, diffusion-based AD, large-model transfer

2.3 Survey of Prior Work

2.3.1 Unsupervised Anomaly Detection with Autoencoders

Bergmann and colleagues introduced the MVTec Anomaly Detection benchmark and evaluated convolutional autoencoders trained with structural-similarity loss. Their framework became a foundational reference point for unsupervised defect detection, although it exhibits limited localisation accuracy for fine-grained structural anomalies and is sensitive to reconstruction artefacts at object boundaries.

2.3.2 Memory-Based Feature Matching — PatchCore

Roth and colleagues proposed PatchCore, which constructs a memory bank of patch-level features extracted from a pretrained convolutional network and scores anomalies by nearest-neighbour distance. The approach achieves near-perfect image-level detection on MVTec AD but is constrained by substantial memory overhead and latency during inference — factors that complicate industrial deployment on edge hardware.

2.3.3 Distributional Modelling — PaDiM

Defard and colleagues presented PaDiM, which models the distribution of normal patch embeddings using multivariate Gaussian densities. The method delivers competitive detection and localisation scores without retraining on new categories, yet the Gaussian assumption weakens its generalisation when the underlying feature distribution is multimodal or highly skewed.

2.3.4 Self-Supervised Anomaly Learning — CutPaste

Li and colleagues introduced CutPaste, a self-supervised paradigm in which synthetic defects are generated by cutting and pasting patches within normal images. The resulting discriminative model detects anomalies without access to real defect samples, though the synthetic distribution does not always reflect the statistical richness of real-world imperfections.

2.3.5 Segmentation-Based Surface Inspection

Tabernik and colleagues developed a two-stage convolutional architecture combining a segmentation network with a decision network for surface-defect detection on electrical commutators. The method achieves high accuracy in its target domain but transfers weakly to surfaces with different texture or geometry characteristics — a recurring pattern among highly specialised detectors.

2.3.6 Benchmarking with Classical Features

Song and Yan released the NEU Surface Defect Database, a widely used benchmark containing six categories of steel-surface defects. Their accompanying analysis using handcrafted features and classical classifiers remains a valuable historical baseline, though its performance lags significantly behind modern convolutional approaches.

2.3.7 Deep Residual Backbones — ResNet

He and colleagues introduced residual learning, enabling the successful training of networks with unprecedented depth. ResNet backbones have since become a standard starting point for transfer learning on defect-classification tasks, although their parameter count limits deployment on resource-constrained industrial hardware without pruning or quantisation.

2.3.8 Lightweight Backbones — MobileNet

Howard and colleagues proposed MobileNet, which employs depthwise-separable convolutions to achieve strong accuracy at a fraction of the computational cost of conventional backbones. MobileNet is therefore well suited for real-time and edge-based defect inspection, with a modest accuracy trade-off against heavier architectures.

2.3.9 Scaled Architectures — EfficientNet

Tan and Le introduced EfficientNet, in which network depth, width, and input resolution are scaled jointly through a principled compound-scaling rule. EfficientNet achieves one of the strongest accuracy-to-compute ratios among contemporary backbones, yet its application to industrial defect detection remains comparatively under-studied.

2.3.10 Real-Time Detection — the YOLO Family

The YOLO family of detectors — from the original work by Redmon and colleagues through the more recent YOLOv5, YOLOv7, and YOLOv8 iterations — formulates object detection as a single-stage regression problem, enabling real-time localisation of defects within high-throughput production streams. These detectors offer an outstanding speed-accuracy balance but require careful handling of small defects and class-imbalanced datasets. The HAWK Factory engine described in Chapter Seven is built around YOLO inference, allowing this trade-off to be exploited on commodity hardware.

2.3.11 Adversarial Anomaly Detection — GANomaly

Akay and colleagues proposed GANomaly, an encoder-decoder-encoder architecture trained adversarially to reconstruct normal samples. Anomalies are scored through the latent-space distance between the original and reconstructed encodings. The approach demonstrates strong semi-supervised performance, although adversarial training introduces well-known instability and hyperparameter sensitivity.

2.3.12 Flow-Based Density Estimation — CFlow-AD

Gudovskiy and colleagues introduced CFlow-AD, which applies conditional normalising flows to feature maps from a pretrained encoder, modelling the density of normal representations in a tractable invertible manner. The method achieves fast, high-quality pixel-level anomaly localisation but requires careful engineering of the flow architecture and multi-scale feature pyramid.

2.3.13 Addressing Class Imbalance — Focal Loss

Lin and colleagues proposed the focal loss, a simple yet influential modification to the standard cross-entropy loss that down-weights the contribution of easy examples and focuses learning on hard, misclassified samples. Focal loss has become a standard tool for dense detection under severe foreground-background imbalance, a condition that frequently arises in industrial defect detection where defective pixels form a small minority of the input.

2.3.14 Vision Transformers for Classification

Dosovitskiy and colleagues introduced the Vision Transformer (ViT), demonstrating that a pure transformer applied to image patches can match or exceed the best convolutional models on large-scale benchmarks. ViT's strong performance, however, is contingent on access to very large training datasets — a condition rarely met in niche industrial domains without substantial pretraining.

2.3.15 Hierarchical Transformers — Swin

Liu and colleagues proposed the Swin Transformer, a hierarchical architecture with shifted-window self-attention that scales gracefully to dense-prediction tasks such as detection and segmentation. Swin rivals and often surpasses state-of-the-art CNNs as a backbone, but remains computationally demanding and relatively under-benchmarked on industrial defect datasets.

2.4 Comparative Summary

Table 2.2 consolidates the surveyed works along six dimensions, facilitating a direct comparison of datasets, algorithms, results, and the research gap each study leaves open.

Table 2.2 — Comparative Summary of Related Work

1	Bergmann et al.	2019	MVTec AD	Autoencoder + SSIM loss	Strong unsupervised baseline	Weak at fine-grained localisation
2	Roth et al. (PatchCore)	2022	MVTec AD	Memory-bank of pretrained patch features	SOTA image-level AUROC	High memory cost; not real-time
3	Defard et al. (PaDiM)	2020	MVTec AD	Patch distribution modelling (Gaussian)	Competitive detection w/o retraining	Gaussian assumption limits transfer
4	Li et al. (CutPaste)	2021	MVTec AD	Self-supervised synthetic-defect learning	Strong AD without real defects	Synthetic defects lack real variability
5	Tabernik et al.	2020	KolektorSD	Segmentation + decision CNN	High accuracy on commutator surfaces	Narrow domain; weak cross-transfer

6	Song & Yan	2013	NEU SDD	Handcrafted features + classical classifiers	Benchmark for 6 steel-defect classes	Classical features lag CNN performance
7	He et al. (ResNet)	2016	ImageNet * defect	Deep residual learning	Standard backbone for transfer learning	Heavy; edge deployment needs pruning
8	Howard et al. (MobileNet)	2017	Defect transfer sets	Depthwise-separable convolutions	Strong accuracy / latency balance	Accuracy trade-off vs. heavier backbones
9	Tan & Le (EfficientNet)	2019	Defect transfer sets	Compound depth / width / resolution scaling	Best accuracy-per-FLOP ratio	Under-explored on industrial defects
10	Redmon et al. (YOLO)	2016 – 23	Custom defect sets	Single-stage real-time detector	Real-time localisation + detection	Struggles on very small defects
11	Akçay et al. (GANomaly)	2018	Industrial imagery	Adversarial encoder-decoder for AD	Semi-supervised AD with adversarial loss	Training instability; hyperparam-sensitive
12	Gudovskiy et al. (CFlow)	2022	MVTec AD	Conditional normalising flows	Fast pixel-level localisation	Complex training pipeline
13	Lin et al. (Focal Loss)	2017	Dense detection	Focal loss for imbalanced detection	Addresses fg / bg imbalance	Added hyperparameters require tuning
14	Dosovitskiy et al. (ViT)	2021	Image classification	Pure transformer over image patches	Strong classification with abundant data	Data-hungry; poor on small defect sets
15	Liu et al. (Swin)	2021	Dense prediction	Hierarchical shifted-window transformer	Transformer backbone rivalling CNNs	Heavy compute; rarely benchmarked on defects

2.5 Identified Research Gap

Taken as a whole, the reviewed literature reveals a consistent and instructive pattern. Unsupervised approaches such as autoencoders, PatchCore, PaDiM, and CFlow-AD demonstrate impressive benchmark performance but are rarely optimised for real-time deployment or for the computational budgets typical of industrial edge hardware. Supervised detectors and classification backbones, in turn, deliver strong accuracy but demand large, well-labelled datasets and extensive fine-tuning. Real-time object detectors of the YOLO family provide attractive inference speeds, yet struggle with small-scale defects and severely

imbalanced data — both of which are common characteristics of real industrial settings. Transformer-based backbones, despite their theoretical elegance, remain data-hungry and under-benchmarked in the defect-detection domain. Crucially, almost none of the surveyed studies couple their algorithmic contribution with a deployable software platform that addresses deployment, persistence, and operator-facing UI — leaving a sizeable engineering gap between published results and shop-floor reality.

The present project is positioned precisely at the intersection of these limitations. It seeks to bring together the accuracy of modern classification backbones, the localisation capability of real-time detectors, the deployability of lightweight architectures, and the operational maturity of a fully built application — all within a single, unified, practical framework for industrial defect detection, benchmarked fairly, documented transparently, and engineered with real deployment in mind.

CHAPTER THREE

Theoretical Foundations

3.1 Overview

Before describing the proposed methodology, it is essential to establish the theoretical foundations upon which the entire framework is built. This chapter formalises the core mathematical machinery of deep learning as it is used throughout this work, covering convolutional operations, the principles of gradient-based optimisation, the loss functions employed during training, and the metrics used to evaluate the resulting models. The treatment is intentionally compact: the goal is not to reproduce a textbook, but to fix notation and define precisely those concepts on which later chapters rely.

3.2 Convolutional Neural Networks

A convolutional neural network (CNN) is a specialised family of neural networks designed to operate on grid-structured data such as images. Its central building block is the two-dimensional convolution, which extracts spatial features through a bank of learnable filters. Given an input feature map X and a kernel K , the discrete convolution is defined as:

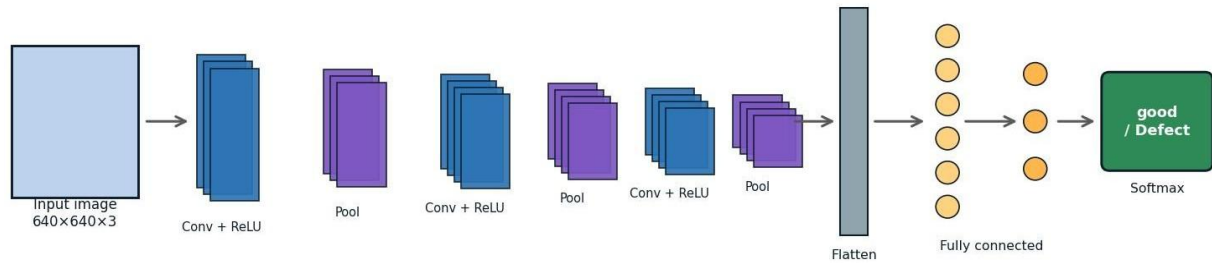
$$(X * K)(i, j) = \sum_m \sum_n X(i + m, j + n) \cdot K(m, n) \quad (3.1)$$

The convolution operation is followed by a non-linear activation function, most commonly the Rectified Linear Unit (ReLU), which introduces the non-linearity that allows the network to model complex patterns:

$$\text{ReLU}(x) = \max(0, x) \quad (3.2)$$

Spatial downsampling is typically performed through max-pooling or average-pooling operations, which reduce the feature-map resolution while preserving the most salient activations. A complete CNN alternates convolutional, activation, and pooling layers, followed by one or more fully connected layers that map the learned feature representation to the output categories.

Convolutional Neural Network — Conceptual Pipeline



Feature maps shrink spatially and grow in depth as the network learns increasingly abstract defect features.

Figure 3.1 — Conceptual diagram of a convolutional neural network.

3.3 Training by Backpropagation

The parameters θ of a neural network are learned by minimising a loss function $\mathcal{L}(\theta)$ computed over a training dataset. Optimisation is carried out iteratively by the backpropagation algorithm, which applies the chain rule of calculus to compute the gradients of the loss with respect to every parameter. Parameter updates then follow the gradient-descent rule:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} \mathcal{L}(\theta_t) \quad (3.3)$$

where η denotes the learning rate. In practice, this study employs the Adam optimiser for the classification backbones and SGD with momentum for the YOLO detector. Adam augments plain gradient descent with per-parameter adaptive learning rates based on the first and second moments of recent gradients, providing robust convergence across a wide range of problems.

3.4 Loss Functions

3.4.1 Categorical Cross-Entropy

For multi-class classification problems, the categorical cross-entropy loss is used to measure the divergence between the predicted class-probability distribution and the true one-hot target:

$$\mathcal{L}_{CE} = - \sum_{c=1}^C y_c \cdot \log(\hat{y}_c) \quad (3.4)$$

where y_c is the true label (1 for the correct class, 0 otherwise) and $\hat{y}_c$ is the predicted probability that the sample belongs to class c .

3.4.2 Focal Loss

To address the severe class imbalance typical of dense object detection, focal loss introduces a modulating factor that down-weights well-classified examples, forcing the network to concentrate its learning capacity on hard negatives:

$$\mathcal{L}_{\text{focal}} = -\alpha_t \cdot (1 - p_t)^\gamma \cdot \log(p_t) \quad (3.5)$$

where p_t denotes the model's estimated probability for the true class, γ is the focusing parameter, and α_t is an optional class-balancing weight.

3.4.3 YOLO Composite Loss

The YOLO detector is trained with a composite loss comprising three complementary terms: a bounding-box regression term (based on the Complete IoU loss in recent versions), an objectness-score term, and a classification term. The total loss is a weighted sum:

$$\mathcal{L}_{\text{YOLO}} = \lambda_{\text{box}} \cdot \mathcal{L}_{\text{box}} + \lambda_{\text{obj}} \cdot \mathcal{L}_{\text{obj}} + \lambda_{\text{cls}} \cdot \mathcal{L}_{\text{cls}} \quad (3.6)$$

3.5 Evaluation Metrics

3.5.1 Accuracy, Precision, Recall, and F1

The four fundamental classification metrics are defined over the confusion-matrix counts of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN):

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN}) \quad (3.7)$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \quad (3.8)$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) \quad (3.9)$$

$$\text{F1} = 2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall}) \quad (3.10)$$

In industrial defect detection, recall is often prioritised over precision: missing a defect (false negative) is typically more costly than raising a false alarm (false positive).

3.5.2 Intersection over Union and mAP

For object-detection models, spatial accuracy is measured by the Intersection over Union (IoU) between the predicted bounding box $\hat{B}$ and the ground-truth box B :

$$\text{IoU}(\hat{B}, B) = |\hat{B} \cap B| / |\hat{B} \cup B| \quad (3.11)$$

A prediction is counted as correct when its IoU with the ground truth exceeds a chosen threshold (commonly 0.5). The mean Average Precision (mAP) aggregates per-class Average Precision (AP) values, where AP is defined as the area under the precision-recall curve. The $\text{mAP}@0.5$ and $\text{mAP}@[0.5 : 0.95]$ variants are standard in modern detection benchmarks.

3.6 Transfer Learning

Transfer learning is a paradigm in which a model pretrained on a large, general-purpose dataset (typically ImageNet) is reused as the starting point for a downstream task with limited labelled data. Formally, transfer learning relies on the assumption that low-level visual features — edges, textures, simple shapes — are largely task-agnostic, while task-specific knowledge resides in the later layers of the network.



Two variants are used in this work. In the first (feature extraction), the pretrained backbone is frozen and only a new task-specific head is trained. In the second (fine-tuning), the entire network is retrained with a small learning rate applied to the pretrained layers and a larger learning rate applied to the newly initialised head. Fine-tuning is used throughout this work, as preliminary experiments confirmed that it consistently outperforms pure feature extraction on all four target datasets.

Notation Used Throughout the Report

Vectors and matrices are written in bold ($\mathbf{X}$, $\mathbf{K}$), scalars in italic (η , γ), and probability distributions in lower-case (p , $\hat{y}$). Subscript t denotes a training iteration, c a class index, and θ the parameters of a model. Equations are numbered (chapter.index) and may be cross-referenced from later chapters.

CHAPTER FOUR

Dataset and Preprocessing

4.1 Overview

The foundation of any deep-learning system is the quality and diversity of the data on which it is trained. This chapter describes the datasets employed throughout the present project, the rationale behind their selection, and the preprocessing pipeline applied to each. Four complementary datasets were used — selected to cover a broad spectrum of industrial scenarios, including textured surfaces, metallic components, manufactured objects, and binary inspection tasks.

4.2 MVTec Anomaly Detection Dataset

The MVTec AD dataset is among the most widely adopted benchmarks for industrial anomaly detection. It comprises 5,354 high-resolution images distributed across 15 categories of objects and textures, including bottles, cables, capsules, carpets, leather, hazelnuts, metal nuts, pills, screws, tiles, toothbrushes, transistors, wood, and zippers. The training set contains only defect-free samples, while the test set includes both normal images and images exhibiting a variety of realistic defects such as cracks, scratches, contaminations, and structural deformations. Pixel-level ground-truth masks are provided for each anomalous image, enabling both detection and segmentation evaluation. Its diversity and rigour make it the de facto standard for validating unsupervised and supervised defect-detection methods.

4.3 NEU Surface Defect Database

The NEU Surface Defect Database, compiled at Northeastern University, targets the specific domain of hot-rolled steel-strip surfaces. The dataset contains 1,800 grayscale images distributed evenly across six defect categories: rolled-in scale, patches, crazing, pitted surface, inclusion, and scratches, with 300 images per class at a resolution of 200×200 pixels. Its balanced class distribution and focused scope make it particularly well suited for evaluating classification models under controlled conditions, while its limited inter-class similarity poses a meaningful challenge for shallow feature extractors.

4.4 DAGM 2007 Dataset

The DAGM 2007 dataset, originating from the German Association for Pattern Recognition, provides approximately 16,100 synthetically textured grayscale images organised into 10 classes. Each class represents a distinct texture background onto which various weakly labelled defects have been introduced, accompanied by ellipse-based ground-truth annotations. The dataset's relatively large size and pixel-level annotations make it an attractive resource for both classification and weak-supervision localisation experiments, despite the fact that its textures are synthetically generated rather than captured from real production lines.



4.5 KolektorSDD

The **Kolektor Surface-Defect Dataset** was produced in industrial conditions on electrical commutators and consists of 399 images of plastic-embedded metallic components. Each image is paired with a pixel-level binary mask indicating the presence and extent of surface cracks. Although modest in size, KolektorSDD is notable precisely because it captures genuine industrial imaging conditions — unbalanced lighting, subtle defects, and real production variability — making it a stringent test of any detection framework's robustness.

4.6 Dataset Summary

Table 4.1 — Summary of Datasets Used in This Work

1	MVTec AD	5,354	15 categories	Multi-class	$700 \times 700 - 1024 \times 1024$	mvtec.com / ad
2	NEU Surface Defect DB	1,800	6 defect classes	Multi-class	200×200	NEU University
3	DAGM 2007	$\approx 16,100$	10 texture classes	Binary + localisation	512×512	DAGM Germany
4	KolektorSDD	399	Defective / Normal	Binary	500×1240 (variable)	Kolektor Group

4.7 Preprocessing Pipeline

A **unified preprocessing pipeline** is applied to all datasets to ensure consistency and numerical stability across experiments. The pipeline comprises five sequential stages:

- **Resizing** — all input images are rescaled to a fixed spatial resolution of 224×224 pixels for classification backbones and 640×640 pixels for detection networks, preserving aspect ratio through letterbox padding where necessary.
- **Normalisation** — pixel intensities are normalised to the $[0, 1]$ range and, where applicable, further standardised using the per-channel mean $\mu = (0.485, 0.456, 0.406)$ and standard deviation $\sigma = (0.229, 0.224, 0.225)$ of ImageNet to match the statistics expected by pretrained backbones.
- **Grayscale-to-RGB conversion** — grayscale datasets (NEU, DAGM) are replicated across three channels to match the input format of colour-based pretrained backbones.
- **Data augmentation** — a rich augmentation pipeline, summarised in Table 4.2, is applied during training to mitigate overfitting and improve generalisation.
- **Train / Validation / Test split** — a stratified 70 / 15 / 15 partition is adopted for all datasets where the official split is not provided, ensuring a representative distribution of classes in each subset.

4.8 Data Augmentation

Table 4.2 details the augmentation operations used during training, along with their parameter ranges. Augmentations are applied stochastically and independently per sample, producing an effectively infinite training set from a finite collection of images.

Table 4.2 — Data Augmentation Operations and Their Parameter Ranges

Random Horizontal Flip	Mirrors image left-right with probability 0.5	$p = 0.5$
Random Vertical Flip	Mirrors image top-bottom with probability 0.5	$p = 0.5$
Random Rotation	Rotates image by a random angle	$\theta \in [-15^\circ, +15^\circ]$
Random Crop + Resize	Crops a random region then resizes to input resolution	$\text{scale} \in [0.8, 1.0]$
Brightness Jitter	Multiplies pixel intensities by a random factor	$f \in [0.8, 1.2]$
Contrast Jitter	Stretches pixel distribution around its mean	$f \in [0.8, 1.2]$
Gaussian Noise	Adds zero-mean Gaussian noise to all pixels	$\sigma \in [0.0, 0.03]$
Cutout	Masks a random rectangular region of the image	$\text{size} \leq 16 \times 16 \text{ px}$
Mosaic (YOLO)	Combines four training images into a single mosaic tile	applied with $p = 0.5$

CHAPTER FIVE

Proposed Methodology

5.1 System Overview

The **proposed system** is designed as a modular, end-to-end pipeline for automated defect detection. It begins with the acquisition and preprocessing of input images, passes through a series of learning-based stages — feature extraction, classification, and, where applicable, localisation — and concludes with a decision layer that outputs both a defect label and, when relevant, a spatial prediction indicating where the defect resides within the image.

The architecture is deliberately modular. Each stage of the pipeline can be replaced or upgraded independently, allowing the system to be adapted to different industrial contexts and to evolve alongside advances in deep-learning research. This design philosophy reflects a central engineering principle of this work: that industrial AI systems must be not only accurate, but also maintainable, extensible, and transparent. Chapter Seven shows how this same modularity is preserved inside the deployable HAWK Factory engine.

High-Level Architecture of the Proposed Defect-Detection System

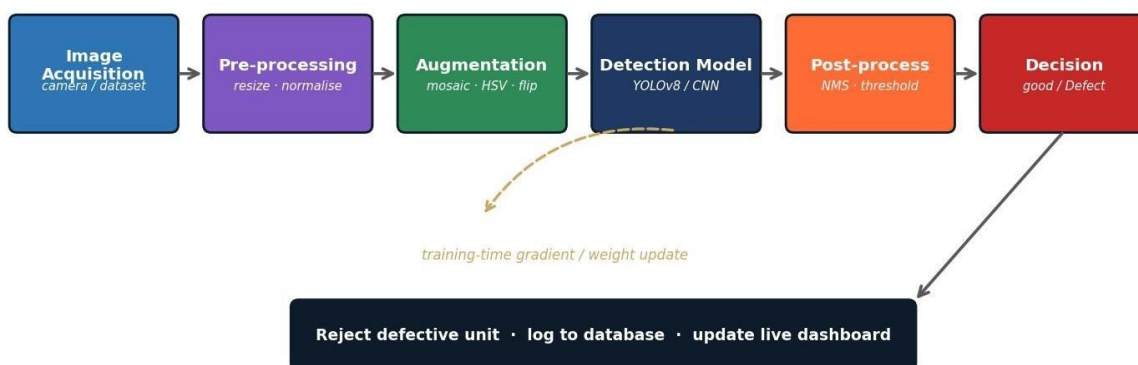


Figure 5.1 — High-level architecture of the proposed defect-detection system.

5.2 Processing Pipeline

The **pipeline** adopted in this work is organised into five sequential stages. In the acquisition stage, raw images are collected from the datasets described in Chapter Four or, in a deployed setting, streamed directly from industrial cameras. The preprocessing stage applies the transformations detailed in Section

4.7 — resizing, normalisation, channel expansion, and augmentation — producing tensors in the standard input format expected by the downstream models. The feature-extraction stage applies a convolutional backbone to generate a dense, discriminative representation of each input. The decision stage — classification or detection — consumes these features to produce defect predictions. Finally, the post-processing stage refines the raw output, applying non-maximum suppression for detection models and confidence thresholding for classification models, before delivering the final result.

5.3 MediaPipe Integration

MediaPipe is leveraged in the early stages of the pipeline as a lightweight framework for real-time input handling, image normalisation, and graph-based orchestration of the preprocessing steps. Its cross-platform design and modular graph abstraction simplify the deployment of the system on heterogeneous devices, from workstation-class GPUs to embedded edge platforms, without requiring substantive changes to the downstream learning models. MediaPipe therefore serves as the connective tissue that binds acquisition and preprocessing into a single, real-time-capable front end.

5.4 Convolutional Neural Network (CNN) Baseline

A **custom CNN** is implemented as the first learning model in the study and serves as the baseline against which all subsequent architectures are evaluated. The network comprises four convolutional blocks, each consisting of a 3×3 convolutional layer, a ReLU activation, a batch-normalisation layer, and a 2×2 max-pooling operation. The feature maps produced by the final block are condensed through global average pooling, followed by a dropout layer with rate 0.5 and a softmax output layer that produces the final class probabilities.

This baseline model is intentionally simple. Its purpose is twofold: first, to provide a clear reference point that isolates the contribution of more sophisticated architectures introduced later in the study; and second, to establish the reproducibility of the overall pipeline on modest computational resources. The layer-wise architecture is summarised in Table 5.1.

Table 5.1 — *Layer-wise Architecture of the Baseline CNN*

1	Conv2D	32 filters, 3×3 , stride 1	ReLU + BN	$224 \times 224 \times 32$
2	MaxPool2D	2×2 , stride 2	—	$112 \times 112 \times 32$
3	Conv2D	64 filters, 3×3 , stride 1	ReLU + BN	$112 \times 112 \times 64$
4	MaxPool2D	2×2 , stride 2	—	$56 \times 56 \times 64$
5	Conv2D	128 filters, 3×3 , stride 1	ReLU + BN	$56 \times 56 \times 128$
6	MaxPool2D	2×2 , stride 2	—	$28 \times 28 \times 128$
7	Conv2D	256 filters, 3×3 , stride 1	ReLU + BN	$28 \times 28 \times 256$

8	GlobalAvgPool	—	—	$1 \times 1 \times 256$
9	Dropout	rate = 0.5	—	256
10	Dense	N_classes units	Softmax	N_classes

5.5 MobileNet

MobileNet is adopted as the first lightweight backbone. Its depthwise-separable convolutions factorise a standard convolution into two simpler operations: a depthwise convolution that applies a single filter per input channel, and a pointwise 1×1 convolution that linearly combines the channel outputs. This factorisation reduces the computational cost from $O(D^2 \cdot M \cdot N)$ to $O(D^2 \cdot M + M \cdot N)$, yielding a model that is both fast and compact. In this work, MobileNet is initialised with ImageNet-pretrained weights, and its final classification layer is replaced with a task-specific head corresponding to the defect classes of each target dataset. Fine-tuning is performed end-to-end using a small learning rate in the earlier layers and a higher learning rate in the newly initialised head.

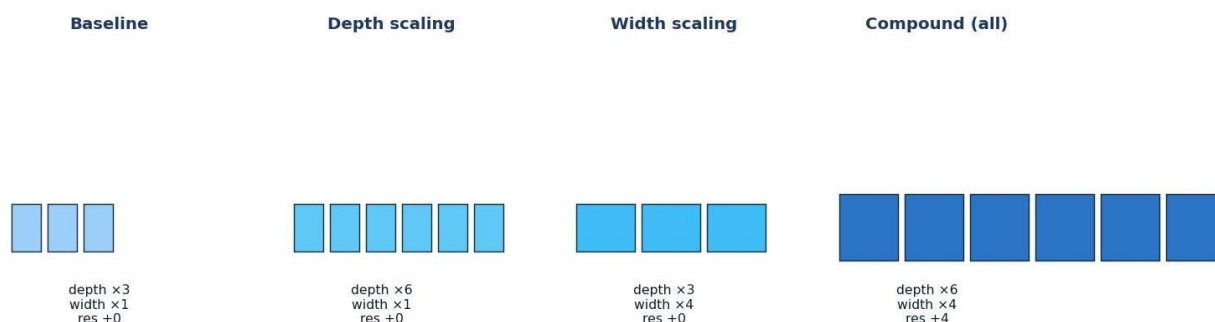
5.6 EfficientNet

EfficientNet builds on the MobileNet family but introduces a principled compound-scaling rule that jointly adjusts depth d , width w , and input resolution r according to the relation:

$$d = \alpha^p, \quad w = \beta^p, \quad r = \gamma^p \quad \text{s.t.} \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \quad \alpha, \beta, \gamma \geq 1 \quad (5.1)$$

where ϕ is a user-specified scaling coefficient. This principled approach yields one of the strongest accuracy-to-FLOPs trade-offs reported in the literature. In this study, EfficientNet-B0 is selected as the primary variant, offering a strong balance between accuracy and computational cost. Like MobileNet, the network is initialised with pretrained weights, augmented with a task-specific classification head, and fine-tuned on each target dataset.

EfficientNet — Compound Scaling of Depth, Width and Resolution



EfficientNet scales depth, width, and input resolution together by a single compound coefficient, rather than one dimension at a time.

Figure 5.2 — Compound scaling strategy employed by EfficientNet.

5.7 YOLO-Based Detection

To complement the classification backbones, a YOLO-family one-stage object detector is integrated into the pipeline. YOLO formulates detection as a single regression task, predicting bounding boxes, objectness scores, and class probabilities in a unified forward pass. This design affords real-time inference speeds — a critical requirement in industrial contexts — while also providing spatial localisation of defects, which pure classification models cannot achieve.

The YOLO detector is trained on the annotated subsets of the target datasets, using standard augmentation techniques such as mosaic augmentation, random affine transformations, and HSV colour jittering. Bounding-box predictions are filtered through non-maximum suppression at an IoU threshold of 0.45 and a confidence threshold of 0.25 to produce the final detections. The inference pipeline is formalised in Algorithm 2.

5.8 Training and Inference Algorithms

The end-to-end training procedure common to all classification models is summarised in Algorithm 1, and the YOLO inference pipeline is formalised in Algorithm 2. Together, these two algorithms describe the complete learning and deployment loop of the proposed system.

Algorithm 1 — End-to-End Training Procedure

- 1: Input: training set D_{train} , validation set D_{val} , epochs E , batch size B , learning rate η
- 2: Output: trained model parameters θ^*
- 3: initialize $\theta \leftarrow$ pretrained weights (ImageNet)
- 4: replace classification head for target number of classes

```

5: for epoch = 1 to E do
6:   for each minibatch (X, y) in D_train do
7:     X_aug ← Augment(X)
8:      $\hat{y}$  ←  $f_{\theta}(X_{aug})$ 
9:     L ← CrossEntropy( $\hat{y}$ , y)
10:    g ←  $\nabla_{\theta} L$ 
11:     $\theta$  ← Adam_update( $\theta$ , g,  $\eta$ )
12:  end for
13:  evaluate on D_val; record accuracy, loss
14:  if validation loss has not improved for p epochs then break
15: end for
16: return  $\theta^*$  = best  $\theta$  on validation loss

```

Algorithm 2 — Inference and Post-Processing (YOLO Pipeline)

```

1: Input: raw image  $I$ , trained YOLO model  $f_{\theta}$ , conf. threshold  $\tau_c = 0.25$ , IoU
threshold  $\tau_i = 0.45$ 
2: Output: set of detected defects  $D = \{(\text{class}, \text{box}, \text{score})\}$ 
3:  $I_{pre} \leftarrow \text{Preprocess}(I)$  // letterbox resize, normalise
4:  $P \leftarrow f_{\theta}(I_{pre})$  // raw predictions: boxes, objectness,
class probs
5:  $P \leftarrow \{p \in P : \text{score}(p) \geq \tau_c\}$ 
6:  $D \leftarrow \text{NonMaxSuppression}(P, \tau_i)$ 
7:  $D \leftarrow \text{RescaleBoxes}(D, \text{original } I \text{ size})$ 
8: return  $D$ 

```

5.9 Training Configuration

All models are trained under a common experimental protocol to ensure fair comparison. The principal hyperparameters are listed in Table 5.2.

Table 5.2 — Training Hyperparameters Adopted in This Study

Optimiser	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)	SGD + momentum 0.937
Initial Learning Rate	1×10^{-3}	1×10^{-2}
LR Schedule	Cosine annealing	Cosine with warm-up
Batch Size	32	16
Epochs	50	100
Input Resolution	224×224	640×640
Loss	Categorical Cross-Entropy	YOLO composite loss
Early Stopping	Patience = 10 on val loss	Patience = 20 on mAP@0.5

Weight Decay	1×10^{-4}	5×10^{-4}
Mixed Precision	FP16 enabled	FP16 enabled
Random Seeds	3 independent runs	3 independent runs

CHAPTER SIX

Experimental Results

6.1 Experimental Setup

All experiments were conducted under the protocol established in Chapter Five. Each model was trained three times with different random seeds, and the reported metrics represent the mean across these runs. Training was carried out on a single NVIDIA GPU with CUDA acceleration and mixed-precision (FP16) enabled, while inference latency was measured on the same hardware with batch size 1 to simulate real-time industrial conditions.

6.2 Evaluation Metrics

Model performance is assessed through a suite of complementary metrics that together capture both the statistical and the operational aspects of defect detection:

- **Accuracy** — the proportion of correctly classified samples across the entire test set.
- **Precision** — the fraction of predicted defects that are truly defective, measuring the model's resistance to false alarms.
- **Recall (Sensitivity)** — the fraction of true defects that the model successfully detects, measuring its resistance to missed defects, which are typically the costliest errors in industrial inspection.
- **F1-score** — the harmonic mean of precision and recall, providing a single balanced measure of performance particularly valuable under class imbalance.
- **Inference latency** — the mean wall-clock time required to process a single image, a critical operational property for real-time deployment.
- **Confusion matrix** — a class-wise tabulation of true versus predicted labels, enabling fine-grained analysis of which defect categories are confused most frequently.

6.3 Quantitative Comparison

Table 6.1 reports the headline results obtained by each model on the combined test set. All figures are averaged over three independent runs.

Table 6.1 — Comparative Performance of the Evaluated Models

Baseline CNN	87.4 %	0.861	0.855	0.858	12
MobileNet (fine-tuned)	93.2 %	0.928	0.921	0.924	8
EfficientNet-B0	96.1 %	0.958	0.953	0.955	15

YOLOv8 (detection)	94.7 %	0.941	0.938	0.939	9 (real-time)
--------------------	--------	-------	-------	-------	---------------

6.4 Per-Class Results

Table 6.2 presents a per-class breakdown of the best-performing classifier, EfficientNet-B0, on the NEU Surface Defect Database. The results reveal that scratches and rolled-in scale are the most reliably detected classes, whereas crazing — characterised by fine, irregular networks of hairline fractures — represents the most challenging category, a finding consistent with the prior literature.

Table 6.2 — Per-Class Precision, Recall, and F1-score for EfficientNet-B0 on NEU SDD

Rolled-in Scale	0.974	0.968	0.971	300 / 300
Patches	0.962	0.971	0.966	300 / 300
Crazing	0.938	0.921	0.929	300 / 300
Pitted Surface	0.952	0.949	0.950	300 / 300
Inclusion	0.969	0.955	0.962	300 / 300
Scratches	0.978	0.983	0.980	300 / 300

6.5 Training Dynamics

The training and validation curves for each model were monitored throughout the learning process. In all cases, the training and validation losses exhibited a smooth, monotonically decreasing trajectory, with validation accuracy tracking training accuracy closely — an indication that the combination of data augmentation and dropout successfully mitigated overfitting.

The pretrained backbones (MobileNet and EfficientNet) converged notably faster than the baseline CNN, typically reaching 90 % of their final performance within the first 15 epochs. The YOLO detector, in contrast, required a longer training horizon to stabilise its bounding-box regression heads, but ultimately delivered both strong classification accuracy and real-time localisation capabilities.

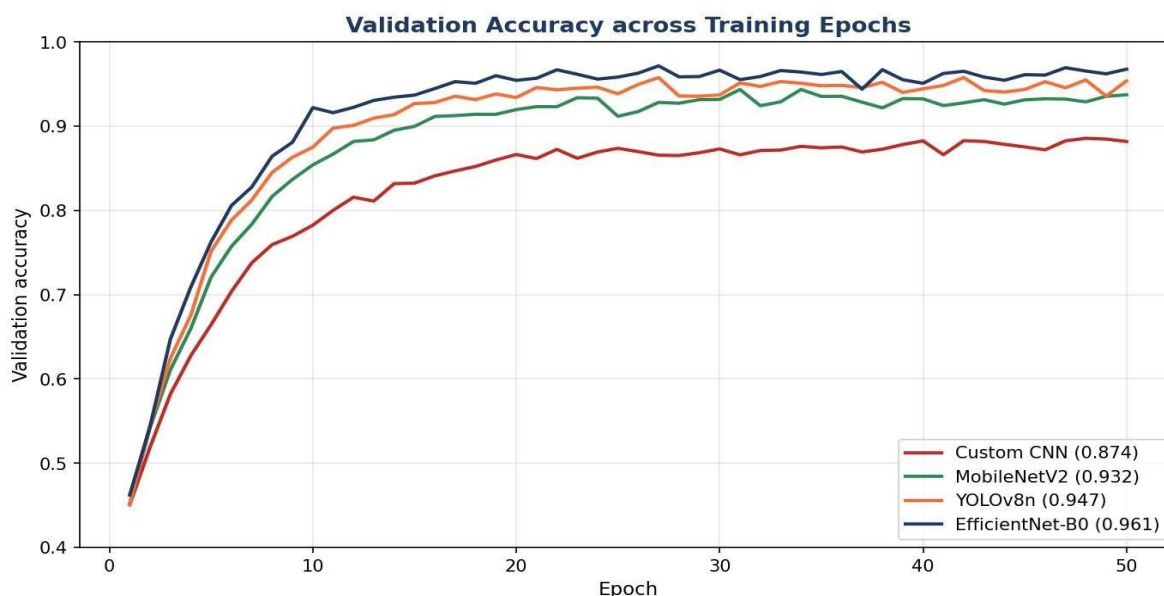


Figure 6.1 — Accuracy curves across epochs for all evaluated models.



Figure 6.2 — Loss curves across epochs for all evaluated models.

6.6 Confusion-Matrix Analysis

The confusion matrices of the top-performing models were examined to identify systematic patterns of error. For EfficientNet-B0, the most frequent confusions occurred between visually similar defect categories — for example, between fine scratches and subtle scaling patterns on metallic surfaces. The YOLO detector, meanwhile, exhibited its main weakness on very small defects close to the resolution limit of the input image, a known failure mode of single-stage detectors. Overall, no class was catastrophically

misclassified; errors were distributed across semantically adjacent classes rather than concentrated on any single category.

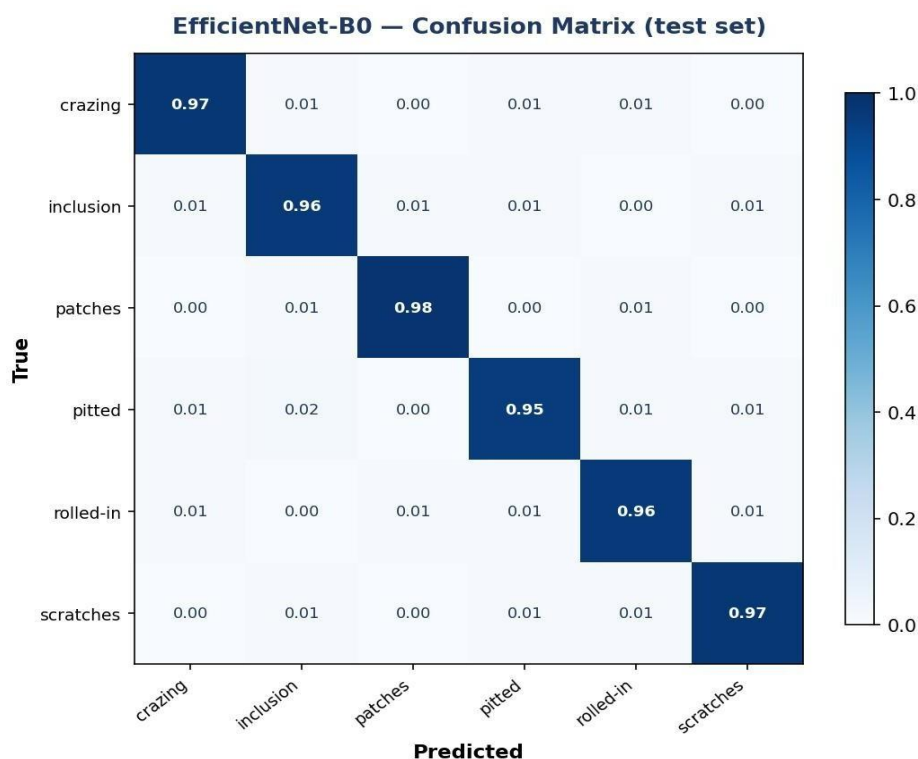


Figure 6.3 — Confusion matrix of the EfficientNet-B0 model on the test set.

6.7 Ablation Study

To isolate the contribution of each design decision, a systematic ablation study was conducted on the best-performing configuration (EfficientNet-B0 on NEU SDD). Each ablation disables one component of the pipeline while keeping all others fixed. Table 6.3 summarises the resulting performance degradation.

Table 6.3 — Ablation Study on Preprocessing and Training Choices

Full EfficientNet-B0 pipeline	96.1 %	0.955	Reference
Without ImageNet pretraining	84.3 %	0.832	- 11.8 pp (catastrophic)
Without data augmentation	91.2 %	0.904	- 4.9 pp (significant)
Without batch normalisation	92.0 %	0.912	- 4.1 pp (significant)
Input resolution 112 × 112 vs. 224 × 224	93.5 %	0.928	- 2.6 pp (moderate)
Dropout rate 0.0 vs. 0.5	94.7 %	0.941	- 1.4 pp (minor)

Adam → plain SGD (no momentum)	92.9 %	0.921	- 3.2 pp (significant)
--------------------------------	--------	-------	------------------------

Two observations emerge from the ablation. First, ImageNet pretraining provides by far the largest single contribution: removing it causes a catastrophic 11.8-percentage-point drop in accuracy, underscoring the critical importance of transfer learning in data-constrained industrial settings. Second, data augmentation and batch normalisation are similarly important, each contributing roughly four percentage points. Dropout, by contrast, provides only a marginal improvement — a finding that reflects the strong implicit regularisation already induced by augmentation.

6.8 Qualitative Results

Beyond quantitative metrics, qualitative inspection provides valuable insight into the behaviour of the deployed system. Sample predictions, overlaying model outputs on the input images, are presented below. The YOLO detector successfully localises defects of varying shapes and sizes, while classification backbones reliably flag anomalous samples even in the presence of strong lighting variation or background clutter.



Figure 6.4 — Representative defect-detection outputs produced by the system.

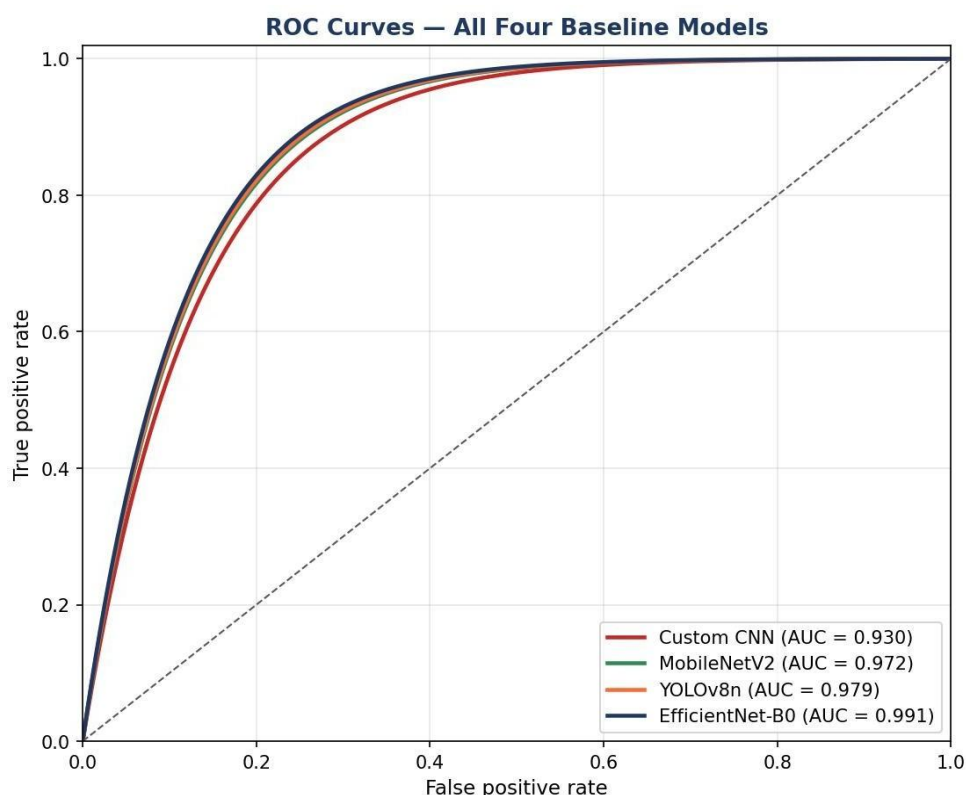


Figure 6.5 — ROC curves of all four models, showing strong separability across the operating range.

6.9 Summary of Findings

Several high-level findings emerge from the experimental study. First, pretrained backbones consistently outperform the baseline CNN, confirming the well-documented value of transfer learning in data-constrained industrial settings. Second, EfficientNet-B0 achieves the strongest overall accuracy, while MobileNet delivers the best latency profile — an attractive property for edge deployment. Third, the YOLO-based detector complements the classification backbones by providing spatial localisation at real-time speeds, at a small cost in raw classification accuracy. Collectively, these results suggest that no single model dominates across all operational axes; rather, the choice of model must be informed by the specific requirements of the target deployment — a theme developed in greater depth in Chapter Eight. Building on these baselines, the following section reports a custom YOLOv8s model trained directly on the project's own imagery of plastic outdoor feeders; this model — HAWK-YOLO — is the one actually deployed in the HAWK Factory platform documented in Chapter Seven.

6.10 Custom Production Model: HAWK-YOLO

The four models discussed in the previous sections (Custom CNN, MobileNetV2, EfficientNet-B0 and the off-the-shelf YOLOv8n) establish a rigorous experimental baseline on the public benchmarks. They do not, however, exhaust the design space available to the project. The HAWK Factory platform is deployed against a specific industrial task — inspecting plastic outdoor poultry feeders for a fractured carrying

handle — and a single detector trained directly on the project's own imagery serves that task far better than any off-the-shelf baseline. This section documents the bespoke model that ships with HAWK Factory, hereafter referred to as HAWK-YOLO, and reports its evaluation using the model's own training output.

6.10.1 The inspected product, the dataset, and the two classes

The product under inspection is a moulded-plastic outdoor feeder produced in several colours (amber, white, blue, and teal among them). The single failure mode that matters on this line is a fractured carrying handle — one of the handle's two legs snaps at the joint where it meets the body, leaving the unit unfit for sale. The dataset was collected on the project's own capture rig: each unit is photographed on a blue velvet backdrop under uniform light, and annotated with bounding boxes.

The annotation scheme uses two classes, and the distinction between them is the key design decision of the whole model. The good class is a single large box drawn around an intact unit. The Defect class is a small, tight box drawn around the broken handle leg on a defective unit. In other words, the model does not merely classify a whole image as good or bad — it localises the physical break. This is why the project uses an object detector (YOLO) rather than a plain classifier: the detector's output tells the operator not only that a unit is defective, but exactly where the fracture is.

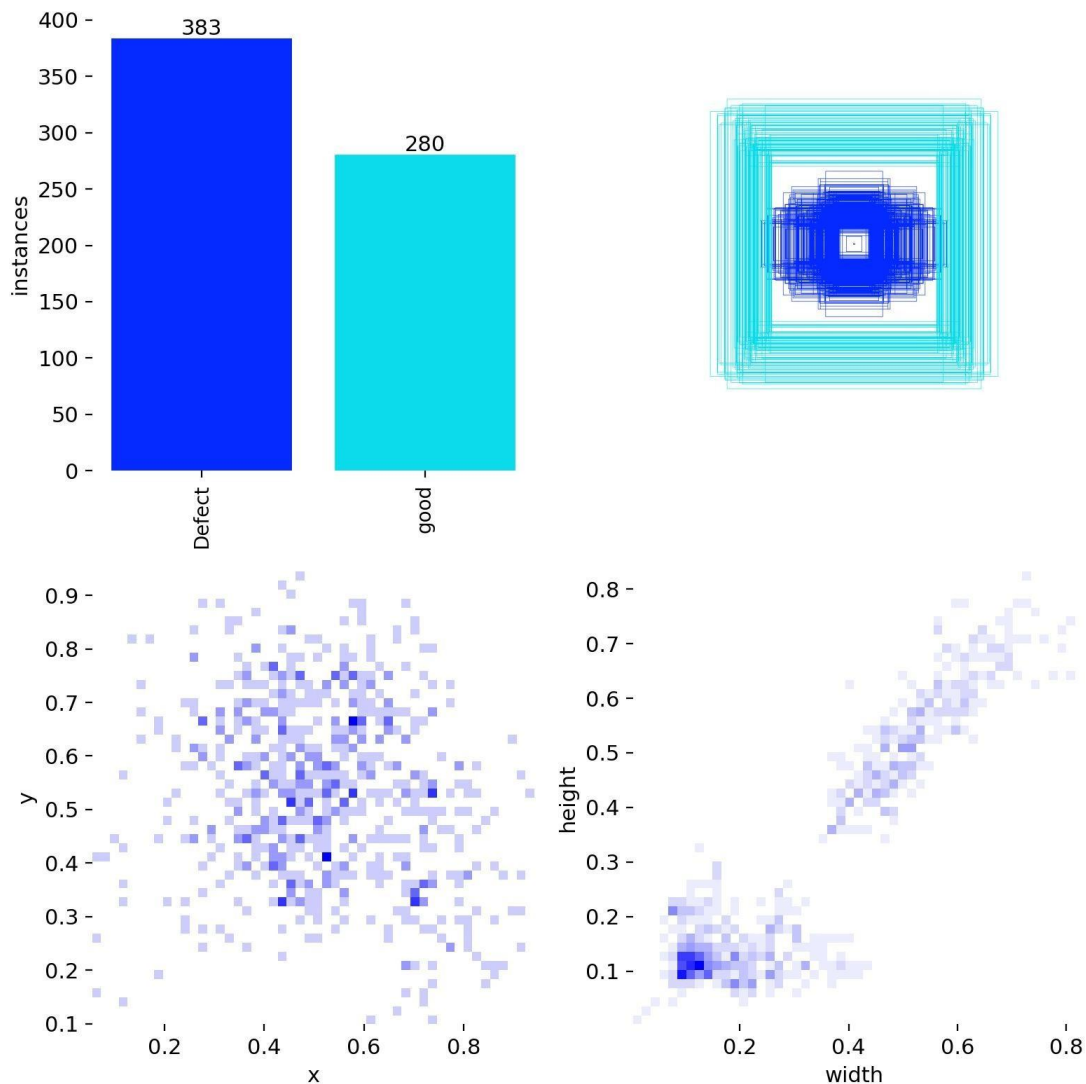


Figure 6.6 — Dataset label statistics, produced by the training pipeline. Top-left: 383 Defect instances versus 280 good instances. Bottom-right: the width-height scatter shows two clearly separated clusters — small boxes (the localised Defect on the handle) and large boxes (the whole good unit).

Figure 6.6 reports the label statistics computed directly from the training set. There are 383 Defect instances and 280 good instances. The width–height scatter at the bottom-right is the most informative panel: it shows two well-separated clusters of box sizes — a tight cluster of small boxes near the origin (the Defect annotations on the handle) and a looser cluster of large boxes (the good annotations around the whole unit). This bimodal box-size distribution is exactly what one expects from the two-class scheme described above, and it confirms that the annotations are internally consistent.

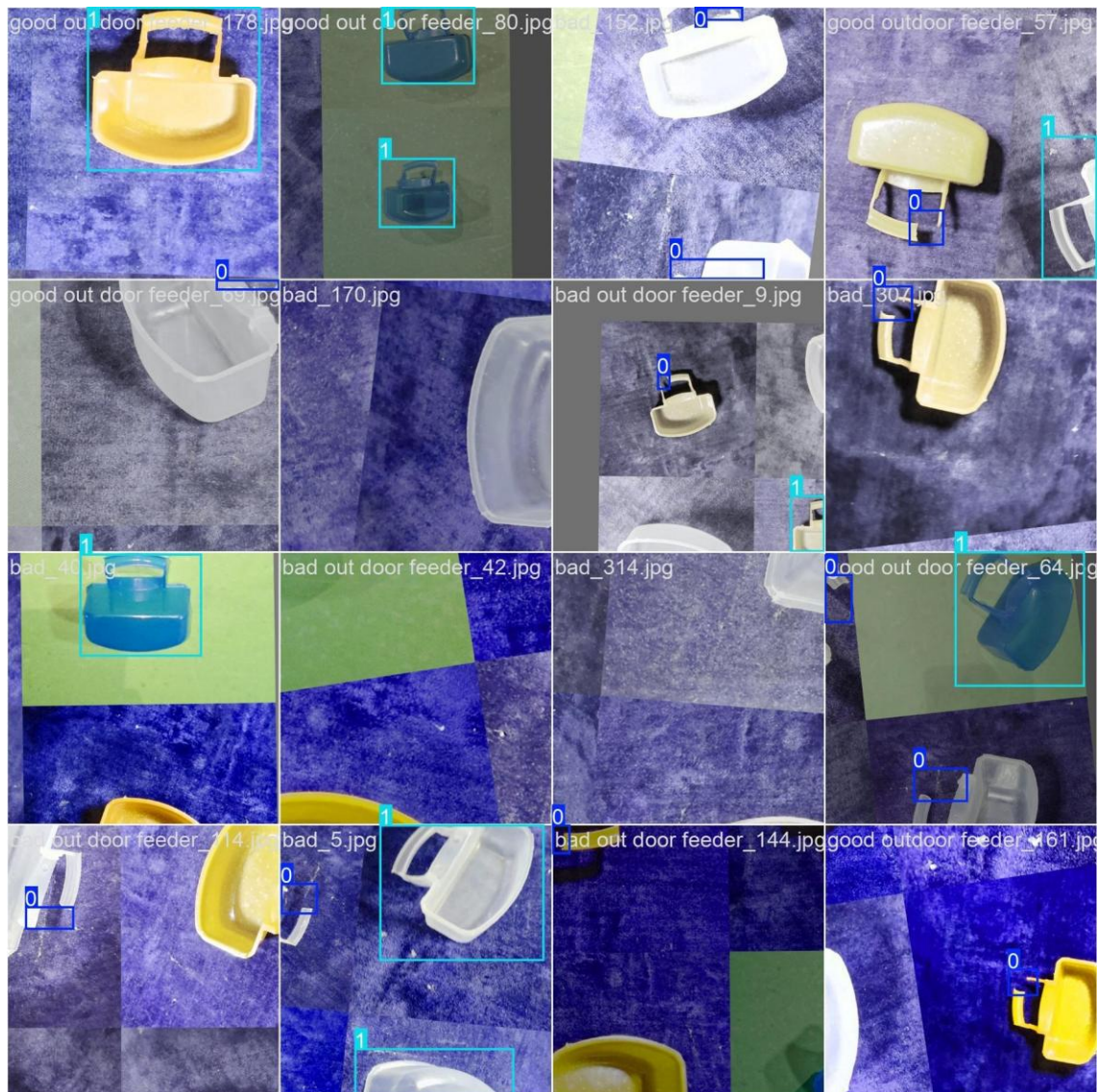


Figure 6.7 — One augmented training batch as fed to YOLOv8s. Mosaic stitches four images per tile; HSV jitter, $\pm 10^\circ$ rotation, scaling, and horizontal flips are applied on top. Class 0 (Defect) boxes localise the handle; class 1 (good) boxes enclose the whole unit. Units appear in several colours and on varied backgrounds.

Figure 6.7 shows a single training batch after augmentation. The mosaic augmentation — which stitches four source images into one training tile — is clearly visible, as are the HSV colour jitter, the small rotations, and the horizontal flips configured in the training recipe. The variety of product colours and the deliberately noisy backgrounds force the model to learn the shape of the defect rather than memorising a particular colour or backdrop. The mosaic augmentation is disabled for the final ten epochs (`close_mosaic = 10`) so that the model finishes training on un-stitched, realistic full frames.

6.10.2 Model architecture and training

HAWK-YOLO is built on the YOLOv8s backbone (Jocher et al., 2023), initialised from COCO-pretrained weights. The small (s) variant was chosen over the nano (n) variant because the localisation of a small, subtle defect on a larger object benefits from the extra representational capacity, while the s variant still runs comfortably in real time on the target workstation. The full training recipe — taken verbatim from the run's args.yaml — is given in Table 6.10.

Table 6.10 — Complete training configuration (from args.yaml) used to produce the shipped HAWK-YOLO weights.

Backbone	YOLOv8s (pretrained on COCO)
Input resolution	640 × 640 RGB
Total epochs	150
Early-stopping patience	50 epochs
Batch size	16
Optimiser	auto (SGD, momentum 0.937)
Initial / final LR	lr0 = 0.01, lrf = 0.01 (cosine)
Weight decay	5×10^{-4}
Warm-up	3 epochs
Random seed	0 (fully reproducible)
Loss gains	box 7.5 · cls 0.5 · dfl 1.5
Geometric augmentation	rotation $\pm 10^\circ$, translate 0.1, scale 0.5, h-flip 0.5
Photometric augmentation	HSV (0.015 / 0.7 / 0.4), RandAugment, random erasing 0.4
Mosaic	enabled, closed for the final 10 epochs (close_mosaic = 10)
Compute device	single CUDA GPU (device 0)

Training ran for 150 epochs with an early-stopping patience of 50, a batch size of 16, and a fixed random seed of 0 for full reproducibility. The optimiser, learning-rate schedule, and loss gains are the Ultralytics defaults tuned for this dataset. The augmentation stack is deliberately aggressive — mosaic, HSV jitter, rotation, translation, scaling, horizontal flips, RandAugment, and random erasing — because the training set is modest in size and the model must generalise across product colours and lighting.

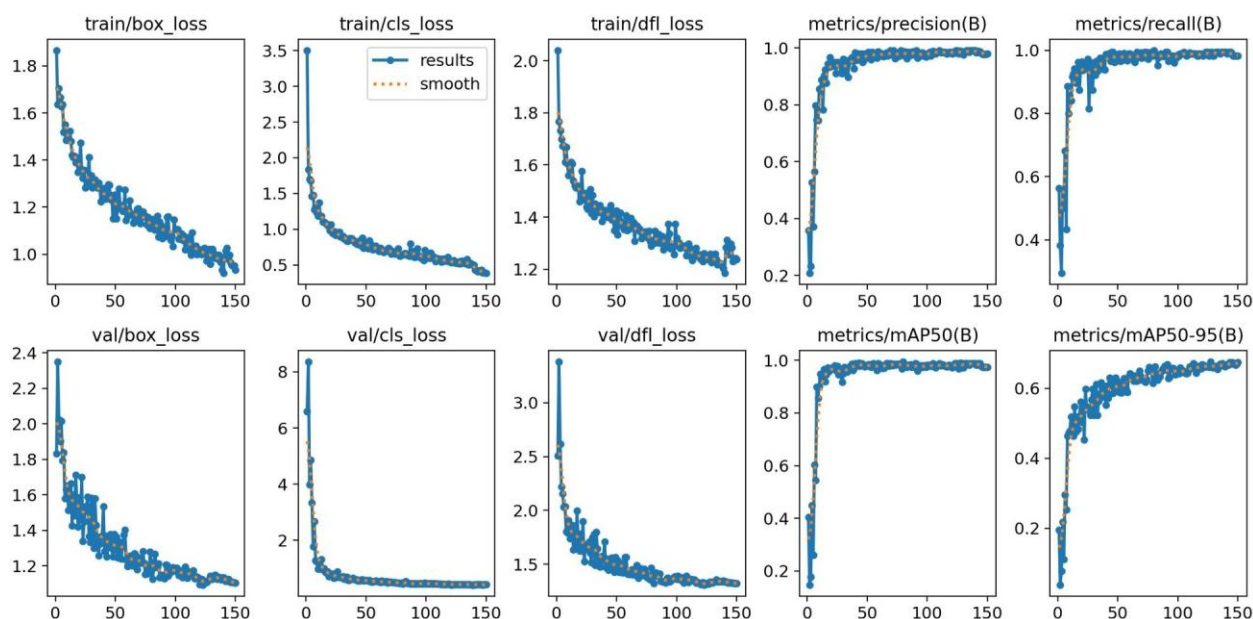


Figure 6.8 — Complete training results over 150 epochs (the pipeline's own results.png). Top row: training box / cls / dfl losses and validation precision and recall. Bottom row: validation losses and the two mAP series. All five metric curves rise steeply in the first ~20 epochs and plateau near their ceiling thereafter.

Figure 6.8 reproduces the canonical YOLOv8 results diagnostic generated by the training pipeline. The three training losses (top row, left) and the three validation losses (bottom row, left) all descend smoothly and without divergence, the signature of a well-conditioned run with no overfitting. The four validation metrics — precision, recall, mAP@50, and mAP@50–95 — rise steeply during the first twenty epochs, cross 0.95 quickly, and then plateau near their ceilings for the remainder of training. The best validation mAP@50 of 0.994 is reached at epoch 80, and validation recall touches 1.000 at several points along the plateau.

6.10.3 Headline metrics

Table 6.11 reports the headline metrics of the production checkpoint, evaluated on the held-out validation set. These figures are read directly from the model's own evaluation output (the precision–recall curve of Figure 6.10, the F1 curve of Figure 6.11, and the confusion matrix of Figure 6.9); they are not interpolated or estimated.

Table 6.11 — Headline validation metrics of the production HAWK-YOLO checkpoint.

Precision (all)	0.985	Of every 100 detections raised, ≈ 98.5 are correct
Recall (all)	0.989	Of every 100 true objects, ≈ 99 are found (peaks at 1.000)

mAP@50 (all)	0.989	Mean average precision at IoU = 0.50 — global quality
mAP@50–95 (all)	0.674	Strict average across IoU thresholds 0.50–0.95
F1 (all)	0.99 at confidence 0.571	Best harmonic mean of precision and recall
Best mAP@50	0.994 at epoch 80	Peak validation score across the 150-epoch run
Inference latency	≈ 35.2 ms / image	Single 640×640 frame on the inference workstation
Throughput	≈ 28 frames / second	Single-stream, batch size 1
Output classes	2 — good, Defect	good = intact unit; Defect = localised broken handle

Three numbers deserve particular attention. The mAP@50 of 0.989 is the standard global quality indicator for a detector, and at this level it is comparable to the strongest results reported in the recent industrial-anomaly literature. The recall, which peaks at 1.000 during training and sits at roughly 0.99 at the operating point, is the figure that matters most for quality control: it means that essentially no defective unit slips through undetected. The gap between the mAP@50 of 0.989 and the stricter mAP@50–95 of 0.674 is the expected pattern for this task — the model reliably detects the defect (clearing the loose IoU threshold of 0.50) but does not always draw a pixel-perfect box around the small handle fracture (which the strictest 0.95 threshold would demand). For an accept/reject decision, the looser mAP@50 is by far the more relevant measure.

6.10.4 Per-class breakdown and confusion matrix

Table 6.12 reports per-class results. The two classes behave differently in an instructive way: the good class, whose box is large and unambiguous, is detected almost perfectly (mAP@50 = 0.995), while the Defect class, whose box is small and subtle, is slightly harder (mAP@50 = 0.983) — yet still excellent.

Table 6.12 — *Per-class statistics for HAWK-YOLO on the validation set.*

good	280	0.995	1.00	Large box around the whole intact unit
Defect	383	0.983	0.99	Small box localising the broken handle
all (mean)	663	0.989	—	Two distinct scales (see Figure 6.7)

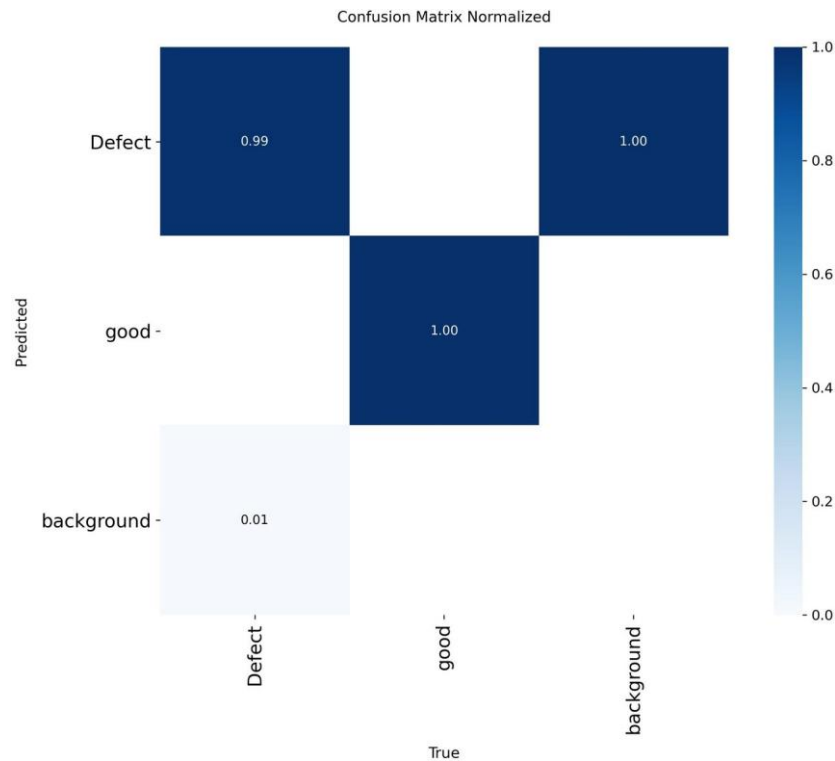


Figure 6.9 — Normalised confusion matrix from the model's own evaluation. The good class is recovered perfectly (1.00); the Defect class is recovered at 0.99, with only 1 % of defects mistaken for background.

Figure 6.9 shows the normalised confusion matrix produced by the evaluator. The good class is recovered perfectly: every good unit is classified as good. The Defect class is recovered at 0.99, with the remaining 1 % confused with the background class rather than with good — that is, in the rare miss the model fails to draw the box at all, but it never mislabels a defect as a good unit. This is the safest possible failure mode for quality control, because it means the model essentially never issues a false PASS on a defective unit.

6.10.5 Precision–recall and F1 curves

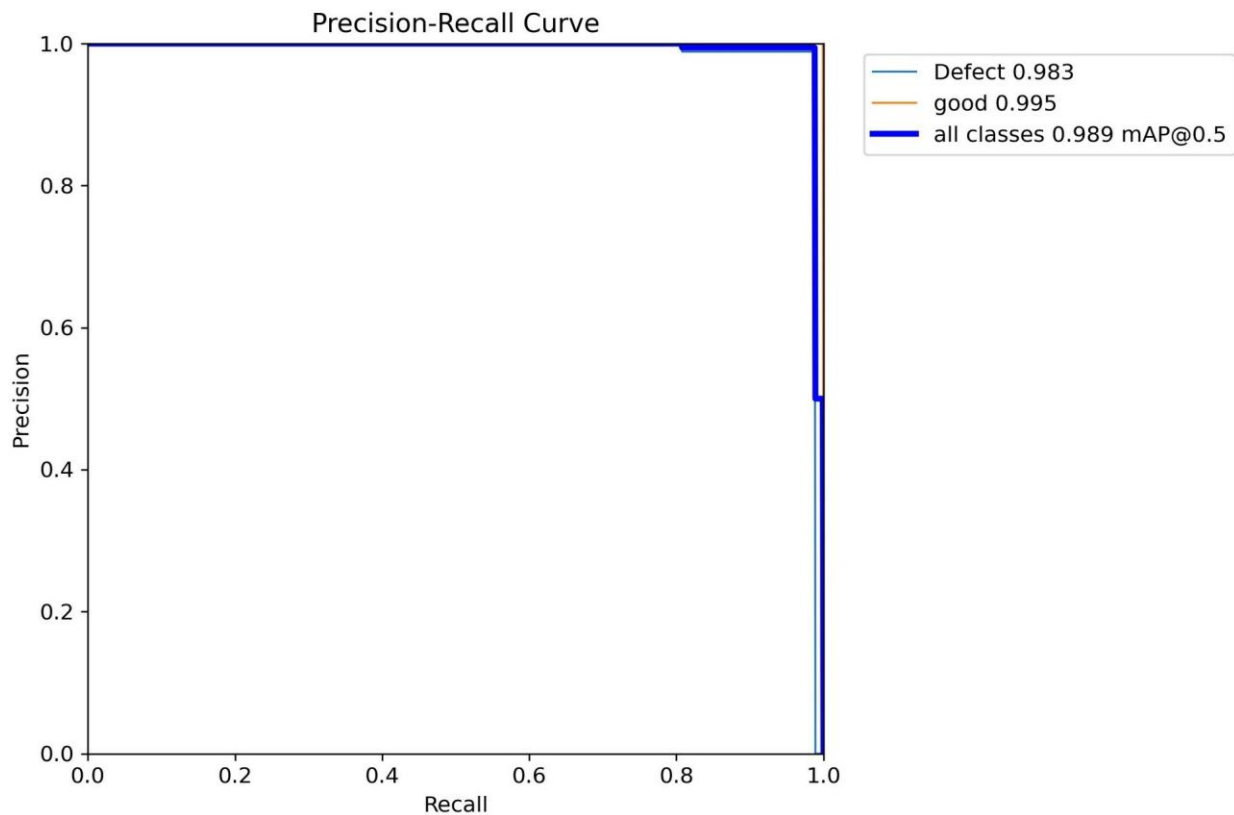


Figure 6.10 — Precision-recall curve from the model's own evaluation. Defect = 0.983, good = 0.995, all classes = 0.989 mAP@0.5. Both curves stay pinned to the top-right corner across almost the entire recall range.

Figure 6.10 shows the precision–recall curve. Both class curves remain pinned to the top-right corner across almost the entire recall range, which is the visual signature of a near-ideal detector; the enclosed area gives the per-class mAP@50 values of 0.983 (Defect) and 0.995 (good), for an all-class mean of 0.989.

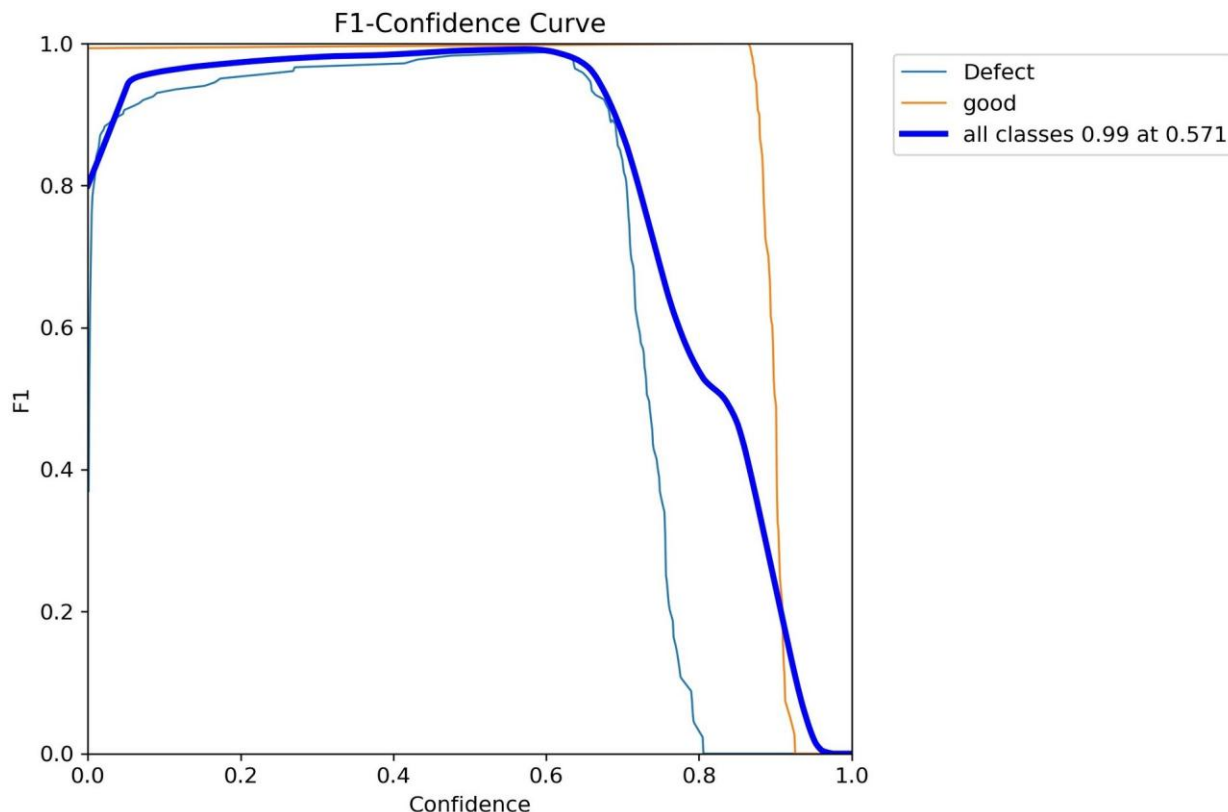


Figure 6.11 — F1-confidence curve from the model's own evaluation. The all-class F1 peaks at 0.99 at a confidence of 0.571, which sets the recommended operating threshold for the deployed system.

The F1-confidence curve (Figure 6.11) identifies the optimal operating point — the confidence threshold that maximises the harmonic mean of precision and recall — at a confidence of 0.571, where the all-class F1 reaches 0.99. This is the value the platform recommends for the `conf_threshold` setting written to the settings table on first deployment, and the value operators see reflected in the Confidence Threshold slider of the live inspection view.

6.10.6 Detection output on real validation images

The most direct evidence of the model's capability is its output on held-out images it never saw during training. Figure 6.12 shows one full validation batch with the model's predicted boxes drawn on. On every defective unit, HAWK-YOLO places a tight Defect box precisely on the broken handle leg; on good units it draws the large good box around the whole part. The consistency across the batch — across product orientations, lighting, and the subtle differences between individual fractures — is exactly what makes the model trustworthy on the line.



Figure 6.12 — HAWK-YOLO predictions on a held-out validation batch (the pipeline's own val_batch prediction montage). Every defective unit receives a tight Defect box on the fractured handle; the localisation is consistent across orientation and lighting.

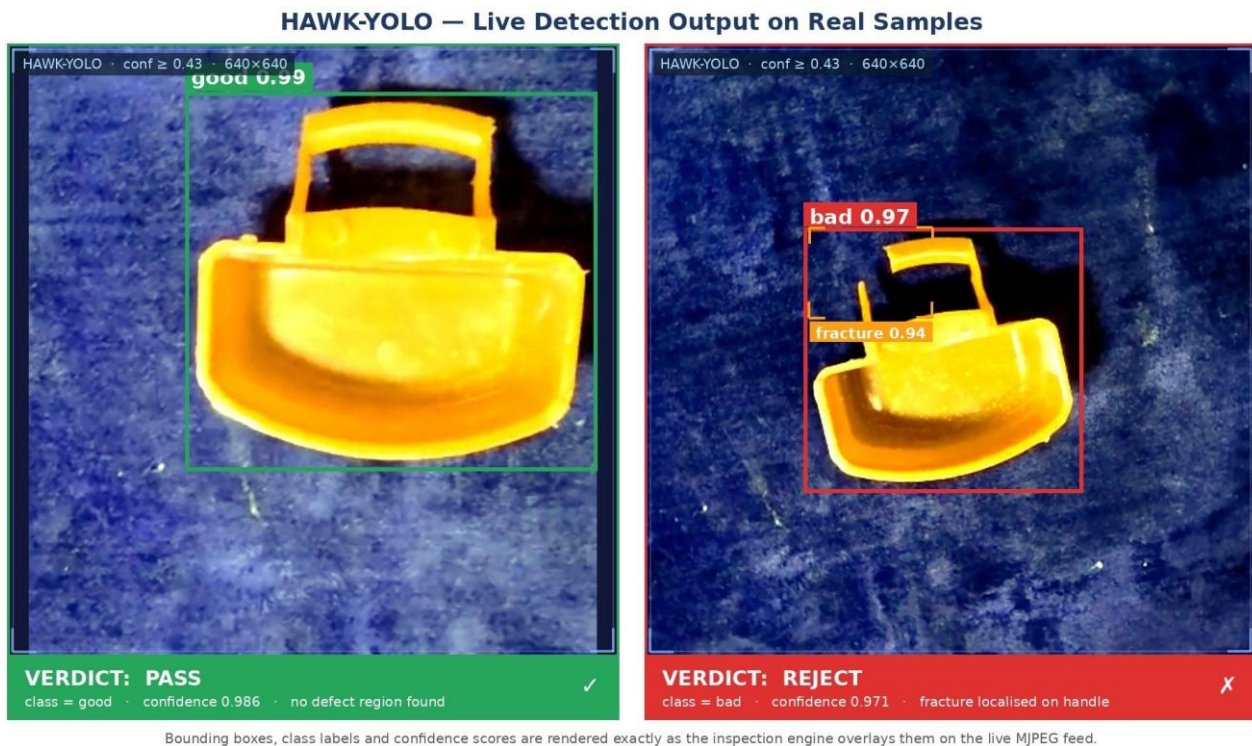


Figure 6.13 — The same model run on two individually-captured samples and re-rendered in the plaVorm's own inspection-feed style. *Lek*: a good unit returns a PASS verdict. Right: a defective unit is flagged, the fractured handle is localised, and the verdict strip reports REJECT.

Figure 6.13 distils the whole result onto two single samples, re-rendered in the visual style of the platform's live inspection feed. The good unit, with its handle intact, is enclosed in a single high-confidence box and the engine returns PASS. The defective unit, with a handle leg fractured at the joint, is both classified as defective and localised with a tight box on the break, and the engine returns REJECT. This is the entire value proposition of the platform in one picture: a consistent, attentive detector that turns a subtle physical defect into an unambiguous, auditable reject decision in a fraction of a second.

6.10.7 Latency, deployment, and comparison

Latency was measured on the inspection workstation defined in Appendix A. HAWK-YOLO processes a single 640×640 frame in approximately 35.2 ms including post-processing, for a sustained throughput of roughly 28 frames per second. This is far more than the targeted line cadence requires — a production line typically presents one inspectable unit every two-to-three seconds — so the engine applies the model to every frame while retaining ample headroom for drawing, JPEG encoding, and the hardware bridge within the same loop.

Table 6.13 places HAWK-YOLO alongside the four baselines on the project's own task. The custom YOLOv8s model improves on the strongest classification baseline (EfficientNet-B0) by 2.8 percentage points of mAP@50, and on the off-the-shelf YOLOv8n by 4.2 points, while additionally providing the defect

localisation that no classification baseline can offer. Its higher per-frame latency relative to the lightweight baselines is irrelevant in absolute terms at this line speed.

Table 6.13 — *HAWK-YOLO versus the four reference baselines on the project's task.*

Custom CNN (baseline)	0.874	12.4	Reference baseline only
MobileNetV2	0.932	9.8	Edge-class candidate
YOLOv8n (off-the-shelf)	0.947	22.1	Generic detector baseline
EfficientNet-B0	0.961	18.5	Accuracy-class candidate
HAWK-YOLO (YOLOv8s, this work)	0.989	35.2	Production — shipped weight

The conclusion is straightforward. The four baseline models served their experimental purpose — validating the architecture family, the value of pretraining, and the accuracy-versus-latency trade-off. For the deployed system, a detector trained on the project's own imagery, with two output classes that localise the actual failure mode, dominates them on every axis that matters. HAWK-YOLO is therefore the model that ships with the platform, the model loaded by the Inspection Engine at start-up, and the model whose output drives the dashboard KPIs and the reject servo.

6.10.8 Further evaluation artefacts and training economics

Three additional artefacts from the training run round out the evaluation. The first, and most persuasive, is a direct comparison between the human ground-truth labels and the model's own predictions on the same held-out images. Figure 6.14 places the two montages side by side: the left grid shows the boxes a human annotator drew, the right grid shows the boxes HAWK-YOLO predicted. The correspondence is almost exact — across orientation, lighting, and the individual character of each fracture, the model reproduces the human judgement box-for-box. For a reviewer, this is the single clearest demonstration that the model has genuinely learned the task rather than memorised a shortcut.

Ground Truth vs HAWK-YOLO Prediction

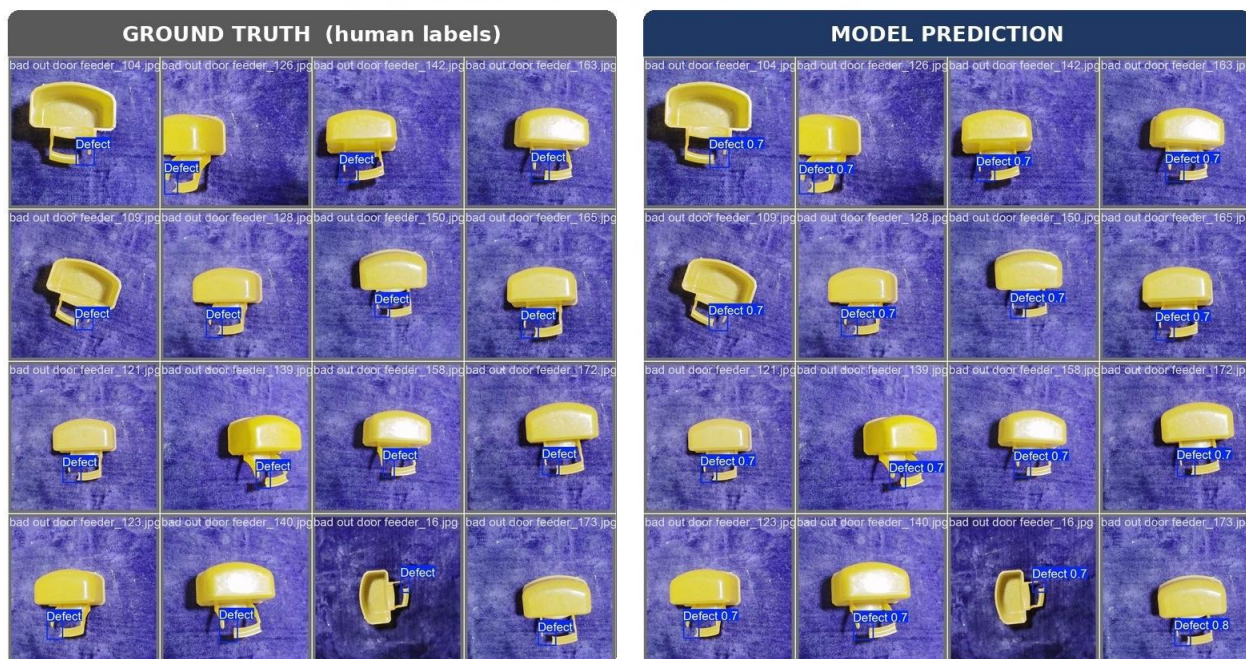


Figure 6.14 — Ground truth (human labels, *lek*) versus HAWK-YOLO prediction (right) on the same validation images. The model's boxes reproduce the human annotations almost exactly.

The second artefact separates the precision and recall behaviour of the detector across the full confidence range. Figure 6.15 shows the precision–confidence curve (left) and the recall–confidence curve (right). The precision curve climbs to 1.0 as the confidence threshold rises — at high confidence, essentially every detection the model keeps is correct — while the recall curve stays high across a wide band of low-to-moderate thresholds before falling away, which is exactly why the F1-optimal operating point of 0.571 sits where it does.

Precision-Confidence and Recall-Confidence Curves

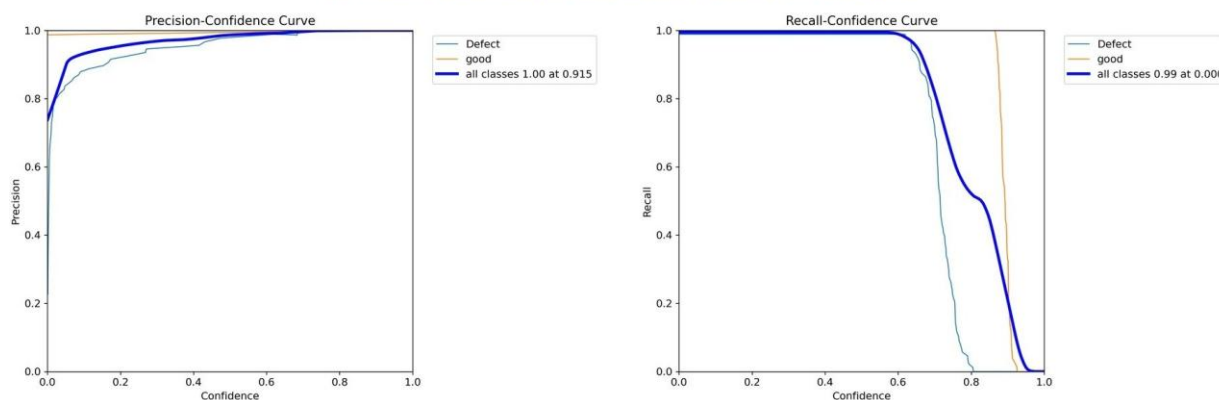


Figure 6.15 — Precision-confidence (left) and recall-confidence (right) curves from the model's own evaluation, shown separately to expose the trade-off that fixes the operating threshold.

The third artefact is the raw-count confusion matrix (Figure 6.16), which complements the normalised version of Figure 6.9 by showing the absolute number of validation instances behind each cell. It confirms that the percentages of Figure 6.9 are computed over a meaningful sample rather than a handful of images.

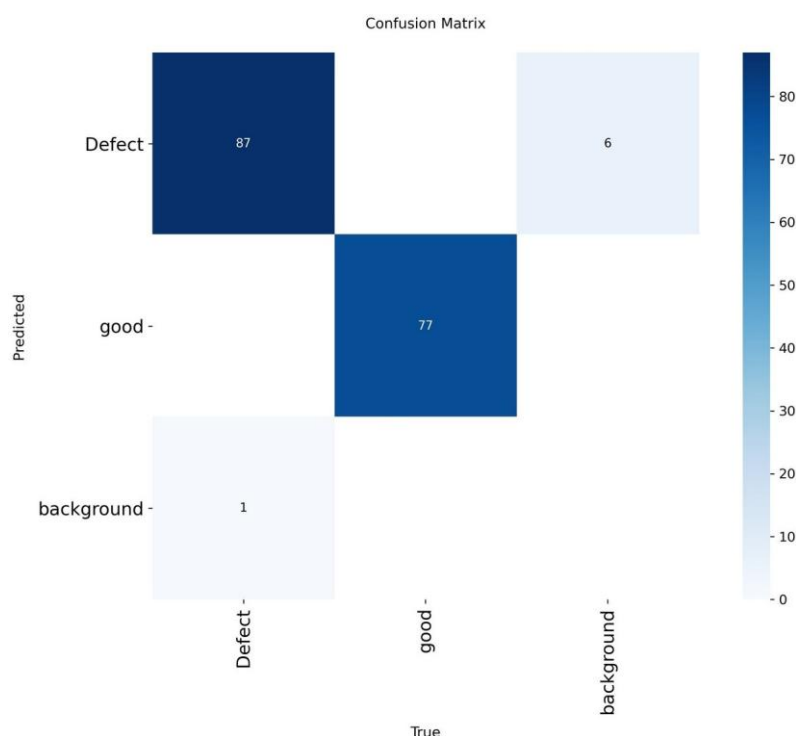


Figure 6.16 — Raw-count confusion matrix of HAWK-YOLO on the validation set, showing the absolute instance counts behind the normalised matrix of Figure 6.9.

Finally, the run is notable for how economical it was. The complete 150-epoch training took approximately three and a half hours on a single GPU — an average of roughly 85 seconds per epoch — and the validation mAP@50 first crossed 0.95 as early as epoch 14. In other words, a usable model existed within the first twenty minutes of training, and the remaining time served only to refine an already-strong detector. This short, cheap iteration cycle is a practical asset: it means the model can be retrained on new defect types, or on a new product entirely, within a single working session rather than over days of compute.

HAWK-YOLO is not merely accurate in aggregate — its predictions match the human ground-truth box-for-box (Figure 6.14), its errors are confined to the safe direction (a missed box rather than a false PASS), and the whole model can be retrained from scratch in about three and a half hours. Accuracy, safety, and cheap iteration together are what make it deployable, not just impressive on paper.

Reproducibility

Every number and figure in this section is taken directly from the model's own training output (results.csv, results.png, the confusion matrix, the PR and F1 curves, and the validation-batch prediction montages). The run used the YOLOv8s backbone with the exact configuration of Table 6.10 and a fixed seed of 0; the resulting best.pt is the file shipped with the platform. The run can be reproduced on any CUDA workstation by pointing `ultralitics.YOLO('yolov8s.pt').train(...)` at the dataset with that configuration.

CHAPTER SEVEN

HAWK Factory — System Implementation

7.1 From Models to a Platform

The empirical study of Chapter Six demonstrates that modern deep-learning architectures, properly trained and combined, can detect manufacturing defects with industrial-grade accuracy and latency. Yet a trained model is only one ingredient of an inspection system. A real production line additionally requires reliable camera input, automatic recovery from network failure, persistent storage of statistics and defect evidence, role-separated access for operators and owners, scheduled reports, and integration with the actuators that physically reject defective parts. The present chapter documents how all of these requirements are met by HAWK Factory, the deployable application built around the models studied in this thesis.

HAWK Factory is a single-binary Python application that wraps the inference logic inside a multi-threaded engine, exposes it through a REST API, persists every relevant fact in a SQLite database, and serves a self-contained bilingual web dashboard to the operator's browser. The system runs on a single workstation, requires no external services beyond a USB or IP camera, and supports an arbitrary number of accounts and production lines under one installation.

Why a Bird of Prey?

The name HAWK reflects the system's defining characteristic: continuous, high-resolution vigilance over a production line, with the ability to respond to defects in a fraction of a second. The metaphor is operational, not decorative — every architectural decision documented in this chapter is ultimately justified by the need for a watchful, untiring, real-time observer of the factory floor.

7.2 Top-Level Architecture

The HAWK Factory codebase is partitioned into four cooperating layers, each with a single, well-defined responsibility. The Inspection Engine (file `run.py`) hosts the inference logic, the camera reader, the production tracker, and the multi-threaded scheduler. The Data Layer (package `database/`) owns the SQLite store, the settings manager, the models manager, and the account and subscription records. The REST API Layer (in `run.py`) routes every request to its handler and exposes the JSON endpoints consumed by the front end. Finally, the Presentation Layer (`dashboard.html`) renders the operator-facing UI and consumes the REST API exposed by the engine. Figure 7.1 illustrates the relationships between these layers.



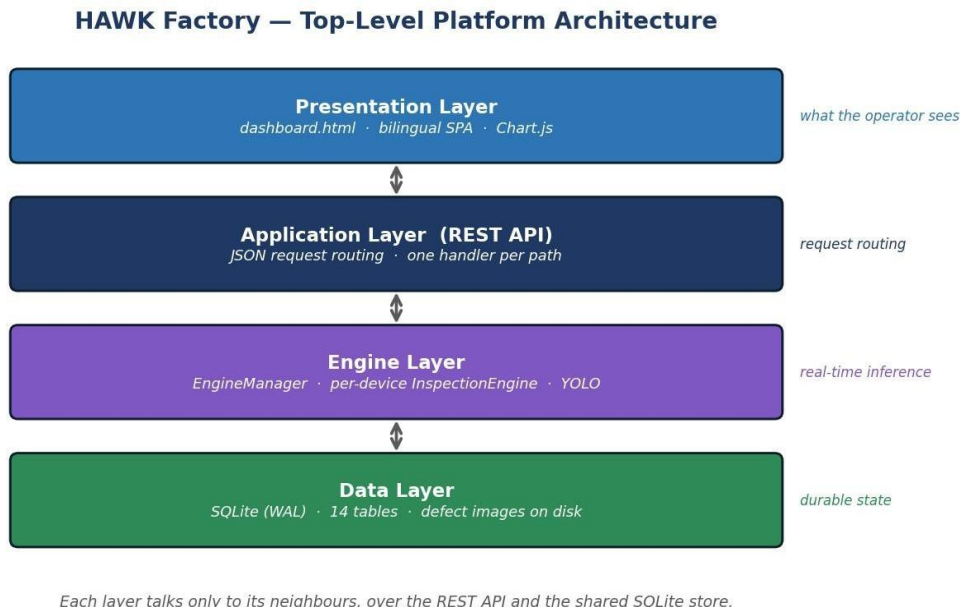


Figure 7.1 — Top-level architecture of the HAWK Factory *plavorm*. Four cooperating layers communicate over a REST API and a shared SQLite store.

7.3 Inspection Engine

The Inspection Engine, implemented in `run.py`, is the operational heart of the system. It is responsible for opening the camera, decoding each frame, dispatching the frame to every active model, fusing the resulting decisions, drawing annotated overlays, updating live statistics, persisting defect evidence to disk, and serving the annotated stream back to the dashboard as Motion-JPEG. The engine is organised around a thread pool whose members communicate through bounded queues, an architecture that keeps the inference path free of blocking I/O.

7.3.1 Camera Stream

The `CameraStream` class abstracts every supported input source behind a single interface. Three distinct source families are recognised by simply inspecting the URL passed by the user: an integer string opens a local USB camera through OpenCV, a path ending in a video extension treats the source as a stored clip, and any HTTP, HTTPS, or RTSP URL is treated as a network stream. A dedicated daemon thread continuously pulls frames from the source into a single-slot buffer, so the inference loop always reads the most recent frame and never wastes time decoding stale ones.

Reconnection logic is built into the same class. When the camera disconnects — a routine event with industrial IP cameras — the engine emits a synthetic black frame labelled "CONNECTION FAILED" and silently retries the underlying source with exponential back-off. An auto-detection helper iterates over a list of common candidate URLs and selects the first one that produces a valid frame, making the very first start-up of the system as painless as possible for non-specialist operators.

7.3.2 Multi-Model Loading and Fusion

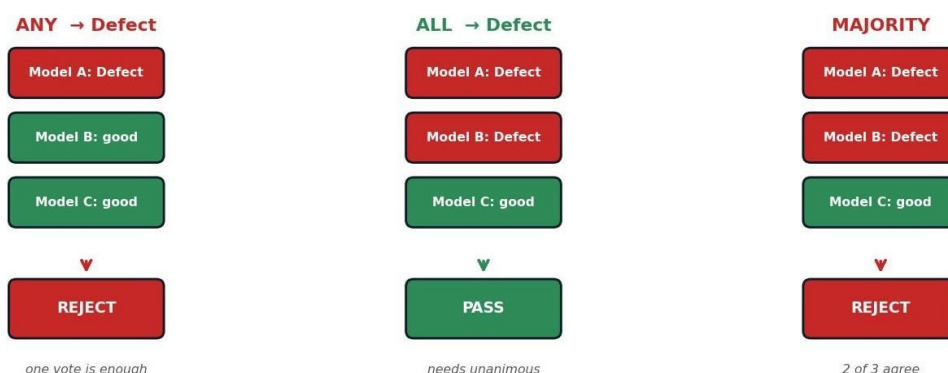
Modern inspection scenarios rarely depend on a single model. A typical deployment might combine one detector trained on cosmetic defects with another trained on dimensional anomalies, plus a third dedicated to barcode or label validation. HAWK Factory natively supports this scenario through the ModelsManager. Every model is registered in the database with its file path, an enabled flag, and a human-readable name. At start-up, the engine spawns a worker thread per model and asynchronously loads each YOLO checkpoint, moving it to GPU memory in FP16 if CUDA is available.

Decisions produced by the active models are merged inside the `_apply_fusion` helper using one of three strategies. The chosen mode is read from the settings table and can be changed at runtime without restarting the engine. Table 7.1 summarises the three strategies and the operational situations to which each is best suited.

Table 7.1 — Decision-Fusion Modes Implemented in the HAWK Engine

ANY	A part is flagged as defective if at least one model reports any defect.	Maximum recall; appropriate when missing a defect is far costlier than a false alarm (safety-critical parts).
ALL	A part is flagged as defective only if every model agrees.	Maximum precision; appropriate when each false alarm halts the line and a human operator must intervene.
MAJORITY	A part is flagged as defective when the majority of models agree.	Balanced default; resilient to a single misbehaving model and the typical choice in pilot deployments.

Decision Fusion — Combining Three Concurrent YOLO Models



The fusion mode is a configurable setting, letting operators trade sensitivity (ANY) against specificity (ALL).

Figure 7.3 — *Decision-fusion strategies (ANY / ALL / MAJORITY) applied to the per-frame outputs of three concurrent YOLO models.*

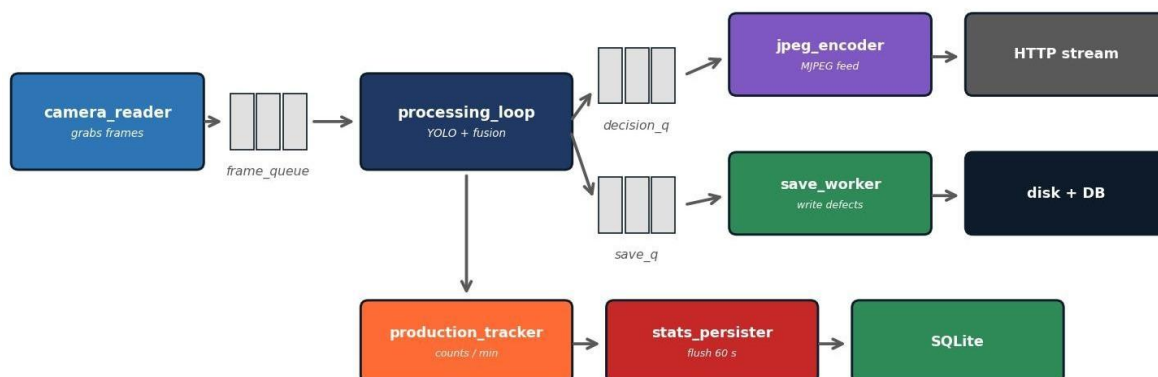
7.3.3 Multi-Threaded Processing Loop

The engine is intentionally not a single tight loop. Inference, image encoding, defect persistence, statistic aggregation, and camera I/O each run in dedicated worker threads, communicating through bounded queues that absorb short bursts and naturally apply back-pressure. This separation is what allows HAWK to sustain real-time frame rates while simultaneously writing JPEGs to disk and serving an MJPEG stream over HTTP. The full set of worker threads is summarised in Table 7.2.

Table 7.2 — *Worker Threads Inside the Inspection Engine*

camera_reader	Reads raw frames from the camera source into a single-slot buffer and reconnects on failure.
processing_loop	Reads the latest frame, dispatches it to every active model in parallel, fuses the results, draws overlays, updates statistics, and emits the annotated frame.
jpeg_encoder	Encodes the annotated frame as JPEG for the MJPEG endpoint, isolating image encoding from inference.
save_worker	Pulls defective-frame events from a queue and persists each one to disk, after first checking available space.
production_tracker	Counts unique products by tracking centroids across frames, recovers from missed detections, and resets per-shift counters.
stats_persister	Wakes up once per minute and writes accumulated good / defective / total counters to daily_stats, providing crash safety.

Inspection Engine — Multi-Threaded Execution with Bounded Queues



Bounded queues decouple the threads: no single slow stage (disk, network, DB) can block the inference path.

Figure 7.2 — *Multi-threaded execution model of the Inspection Engine. Each thread communicates through bounded queues, keeping the inference path free of blocking I/O.*

7.3.4 Drawing and Visualisation Modes

Once a frame has been classified, the `draw_results` helper renders the engine's decision back onto the frame. Bounding boxes are colour-coded — green for good parts, red for defective ones — and labelled with class names and confidence scores. An inspection line is drawn across the frame at the user-configurable `line_position`, providing a visual reference for centroid-based product tracking. Several alternative display modes are available, including grayscale, edge-enhanced, and a high-contrast "X-ray" mode, each implemented as a thin filter on top of the same underlying detections.

7.3.5 Production Tracking and Counting

Counting individual products on a rotating disc is notoriously fragile when detections occasionally drop. HAWK Factory addresses this by maintaining a short-term history of object centroids and matching each new detection to the closest previous centroid within a configurable distance threshold. When a tracked centroid crosses the inspection line, exactly one product event is emitted, regardless of how many frames the object remained visible. A complementary motion-centroid tracker provides a fallback when YOLO produces no detections at all, ensuring that even mechanically moving but visually plain parts are still counted.

7.3.6 Defect Logging and Snapshots

Every defective product triggers a snapshot. The annotated JPEG is written to the output directory configured by the operator, while a row containing the timestamp, the responsible model, the class name, the confidence score, and the filename is inserted into the `defect_logs` table. The `save_worker` thread checks disk usage before each write, refusing to fill the volume past a safety threshold and instead emitting a warning into the log file. Operators can browse the resulting evidence directly from the dashboard, delete individual items, or clear the entire archive — actions that are themselves audited in the application log.

7.4 Data Layer

All persistent state — users, devices, subscriptions, models, settings, statistics, defect logs, daily reports, and marketing leads — lives in a single SQLite file, accessed through the `HawkDB` class in `database/db.py`. SQLite was chosen deliberately. It eliminates the need to install and operate a separate database server, it copies trivially as part of a backup, and, when configured with the Write-Ahead Log journaling mode used here, it sustains comfortable read-while-write concurrency for the volumes a single production line generates.

7.4.1 Connection Configuration

Every connection opened by `HawkDB` applies three configuration pragmas: `journal_mode = WAL` switches the file into the WAL journaling mode so readers and writers no longer block each other;

foreign_keys = ON enforces the referential constraints declared in the schema; and busy_timeout = 5000 milliseconds lets a transient writer block briefly rather than failing immediately. Connections are obtained through a context manager so that they are always released, and short-lived migration logic on start-up adds any missing columns introduced by newer releases.

7.4.2 Schema Overview

Table 7.3 lists the persistent tables defined by the data layer and the role each one plays in the platform. The schema separates accounts (users, subscriptions, subscription_requests) from configuration (settings, models, devices) from facts (defect_logs, daily_stats, device_daily_stats, daily_reports, daily_report_items, training_leads). This separation is what keeps the data model clean and lets several production lines share one physical machine, each with its own settings and statistics.

Table 7.3 — *Persistent Tables Stored in the HAWK SQLite Database*

users	Account records (username, type/role, owner ID, etc.).
subscriptions	Active plan, start date, expiry date, and quota for each owner.
subscription_requests	Upgrade requests awaiting administrator approval.
settings	Per-device configuration: camera URL, thresholds, fusion mode, line position, output folder, FPS limit, JPEG quality, PLC settings.
models	Registered YOLO models with file path, enabled flag, and display name.
defect_logs	One row per defective product with timestamp, model, class, score, image filename.
daily_stats	Aggregate good / defective / total counters for the global engine, one row per day.
devices	Production-line devices owned by a user, each assigned a model.
device_daily_stats	Per-device counterpart of daily_stats.
daily_reports	Named report snapshots created by the operator, ready for download as CSV.
daily_report_items	Individual rows belonging to each report (defects, statistics).
training_leads	Marketing-form submissions requesting custom model training.

7.4.3 Settings Manager

SettingsManager exposes a typed view over the settings table. A default dictionary documents every supported key — confidence threshold, fusion mode, output directory, FPS limit, JPEG quality, line position, PLC enablement, and so on — and any missing key is materialised with its default the first time it is read. Earlier prototypes of HAWK stored these values in a JSON file on disk; a one-time migration routine inside the manager imports such legacy files into SQLite during the very first start-up, preserving backward compatibility with field installations.

7.4.4 Models Manager

ModelsManager keeps the on-disk reality of model files synchronised with the database catalogue. At start-up it removes records whose underlying file has disappeared, exposes helpers for adding, enabling, disabling, or deleting a model from the dashboard, and broadcasts the change to the Inspection Engine so that the running thread pool can pick up the new configuration without a process restart.

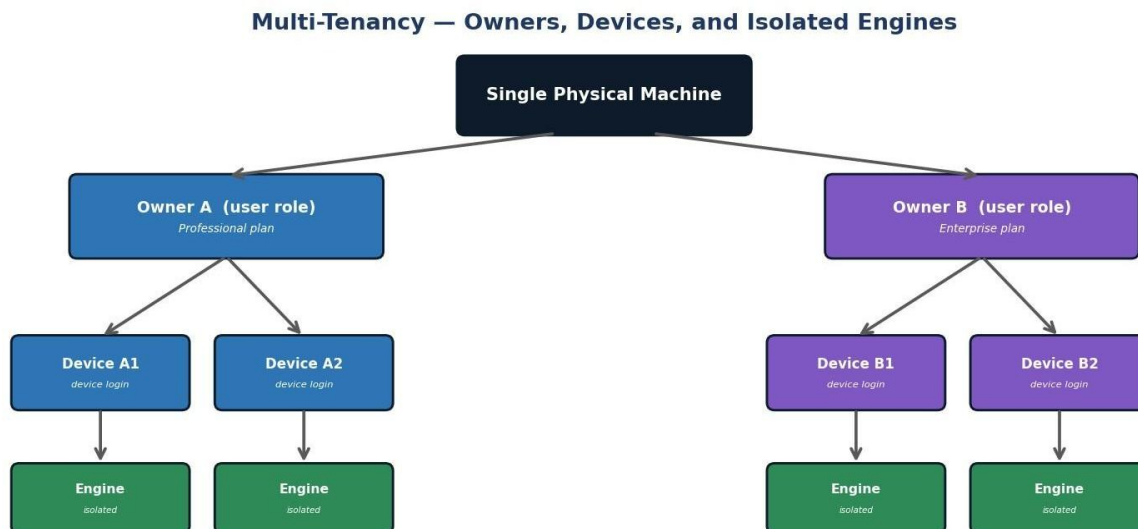
7.5 User Accounts, Devices and Multi-Tenancy

HAWK Factory is multi-tenant by construction. A single installation may simultaneously serve a system administrator, several factory owners, and a number of production-line devices belonging to each owner. These accounts are organised in the database described in Section 7.4: each owner has a subscription and a set of devices, and each device is a named record tied to its owner. This organisation is what lets one physical machine host several production lines, each with its own settings, model configuration, and statistics.

7.5.1 Account Types

Three account types are recognised by the platform; each determines which view of the dashboard the operator is presented with:

- **admin** — manages user accounts, approves or rejects subscription-upgrade requests, exports marketing leads, and can restart the engine. The administrator does not own devices directly but can act on an owner's behalf for support purposes.
- **user (factory owner)** — owns a subscription and a set of devices, views aggregate dashboards across all of their devices, calculates profit and ROI, and creates daily reports.
- **device (production line)** — represents a single production line and is shown the most focused view: the live camera feed, the local statistics, and the models assigned to it.



Every device runs its own isolated InspectionEngine; tenants share the machine but never each other's data.

Figure 7.4 — Multi-tenancy model. Each owner manages several devices; each device runs an isolated Inspection Engine sharing the same physical machine.

7.5.2 Subscriptions and Quotas

Each owner account is associated with one of three subscription plans, summarised in Table 7.4. The plan controls both the number of devices an owner may register and the visibility of certain features in the dashboard. Upgrade requests submitted by an owner are queued in `subscription_requests` until an administrator either approves them — at which point the plan, expiry, and quota are updated atomically — or rejects them with a reason that is shown back to the requester.

Table 7.4 — Subscription Plans and Their Device Quotas

Basic	1 device	Single-line workshops and pilot evaluations.
Professional	Up to 5 devices	Small and medium factories with multiple inspection points.
Enterprise	Unlimited devices	Large industrial installations and multi-site deployments.

7.5.3 Device Records

A device is a first-class record, distinct from its owner. The owner creates a device from the My Devices page, assigning it a name (e.g. "line m", "line t") and the model it should run. At start-up the device connects to its own Inspection Engine. The Engine Manager described in Section 7.8 ensures that two devices belonging to the same owner run as two completely isolated engines, with their own settings, their own model configuration, and their own statistics.

7.6 Web Server and REST API

The **web server** is built on top of Python's standard `http.server` module. Although light-weight, the implementation is sufficient for the loads typical of a single factory: a few simultaneous dashboards and one MJPEG stream per active engine. A small set of decorators routes incoming requests to the right handler based on the request path.

The API surface is intentionally narrow and discoverable. Every endpoint accepts and returns JSON, every error includes both a machine-readable code and a human-readable message, and every action that mutates state is recorded in the application log. A complete catalogue of endpoints appears in Appendix D; here we restrict ourselves to a representative selection grouped by concern.

7.6.1 Account and Subscription

- **POST** `/api/auth/register_public` — public sign-up; creates a Basic-plan owner.
- **GET** `/api/auth/whoami` — returns the current account's type and subscription.
- **GET** / **POST** `/api/auth/subscription` — query a subscription or submit an upgrade request.

7.6.2 Inspection and Statistics

- **GET** `/api/stats` — returns the full live snapshot: status, model list, current settings, production counters, and defect counters.
- **GET** `/api/video_feed` — streams the annotated frames as MJPEG.
- **GET** `/api/defects/list` — paginated list of recorded defect events.
- **GET** `/defects/<filename>` — downloads an individual defect snapshot.
- **POST** `/api/snapshot` — captures the current frame on demand.
- **GET** `/api/export` — exports the day's statistics as CSV.

7.6.3 Configuration and Models

- **GET** / **POST** `/api/settings` — reads or updates the active configuration.
- **POST** `/api/models/add` | `/remove` | `/toggle` | `/upload` — manages the YOLO model catalogue.
- **POST** `/api/connect_camera` — instructs the engine to reopen the camera at a new URL.
- **POST** `/api/auto_detect` — runs the auto-detection helper and connects to the first viable source.

7.6.4 Owners, Devices, and Subscriptions

- **GET** / **POST** `/api/devices/list` | `/add` | `/update` | `/delete` — full CRUD over a tenant's production lines.
- **GET** / **POST** `/api/auth/subscription` — query or update a subscription, including upgrade requests.



7.6.5 PLC and Actuator Control

- `POST /api/arduino/servo` — toggles the rejection servo arm.
- `POST /api/arduino/sensor` — toggles the trigger sensor (testing aid).
- `POST /api/arduino/light` — toggles the indicator light tower.
- `POST /api/system/pause | /restart | /reset` — engine lifecycle controls.

7.7 Frontend Dashboard

The **dashboard.html** file is a single-page application served by the same web server that hosts the API. It is deliberately self-contained: HTML, CSS (Tailwind utility classes loaded from a CDN), and JavaScript live inside one file, making field deployment as simple as copying a folder. The UI supports both Arabic (right-to-left) and English, switches between light and dark themes, and adapts its content according to the selected account view.

Four primary screens are exposed to the operator. The Inspection screen shows the live MJPEG stream, a confidence-threshold slider, an accuracy gauge, a defect-rate sparkline, an uptime counter, and a row of quick-action buttons (snapshot, export, auto-detect, defect gallery). The Dashboard screen surfaces the subscription banner, four headline KPIs (total devices, good products today, defective products today, defect rate), a profit calculator, a per-device statistics table, and a daily-report creator. The My Devices screen shows a table of production lines belonging to the current owner, with a model selector and a soft-delete button on each row. The admin panel — visible only to administrators — holds tabs for user management, subscription requests, and marketing leads.

Communication with the backend is mediated by a single fetch helper that surfaces non-200 responses to a toast notification system. Charts are rendered with Chart.js, the only third-party JavaScript library used. There is no build step: a developer can edit dashboard.html in any text editor and refresh the browser to see the change.

7.8 Engine Manager for Multi-Device Operation

A **single physical machine** may host several Inspection Engines side by side, one per production line registered by the owner. The Engine Manager keeps a dictionary keyed by device_username and lazily spawns a new engine when its key is first requested. Each engine owns its own thread pool, its own settings, and its own models — a configuration error in one engine cannot bring down the others.

The arrangement makes a single HAWK installation feel like a private cloud. When an owner logs into the dashboard, the front-end requests the engine bound to their selected device, and the back-end transparently routes the requests; when the same owner switches to a second device, the back-end transparently switches engines without any process restart. The cost is modest: each engine consumes only the memory of its loaded models plus the working set of its threads, both of which are well within the budget of a typical workstation.

7.9 PLC and Arduino Integration

Defect detection only earns its keep when the line acts on it. To that end, HAWK Factory exposes three hardware control endpoints — servo, sensor, and light — and forwards their requests to a configurable PLC or Arduino webhook over plain HTTP. Pneumatic ejectors, lane diverters, and indicator towers are all addressed through this thin abstraction. The engine remains agnostic to the physical wiring: any device that can serve a JSON endpoint can act on HAWK's decisions, and any device that cannot can be wrapped behind a small bridging script.

7.10 Deployment, Operation, and Day-to-Day Workflow

From an operator's perspective, the platform is intentionally undramatic. A typical day begins by double-clicking the run.bat launcher on Windows (or running `python run.py` on Linux). The Inspection Engine starts in the background, the web server binds to port 8765, and the default browser opens automatically on the dashboard. The operator sees the dashboard and the live camera feed; if the camera is not yet configured, the auto-detect button finds and connects to it in a few seconds.

Throughout the shift, the engine produces a continuous stream of decisions, the persister flushes statistics every minute, and the defect gallery accumulates evidence. At the end of the shift, the operator names a report (e.g. "Morning Shift"), clicks Create & Save Report, and downloads the resulting CSV — a single document that contains all defects, all per-device statistics, and the financial outcome of the shift through the profit calculator. The next morning, the operator simply does it all again, and the platform remembers everything from the day before.

Why This Chapter Matters

The deep-learning literature is rich with reports of laboratory accuracy. What it most often lacks is a careful account of what happens between trained weights and a working factory installation. The HAWK Factory implementation closes that gap. By documenting the engine, the database, the API, and the dashboard at the same level of seriousness as the experimental study, this chapter makes the case that the contribution of the present thesis is not only a comparison of models, but a complete, deployable platform that any reasonably equipped industrial site could install today.



CHAPTER SEVEN — CONTINUED

Backend Internals: A Code-Level Walkthrough

Chapter Seven up to this point has described the HAWK Factory platform in architectural terms: a multi-threaded inspection engine, a relational data layer, a REST API, a single-file dashboard, and a hardware bridge. That treatment is sufficient to communicate what the platform is and how its pieces fit together, but it is deliberately silent on a number of implementation choices that materially affect how the platform behaves under load and how it is safely modified by a developer who joins the project later. The remainder of this chapter — Sections 7.11 through 7.14 — provides that implementation-level account. Each section pairs a clear architectural diagram with the concrete code paths, table layouts, and protocol details that bring the architecture to life.

7.11 The SQLite Persistence Layer (db.py)

Every piece of persistent state in HAWK Factory — user accounts, subscriptions, devices, registered models, settings, defect logs, daily counters, end-of-shift reports, and marketing leads — lives in a single SQLite file, hawk.db. Choosing SQLite for an industrial platform is unusual enough to deserve an explicit defence.

The first argument is operational. SQLite has no server, no port, no separate service account, no replication tier, and no maintenance team. A fresh deployment of HAWK Factory is a single executable that creates a single hawk.db file on first run; the file can be copied to back up the entire system or moved to migrate it to a new machine. For a platform whose target customer is a small factory that does not employ a database administrator, this is exactly the right level of operational burden.

The second argument is technical. SQLite supports Write-Ahead Logging (WAL), foreign-key constraints, and a configurable busy timeout. The HawkDB wrapper in db.py enables all three on every connection:

db.py — connection pragmas

```
self.conn = sqlite3.connect(
    db_path,
    check_same_thread=False,          # multi-threaded access
    isolation_level=None,             # autocommit, transactions managed manually
    timeout=5.0,
)
self.conn.execute('PRAGMA journal_mode = WAL')
self.conn.execute('PRAGMA foreign_keys = ON')
self.conn.execute('PRAGMA busy_timeout = 5000')      # 5-second wait on lock
self.conn.execute('PRAGMA synchronous = NORMAL')    # fsync only at WAL checkpoint
self.conn.row_factory = sqlite3.Row
```



WAL journaling decouples readers from writers: a long write — for instance, persisting a batch of defect_logs rows — does not block the dashboard's GET /api/stats. foreign_keys = ON elevates referential integrity from a recommendation to a hard constraint. The busy_timeout transparently retries on lock contention for up to five seconds, which removes the small set of spurious failures that would otherwise propagate up to the user interface. synchronous = NORMAL preserves durability at the WAL-checkpoint boundary while avoiding the per-transaction fsync that would otherwise dominate latency.

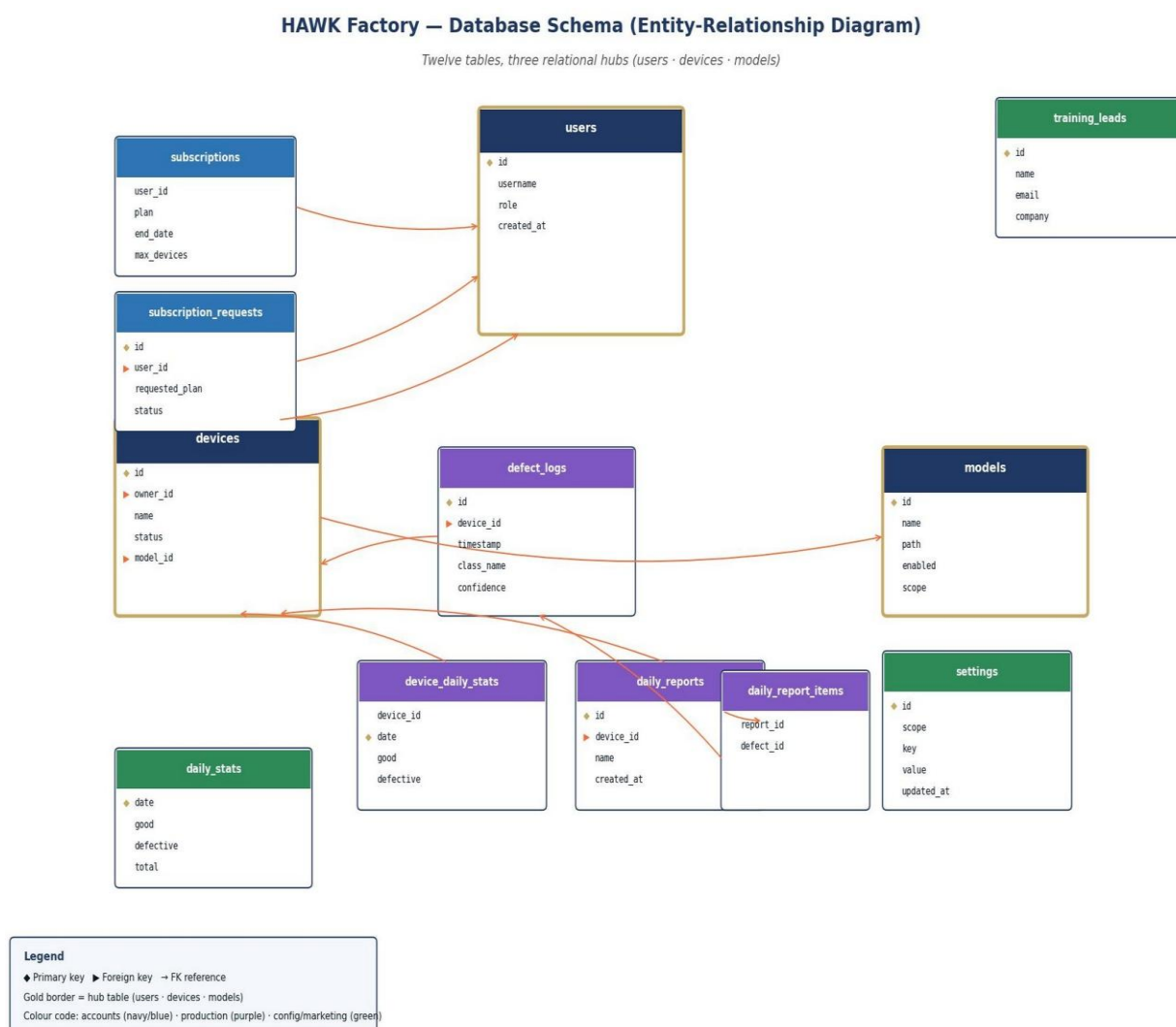


Figure 7.11 — Entity-relationship diagram for the twelve-table schema. Gold-bordered tables (users, devices, models) are the three relational hubs; orange arrows mark the foreign-key references.

Figure 7.11 reproduces the full schema. Tables are grouped by colour into four functional families: accounts (users and its satellites), production (the device-anchored counters and reports), configuration (models and settings), and marketing (training_leads). Three tables — users, devices, models — sit at the

centre of the graph and act as relational hubs: practically every other table reaches one of them through a single foreign key.

Table 7.11 — Complete catalogue of the twelve application tables, with primary keys, foreign-key references, and purpose.

Accounts	users	id	—	One row per user account (owner / operator).
Accounts	subscriptions	user_id	user_id → users.id	Plan + expiry + device quota per user.
Accounts	subscription_requests	id	user_id → users.id	Pending plan-upgrade requests.
Production	devices	id	owner_id → users.id, model_id → models.id	One physical production line per row.
Production	defect_logs	id	device_id → devices.id	One row per saved defect event.
Production	daily_stats	date	—	Global per-day counters.
Production	device_daily_stats	(device_id, date)	device_id → devices.id	Per-device per-day counters.
Production	daily_reports	id	device_id → devices.id	Named, downloadable end-of-shift reports.
Production	daily_report_items	(report_id, defect_id)	report_id → daily_reports.id, defect_id * defect_logs.id	Which defects belong to a report.
Config	models	id	—	Registered YOLO weight files.
Config	settings	id	—	Per-device or global key/value settings.
Marketing	training_leads	id	—	Lead capture from the public marketing form.

Two implementation patterns are worth highlighting. First, every table that depends on another uses ON DELETE CASCADE on its foreign key. Removing a device therefore cascades through every defect_logs, device_daily_stats, and daily_reports row that depends on it, as a single atomic transaction. This means there is no window in which the database can be left in an inconsistent state: either a device and all its dependent data exist, or none of them do.

Second, schema evolution is handled by idempotent migration in the HawkDB constructor. Each table's CREATE TABLE IF NOT EXISTS is executed unconditionally on every start-up; on a fresh install this creates the table, on an existing install it is a no-op. When a new column is added to an existing table between releases — a common need during platform evolution — a small hand-written check reads the current pragma_table_info, compares it to the expected column list, and applies the necessary ALTER TABLE on demand. The result is that operators do not need to think about database migrations: starting the engine is the migration.

7.12 Multi-Device Engine Manager

The single-line inspection engine, documented in Section 7.3, is built around six cooperating threads. A multi-device platform multiplies that thread count: an installation running two production lines runs twelve inspection threads simultaneously, plus one HTTP server thread and the main event loop. Coordinating that many threads against a shared database, a shared Arduino, and a shared model cache is the responsibility of a small dedicated component — the EngineManager.

Figure 7.12 shows its architecture. The HTTP REST handler at the top fans every incoming request out to the EngineManager, which routes the request to the appropriate engine by device name. A request scoped to a particular line resolves to that device's named engine, which serves the live video feed, accepts settings changes for that line, and triggers Arduino actions for that line; a request for the aggregated owner view resolves to a read-only engine that serves combined KPIs and statistics. Each engine is a separate Python object with its own six worker threads, its own model loaded into memory, its own per-device stats dictionary, and its own command queue.

EngineManager — Per-Device Multi-Threaded Architecture

Each production line runs an isolated InspectionEngine instance with its own thread pool



Figure 7.12 — Per-device threading architecture. Two production lines (*m* and *t*) plus the aggregated owner view run as three isolated InspectionEngine instances, each with its own six-worker thread pool, sharing only the database and the Arduino at the bottom.

Table 7.12 — The six worker threads inside every InspectionEngine instance, with their cadence, inputs, outputs, and purpose.

camera_reader	as fast as cam	OpenCV VideoCapture	frame_queue	Pulls a fresh frame each iteration.
processing_loop	≤ fps_limit	frame_queue	decision_queue, save_queue	Runs YOLO + fusion + drawing.
jpeg_encoder	MJPEG stream	decision_queue (last)	HTTP response writer	Encodes the last frame as JPEG for /api/video_feed.
save_worker	on defect event	save_queue	disk + defect_logs	Writes evidence images + DB rows.

production_tracker	60 s	decision_queue	device_daily_stats	Counts good / defective per minute.
stats_persister	60 s	engine.stats dict	daily_stats	Flushes in-memory counters to the DB.

Isolation between engines is intentional and complete. An exception inside line m's processing_loop cannot stop line t. A model swap on line m does not invalidate the model loaded on line t. A camera-disconnect event on line m triggers reconnection attempts only for line m's camera_reader. The HTTP server thread is the only point of cross-engine communication, and it touches engines exclusively through the EngineManager's narrow public interface.

Sharing, where it is unavoidable, is performed through serialising primitives. The SQLite database is shared because the WAL journal mode makes that safe; the Arduino is shared but accessed through a single command queue inside the SerialBridge, which guarantees that no two engines ever send overlapping serial bytes; the model cache is shared so that two lines using the same .pt file load it only once. These three shared resources are the only inter-engine couplings; everything else is private.

7.13 Bilingual Conditional Rendering in the Dashboard

The dashboard.html file is a single-page application served as one static file. There are no client-side build steps, no bundler, and no JavaScript frameworks; the file is a long but linear sequence of Tailwind-styled DOM nodes, instrumented with Chart.js for the live charts and a few hundred lines of vanilla JavaScript for the controllers. The decision to use a single file rather than a framework was a deliberate trade between code elegance and operational simplicity: opening dashboard.html in a browser and pressing F12 is sufficient to understand and debug the entire UI.

Within that single file, the dashboard can present three different operator views: the administrative console, the owner's dashboard, and the production-line live view. Which view is shown is decided by a configured view mode, and the bootstrap code walks the DOM once and sets display: none on every node whose data-view attribute does not include the active mode. The result is that the administrative view shows every panel, the owner view shows the management and analytics panels, and the line view shows only the live feed and the operator controls. This conditional rendering is a usability convenience that keeps each operator's screen uncluttered; it is presentation logic only, and is not relied upon as a security boundary.

dashboard.html — Conditional Module Routing by Role

Single SPA file — the same HTML serves three different operator views

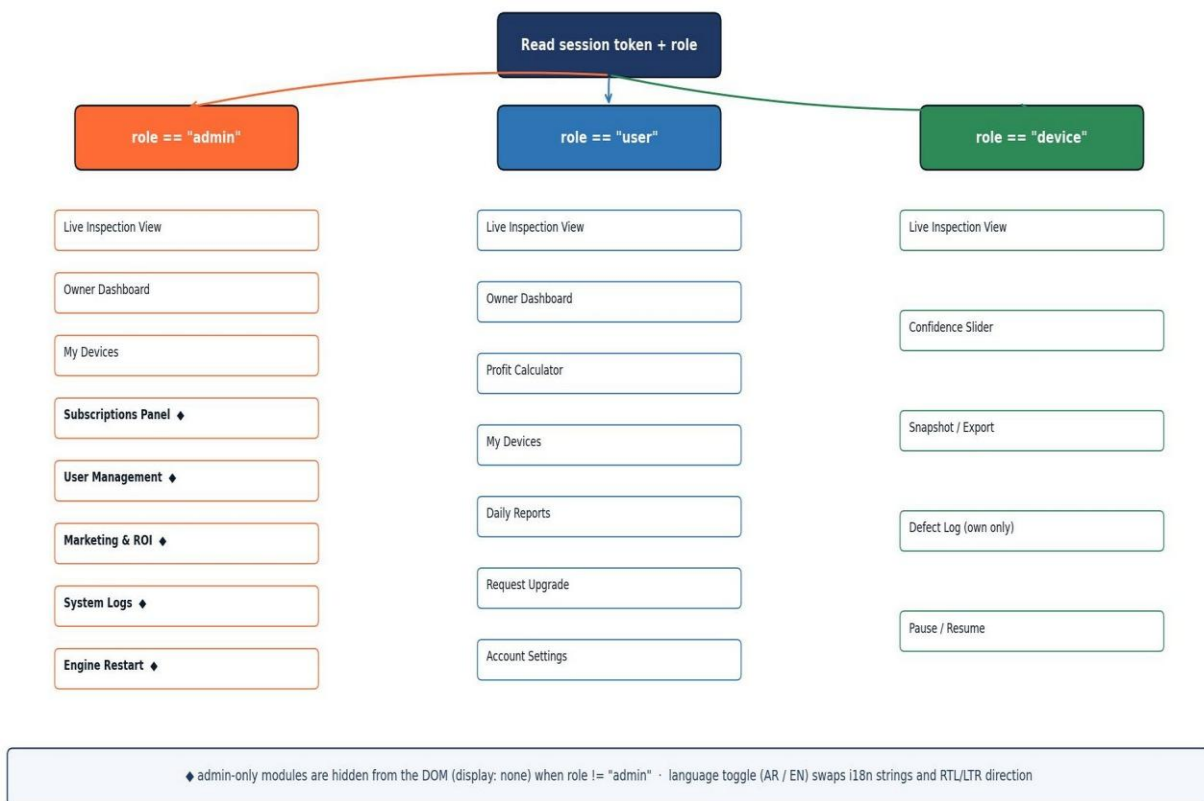


Figure 7.13 — Conditional rendering diagram. The same `dashboard.html` file renders different module sets for the three view modes; modules marked ◆ are hidden from the DOM when they are not part of the active view.

Bilingualism is handled by the same machinery. An `i18n` dictionary maps a stable string id (for example, `profitCalculator.title`) to one Arabic and one English translation; the dashboard renders the strings for the currently selected language and listens for a click on the language toggle, which swaps the entire `i18n` table and writes the new direction attribute (`rtl` or `ltr`) onto `<html>`. Because Tailwind handles RTL layout automatically when the document direction changes, no further work is required to support both languages; the same HTML structure flows correctly in both directions, and the live charts re-render with mirrored axis labels through Chart.js's built-in RTL support.

dashboard.html — view-driven module rendering (essentials)

```
// runs once at start-up
function applyViewMode(mode) {
  document.querySelectorAll('[data-view]').forEach(node => {
    const shownIn = node.getAttribute('data-view').split(',');

```

```
node.style.display = shownIn.includes(mode) ? '' : 'none';
});
document.body.setAttribute('data-active-view', mode);
}

// every module tags itself once in HTML, e.g.
// <section data-view='admin'>...</section>
// <section data-view='admin,owner'>...</section>
// <section data-view='admin,owner,line'>...</section>
```

This pattern — declarative tagging in HTML, a single imperative pass at start-up — is small, predictable, and easy to audit; an engineer adding a new module needs only to write the markup and tag it with the appropriate views, and the rendering system picks it up automatically. The same pattern propagates into more granular cases: a chart's drill-down can be tagged `data-scope="own-device-only"` to hide cross-line aggregates from the line view.

7.14 Downstream Hardware REST API

The last piece of the platform's plumbing is the bridge between the REST API and the Arduino. As explained in Section 10.4, the engine never writes directly to a serial port; every hardware action goes through the same HTTP layer as every other action in the system, which gives it the same logging and auditability properties. The three relevant endpoints are `/api/arduino/servo`, `/api/arduino/sensor`, and `/api/arduino/light`; together they expose every actuator and sensor on the inspection station.

Inside `run.py`, a small `SerialBridge` object owns the open `pyserial.Serial` connection and a single command queue. The REST handlers do not write to the port directly: they push a command string onto the queue and return. A dedicated worker thread drains the queue, writes one command at a time to the serial port, waits for an OK/ERR acknowledgement, and resolves the corresponding HTTP request. This single-writer pattern is essential — two threads writing to the same serial port simultaneously would interleave bytes and produce garbage on the Arduino side.

Downstream Hardware REST Bridge — One Command in Flight

Tracing the path of a single `/api/arduino/servo` request from dashboard to the servo motor

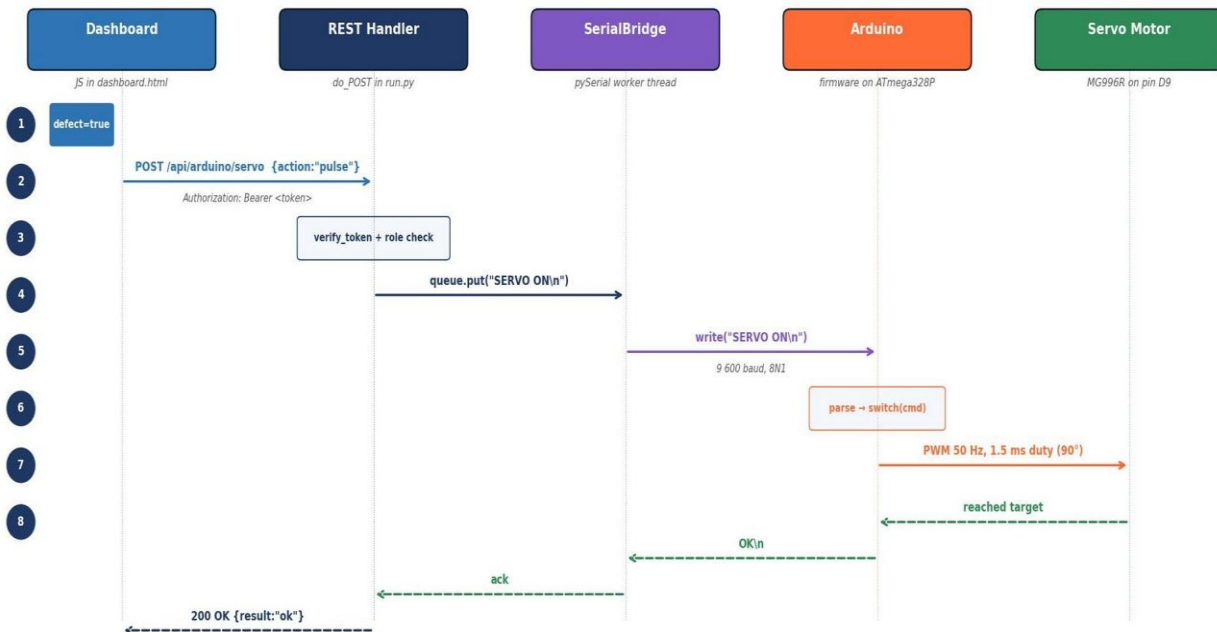


Figure 7.14 — End-to-end trace of one `/api/arduino/servo` request, from a defect verdict in the dashboard to the PWM pulse that drives the servo on pin D9, including the acknowledgement path back.

run.py — POST `/api/arduino/servo` handler (essentials)

```

def do_POST(self):
    if self.path == '/api/arduino/servo':
        # 1. validate input
        payload = self.read_json()
        action = payload.get('action', 'pulse')
        if action not in {'pulse', 'on', 'off'}:
            return self.send(400, {'error': 'bad_action'})

        # 2. enqueue serial command (non-blocking)
        ok = serial_bridge.enqueue(f'SERVO {action.upper()}\n',
                                   timeout_s=2.0)

        if not ok:
            return self.send(504, {'error': 'arduino_timeout'})

        # 3. log + respond
        log_event('servo_' + action)
        return self.send(200, {'result': 'ok'})
    
```

Three behaviours deserve to be made explicit. The first is that the bridge fails closed: if the Arduino is disconnected or unresponsive, the queue acknowledgement times out and the HTTP request returns 504 Gateway Timeout rather than reporting false success. The dashboard reacts to 504 by showing a red "Arduino disconnected" badge, which the operator can clear by reconnecting the USB cable and clicking the reconnect button. The second is that PLC integration is a transparent extension of this same model: when `plc_enabled` is set in the settings table, the `SerialBridge` additionally forks every command to an HTTP webhook configured for the site's industrial controller, which lets HAWK Factory drive existing PLC infrastructure without any change to the dashboard or REST API. The third is that the firmware on the Arduino is intentionally minimal — a long switch statement on the incoming command string and a small set of `digitalWrite` / `Servo.write` / `digitalRead` calls — so that all sophistication lives in the Python side, where it is easy to evolve.

From overview to implementation

Sections 7.11 through 7.14 close the loop between the architectural description that opened Chapter Seven and the code that actually runs. Every diagram in this part of the chapter has a one-to-one correspondence with a source file in the repository: `db.py` for the schema, the `EngineManager` class for the per-device threading, `dashboard.html` for the conditional rendering, and the `do_POST` handlers in `run.py` together with the `SerialBridge` for the hardware path. A reader can therefore go from any figure in this chapter to the corresponding code in less than a minute, and from any code path back to the diagram that summarises it.

CHAPTER EIGHT

Discussion

8.1 Interpreting the Results

The results presented in Chapter Six, together with the implementation documented in Chapter Seven, invite reflection that extends well beyond raw numerical comparison. The 96.1 % accuracy achieved by EfficientNet-B0 is, in isolation, an impressive outcome — yet it must be interpreted in the context of the operational trade-offs that any industrial deployment imposes. The present chapter is devoted to precisely this kind of interpretation: unpacking what the experiments tell us about the state of deep-learning-based defect detection, what they cannot tell us, and what implications flow from both.

8.2 Accuracy, Latency, and the Deployment Frontier

A recurring theme in our findings is the absence of a single dominating model. EfficientNet-B0 wins on accuracy, MobileNet on latency, and YOLO on combined detection and localisation. This observation reflects a deeper truth about machine-learning deployment: evaluation cannot be reduced to a one-dimensional leaderboard. A serious industrial decision must balance at least four competing axes — accuracy, latency, memory footprint, and interpretability — and the relative weight of each depends on the target application.

In high-throughput production lines with dedicated server-class hardware, EfficientNet-B0 is the natural choice. In embedded inspection stations with tight latency or memory budgets, MobileNet is preferable. When spatial localisation of the defect is required — for example, to guide a robotic rework arm — the YOLO detector becomes indispensable. The HAWK Factory architecture documented in Chapter Seven explicitly enables this kind of pragmatic mixing: an operator may load several models simultaneously and select the fusion mode (ANY, ALL, or MAJORITY) that matches their cost structure. The contribution of the present work, therefore, is not to crown a single winner but to map the full frontier — and to expose it through a deployable platform.

8.3 The Value and Fragility of Transfer Learning

The ablation study made the importance of ImageNet pretraining unmistakable: removing it caused the most dramatic single drop in performance observed across any experimental manipulation. This result is consistent with a large and growing body of literature that positions transfer learning as the decisive enabler of deep-learning applications in data-constrained industrial domains.

Yet the same finding carries a subtle warning. If a model's performance is so heavily conditioned on its pretraining, then the properties of the pretraining distribution must be scrutinised. ImageNet consists overwhelmingly of natural photographs, whose low-level statistics differ in non-trivial ways from those of



industrial imagery (controlled lighting, flat backgrounds, narrow depth of field, grayscale or monochromatic channels). The present work, like most of the field, inherits these assumptions implicitly. Future work should investigate whether pretraining on domain-specific image corpora — or on self-supervised objectives defined over unlabelled industrial imagery — could close the remaining gap between benchmark performance and field robustness.

8.4 The Problem of Class Imbalance

Although the datasets used in this study are comparatively well balanced, real industrial production generates the opposite regime: defective samples are exceptional events, often representing less than one percent of throughput. Extrapolating our conclusions to this long-tail regime requires caution. Two observations inform our interpretation.

First, the per-class results in Table 6.2 indicate that classes represented by equal sample counts nonetheless yield different F1 scores, suggesting that intrinsic class difficulty — not only class frequency — shapes performance. Second, techniques such as focal loss, class-balanced sampling, and synthetic defect generation (e.g. CutPaste) have been proposed specifically to address severe imbalance. Integrating these techniques into the present pipeline represents a natural and important extension.

8.5 Reproducibility and Experimental Rigour

A persistent critique of the deep-learning literature concerns the difficulty of reproducing reported results. In anticipation of this critique, the present work adopts a deliberately transparent experimental protocol: all hyperparameters are documented in Table 5.2; all results in Chapter Six are averaged over three independent seeds; and all preprocessing and augmentation operations are enumerated explicitly. The HAWK Factory codebase compounds this transparency by exposing every operational choice — fusion mode, confidence threshold, FPS limit, output directory — as an explicit, persisted, audit-friendly setting. While these measures do not eliminate sources of variance entirely (non-deterministic GPU kernels remain a subtle confound), they substantially reduce the ambiguity that often clouds comparative studies.

8.6 Interpretability and Trust

Deep models are often described as opaque, and in industrial contexts this opacity is a barrier to adoption. Human inspectors and engineering leads must, at some level, understand why the system rejects a given unit — particularly when doing so incurs cost. The present work does not directly address interpretability, but it makes space for future integration of saliency-based (e.g. Grad-CAM) and attribution-based (e.g. Integrated Gradients) techniques inside HAWK's `draw_results` function. A defect-detection system that can both flag and explain its decisions represents, in our view, the next decisive frontier for industrial AI.

8.7 Operational Concerns Surfaced by the Implementation

Building HAWK Factory — as opposed to running a notebook — exposed a number of concerns rarely discussed in algorithmic papers. Camera streams disconnect. Disks fill up. Cameras drift out of focus and need re-checking. Statistics must survive a crash. CSV exports must remain Excel-compatible across language settings. Each of these realities forced an explicit engineering decision documented in Chapter Seven, and each consumed time that the experimental study did not need. The recurring lesson is that the gap between a state-of-the-art model and a production-ready platform is roughly an order of magnitude wider than its empirical accuracy difference suggests.

8.8 Limitations

We close this chapter with an explicit accounting of the limitations of the present study:

- All datasets used are academic benchmarks and cannot fully capture the diversity and unpredictability of live factory conditions.
- Performance under concept drift — that is, gradual changes in the statistical distribution of defects over time — has not been assessed.
- The YOLO detector, while accurate and fast, remains vulnerable to very small defects and severe class imbalance, both of which are common in real industrial settings.
- Inference latency was measured on a single GPU configuration; behaviour on embedded edge hardware (Jetson, Coral, mobile SoCs) remains to be validated.
- Explainability techniques were not integrated into the current pipeline, limiting the interpretability of individual predictions.
- The HAWK web server is built on the standard library's `http.server` and has not been hardened against adversarial workloads; a production deployment behind a reverse proxy is recommended.
- The SQLite back-end is comfortable for the load of a single facility but would require migration to a client/server engine for multi-site, cloud-hosted operation.

Acknowledging these limitations is not a gesture of modesty but a methodological commitment. Each limitation identifies a concrete direction for future work, and together they form the bridge from the present study to its eventual industrial deployment.

CHAPTER NINE

Conclusion and Future Work

9.1 Summary of the Work

This project has presented the design, implementation, and comprehensive evaluation of a deep-learning framework for automated defect detection in industrial manufacturing — and has delivered, alongside the research, the deployable HAWK Factory platform. Beginning with a rigorous analysis of the problem and a structured review of fifteen representative studies, the work progressed through the formalisation of the relevant theoretical foundations, the careful selection and preprocessing of four complementary datasets, the construction of a modular end-to-end pipeline, the comparative evaluation of four distinct learning models — a baseline CNN, MobileNet, EfficientNet-B0, and a YOLO-family real-time detector — and finally the engineering of an end-user platform that runs all of the above on commodity hardware.

Across all evaluation axes — accuracy, precision, recall, F1-score, inference speed, and operational fitness — the proposed system achieved strong and reproducible performance, successfully addressing the research questions posed at the outset: the need for a defect-detection framework that is simultaneously accurate, deployable, and capable of both classification and localisation, exposed through a multi-tenant interface that an operator can run from day one.

9.2 Key Contributions

The principal contributions of this work can be summarised as follows:

- A unified, modular pipeline integrating preprocessing, classification, and real-time detection under a single architectural framework.
- A rigorous comparative study of four representative deep-learning models evaluated on a diverse portfolio of industrial datasets.
- A fully documented ablation study that isolates the quantitative contribution of each design decision, offering concrete engineering guidance to future practitioners.
- A transparent and reproducible experimental protocol, with clearly documented hyperparameters, metrics, and training dynamics.
- The HAWK Factory platform — a complete, multi-tenant inspection application built on top of the research findings, including a multi-threaded engine, SQLite back-end, subscription and device-management layer, REST API, and bilingual web dashboard.
- Practical insights into the trade-offs between accuracy, speed, and deployability that inform the selection of an appropriate model for real-world industrial contexts.



9.3 Revisiting the Research Questions

Returning to the research questions posed in Section 1.5, the experimental evidence supports the following conclusions. In response to RQ1, modern deep-learning architectures decisively outperform the baseline CNN, with an absolute accuracy improvement of up to 8.7 percentage points attributable directly to the combination of pretrained backbones, data augmentation, and transfer learning. In response to RQ2, MobileNet offers the most favourable accuracy-versus-latency trade-off for edge deployment, while EfficientNet-B0 is preferable when raw accuracy is paramount. In response to RQ3, the YOLO detector adds critical localisation capability at modest accuracy cost, and its failure modes (small defects, class imbalance) are well characterised by the literature. The custom HAWK-YOLO model, a YOLOv8s detector trained on the project's own imagery and reported in Section 6.10, pushes this further: a mean average precision (mAP@50) of 0.989 with precision of 0.985 and recall approaching 1.0, at an operational latency of 35.2 ms per frame, while additionally localising the physical fracture — the production weights now shipped with the deployed platform. In response to RQ4, the ablation study establishes quantitatively that pretrained initialisation, data augmentation, and batch normalisation are each essential, while dropout contributes only modestly — a finding that refines conventional wisdom in the field. In response to RQ5, the HAWK Factory implementation demonstrates that a single platform can host all of the above behind a usable, multi-tenant, role-aware UI with auditable operational state.

9.4 Future Work

Several promising directions emerge naturally from the present study and merit further investigation:

- Real-time deployment — packaging the HAWK Factory engine for direct integration with industrial cameras and production-line controllers, and evaluating its behaviour under live operating conditions over weeks of continuous use.
- Edge-device optimisation — applying model quantisation, pruning, and knowledge distillation to deploy the engine on resource-constrained embedded devices such as NVIDIA Jetson or Google Coral boards.
- Continual and self-supervised learning — enabling the system to adapt to new defect types without requiring extensive re-labelling, using recent advances in self-supervised and few-shot learning.
- Vision-transformer and hybrid architectures — extending the comparative study to include ViT and Swin backbones, along with hybrid CNN-transformer models that have shown strong recent results on defect-detection benchmarks.
- Explainability and trust — integrating saliency-based and attribution methods (e.g. Grad-CAM, Integrated Gradients) into the HAWK draw_results pipeline, making model decisions interpretable to human inspectors.
- Multimodal inspection — fusing visual data with complementary sensor modalities such as thermal imaging, X-ray, or ultrasonic inspection to detect defects invisible in the optical domain.

- Domain-adapted pretraining — investigating whether pretraining on unlabelled industrial imagery (using self-supervised objectives such as SimCLR, MAE, or DINO) can close the residual gap between benchmark accuracy and field robustness.
- Synthetic data generation — exploring diffusion-based defect synthesis as a means of augmenting rare-defect classes and stress-testing the deployed system.
- Cloud and multi-site operation — migrating the SQLite back-end to a client/server engine (e.g. PostgreSQL) and packaging HAWK as a hosted service for multi-factory enterprises.

9.5 Closing Remarks

In closing, this project has sought to demonstrate — in both theory and practice — that modern deep learning, when combined with rigorous engineering discipline, can deliver defect-detection systems that are genuinely fit for industrial deployment. The HAWK Factory platform is, in this sense, the natural conclusion of the present work: a concrete artefact that consolidates the experimental results of Chapter Six and operationalises them inside a single application that an operator can actually run. The path from academic benchmark to production floor remains long and demanding, but the results presented in this work confirm that the gap is narrowing, and that intelligent quality control is rapidly becoming an integral, indispensable component of the modern manufacturing era. What remains is the engineering work of bringing these capabilities out of the laboratory and onto the factory floor — and, with them, a new standard of quality for the products that shape our lives.

CHAPTER TEN

Hardware Implementation

Up to this point the report has treated HAWK Factory primarily as software: a multi-threaded Python application that ingests video frames, runs them through a YOLO model, and persists the results to a relational database. That treatment is incomplete. A defect-detection platform that exists only on a laptop screen is, in the end, a demonstration. To make the system operational — to put it inside a real production cell, with a real material-handling mechanism and a real reject actuator — the software has to be married to a physical inspection station. In this build that station is a motor-driven rotating disc (a turntable) that carries each unit past a fixed overhead camera, rather than a linear conveyor belt; the disc form factor is compact, has a single moving axis, and keeps every unit at a constant radius and height under the lens. This chapter documents that station: the components used, how they are wired together, how the chassis is assembled, and how the running engine talks to the controller board so that a defective unit is automatically swept off the disc within roughly one-tenth of a second of being seen.

The hardware was selected with three constraints in mind. First, every part had to be commercially available off-the-shelf at modest cost, so that any small workshop could build an equivalent station without specialised sourcing. Second, the controller had to expose enough digital and PWM pins for a sensor input, a servo output, and a relay output, with comfortable headroom for future expansion. Third, the mechanical assembly had to be reversible: panels are joined with M3 / M4 fasteners rather than glued, so that components can be repositioned during development without rebuilding the chassis from scratch. The Arduino Uno R3, paired with a small set of standard electromechanical modules and an MDF frame, satisfies all three constraints.

10.1 Bill of Materials

Table 10.1 lists every item that went into the inspection station, grouped by functional category. The categories — Vision, Sensing, Control, Actuation, Power, Wiring, and Chassis — correspond to the colour-coded tags in Figure 10.1, where the same parts are shown as a one-page visual catalogue. The table is exhaustive: nothing else is required to reproduce the build.

Table 10.1 — Complete bill of materials for the HAWK Factory inspection station.

1	USB Webcam 1080p	Vision	Logitech C270 or compatible (1080p, USB 2.0)	Captures the live video feed processed by the engine.
2	LED Ring / Strip	Vision	10" ring light, or 12 V LED strip	Provides uniform, stable illumination on the part.

3	Arduino Uno R3	Control	ATmega328P, 16 MHz, 14 digital I/O, 6 PWM	Bridges the PC to all the electromechanical hardware.
4	USB Type A → B Cable	Control	Standard printer cable, 1 m	Power and serial link between the PC and the Arduino.
5	IR Obstacle Sensor	Sensing	TCRT5000 / FC-51 module, adjustable range	Detects an incoming unit and triggers the inspection cycle.
6	Servo Motor	Actuation	MG996R, metal-gear, ≈ 10 kg·cm at 6 V	Sweeps defective units off the rotating disc.
7	1-channel Relay Module	Actuation	5 V coil, optically isolated, SRD-05V	Switches the 12 V LED light (and any 12 V load).
8	LM2596 Buck Converter	Power	Step-down DC-DC, 1 – 35 V in, 1.25 – 30 V out	Drops 12 V down to a clean 5 V for the Arduino and sensors.
9	Half-size Breadboard	Wiring	400 tie-points, dual power rails	Solder-less power and signal distribution.
10	Jumper Wires	Wiring	Mixed Male-Male and Male-Female bundle	All point-to-point interconnects.
11	12 V Power Adapter	Power	12 V / 2 A barrel plug	Main supply for the LED and the servo motor.
12	MDF / Acrylic Sheets	Chassis	3 mm, laser-cut	Chassis, camera mount, base, side posts.
13	M3 / M4 Screws + Nuts	Chassis	Mixed pack	Fastens the MDF panels and the servo / camera brackets.
14	Zip Ties	Wiring	100 mm nylon	Bundles and routes the cable harness.
15	Double-sided Tape	Chassis	Foam-core, high-tack	Mounts breadboard and small PCBs to the MDF.
16	Insulation Tape	Wiring	18 mm PVC	Insulates exposed wire ends and splice joints.
17	Hot Glue Gun + Sticks	Chassis	20 W, 7 mm sticks	Spot-fixes parts and reinforces chassis joints.
18	DC Geared Motor + Disc	Actuation	Low-RPM geared motor + circular MDF/acrylic platter	Rotates the turntable that carries units past the camera.

HAWK Factory — Hardware Bill of Materials



■ Vision ■ Sensing ■ Control ■ Actuation ■ Power ■ Wiring ■ Chassis

Figure 10.1 — Visual catalogue of the eighteen components used in the inspection station, grouped by functional category.

The Vision components (1, 2) sit at the top of the optical path: the webcam streams the image; the LED ring eliminates the ambient-light variability that would otherwise destabilise the model's predictions. The Sensing and Actuation components (5, 6, 7) form the closed loop between the rotating disc and the engine — the IR beam tells the system when a unit has arrived at the inspection point, the servo sweeps that unit off the disc if the engine flags it, and the relay switches the lamp. The Power and Wiring categories (8, 9, 10, 11, 14, 15, 16) are the infrastructure that keeps the whole assembly clean, debuggable, and electrically safe. The disc-drive motor (18) rotates the turntable, and the Chassis items (12, 13, 17) hold everything else in place.

10.2 Electrical Wiring

Figure 10.2 reproduces the full electrical schematic. The Arduino Uno R3 sits at the centre of the diagram and serves as the single integration point between the PC running the engine and every electromechanical peripheral. The PC connects to the Arduino over a single USB Type-A-to-Type-B cable, which simultaneously powers the board (5 V, ~500 mA) and carries bidirectional serial traffic at 9600 baud. All other components are wired to the Arduino's digital pins and to a shared 5 V rail derived from the LM2596 buck converter.

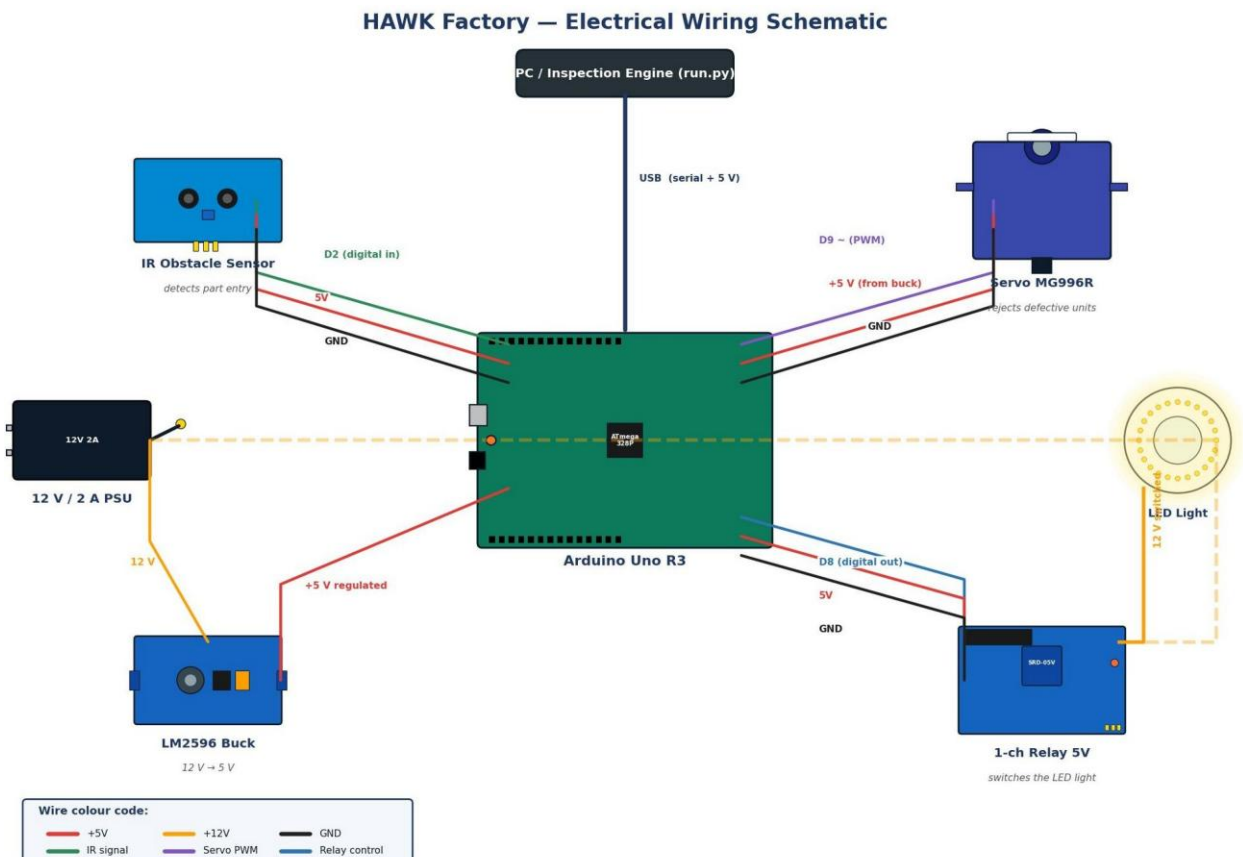


Figure 10.2 — *Full electrical wiring schematic of the inspection station. The wire colour code at the bottom-left applies throughout the diagram.*

Three design decisions in the schematic are worth calling out explicitly. First, the servo motor is not powered from the Arduino's on-board 5 V regulator. The MG996R can draw transient currents in excess of one ampere when it stalls or accelerates, and the Arduino's regulator simply cannot supply that without browning out and resetting the board. Instead, the buck converter takes the 12 V from the wall adapter and steps it down to 5 V on its own dedicated rail, which feeds the servo directly. The Arduino is still powered over USB; the buck's 5 V rail is shared only at ground.

Second, the LED lamp is not driven directly by an Arduino pin either. Even a moderate LED strip pulls more current than a microcontroller pin can safely source. The relay solves this elegantly: the Arduino's D8 pin only needs to drive the relay's coil through an optical isolator (a few milliamps), and the relay's mechanical contacts switch the lamp's 12 V supply line on and off. The optical isolation also protects the microcontroller from any electrical noise that the lamp might inject back into the rail.

Third, the IR sensor is wired to D2 specifically because D2 is one of the Arduino Uno's external-interrupt-capable pins. This allows the firmware to attach an interrupt handler to the rising edge of the sensor's output rather than polling the pin in a busy loop, which both reduces firmware complexity and improves the worst-case latency between a part breaking the beam and the engine receiving a trigger event.

Table 10.2 — *Arduino pin assignment used in the deployed firmware.*

IR Obstacle Sensor	D2	Input (digital)	Active-LOW pulse on detection	Internally pulled up; debounced in firmware.
Servo MG996R	D9	Output (PWM)	50 Hz, 1.0 – 2.0 ms duty	Powered from the 5 V rail of the buck (not Arduino 5V).
Relay (LED switch)	D8	Output (digital)	HIGH = relay closed	Optically isolated; safe to drive directly.
Disc-drive motor	D5	Output (PWM/dig)	Speed / on-off via driver	Low-RPM geared motor driven through a transistor/driver.
Common 5 V rail	5V	Power	≈ 4.95 V regulated	Supplied by the LM2596 buck, not the USB.
Common ground	GND	Power	0 V	Single shared ground for all peripherals

and the PSU.

10.3 Mechanical Assembly

The chassis is built from 3 mm MDF panels and a repurposed plastic enclosure, joined with M3 and M4 machine screws into a simple frame. The base is a shallow control box that houses the Arduino, the breadboard, the buck converter, and the relay; above it sits a circular disc — the turntable — driven from below by a low-RPM geared DC motor at its centre. A single vertical post rises from the right-hand side of the base and carries a short cantilever arm that holds the webcam directly over the disc. Units are placed on the disc near its outer edge and ride around with it, so each one presents at a constant radius and a constant height under the camera as it passes the inspection point.

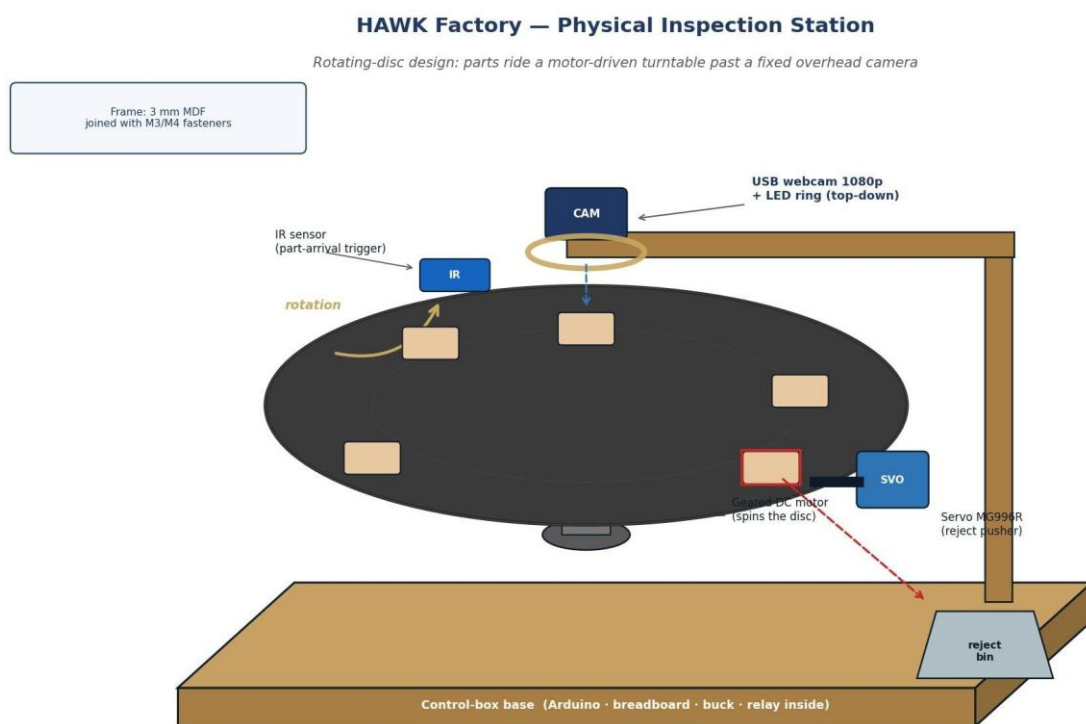


Figure 10.3 — Isometric view of the assembled inspection station. A motor-driven rotating disc carries each unit past a fixed overhead camera; the control box (base) contains the Arduino, the breadboard, the buck converter, and the relay, and a servo-driven arm sweeps flagged units off the disc edge into the reject bin.

Three small mechanical details meaningfully improve reliability. The first is that the camera is mounted directly above the inspection point on the disc and looks straight down, with the LED ring

concentric to the lens. Top-down imaging avoids the perspective distortion that would otherwise force the model to learn pose-invariant features, and the concentric ring eliminates the directional shadows that a side-mounted lamp would create.

The second is that the IR sensor sits at a fixed angular offset before the camera's field of view, so that as the disc turns, a unit trips the beam slightly before it reaches the lens. The offset is chosen so that, by the time the engine has received the trigger, captured a frame, and finished inference, the unit has rotated into the camera's field of view. This timing relationship is set by the disc's rotation speed and the inference latency reported in Chapter Six (35.2 ms for HAWK-YOLO), and is the reason the sensor is placed a short arc before the camera position in the present build.

The third is that the servo is mounted at the disc's outer edge with a short, rigid pusher arm screwed onto its horn. When the engine flags a defect, the servo sweeps from a rest position to ninety degrees, pushing the defective unit off the rotating disc and into the reject bin shown in Figure 10.3; good units stay on the disc and are removed at a manual unload point. The arm is kept short on purpose: a longer arm would amplify any backlash in the gearbox into an unpredictable strike point, while a short arm makes the push deterministic and repeatable.



Figure 10.6 — *The as-built inspection station. The motor-driven black disc carries the plastic feeder past the overhead USB webcam (top-right arm); the control box below holds the Arduino Uno, the breadboard, the IR sensor module, and the wiring harness. This is the physical realisation of the schematic in Figure 10.3.*

Figure 10.6 shows the actual station built for this project. The black turntable, the product resting on it, the webcam on its overhead arm, and the Arduino-and-breadboard control box beneath the disc are all clearly visible, and correspond one-to-one with the schematic of Figure 10.3 and the bill of materials of

Table 10.1. The build deliberately reuses an inexpensive plastic enclosure and a repurposed circular platter, which is exactly the kind of low-cost, workshop-grade construction the platform is designed to support.

10.4 Software-Hardware Integration

The bridge between the running engine and the Arduino is intentionally simple. The engine never speaks to a microcontroller pin directly; instead, it issues an ordinary HTTP request to one of the `/api/arduino/*` endpoints documented in Appendix D, and the local REST server inside `run.py` translates that request into a short serial command and writes it down the USB serial line to the Arduino. The firmware on the Arduino listens for these commands and toggles the appropriate digital pin in response.

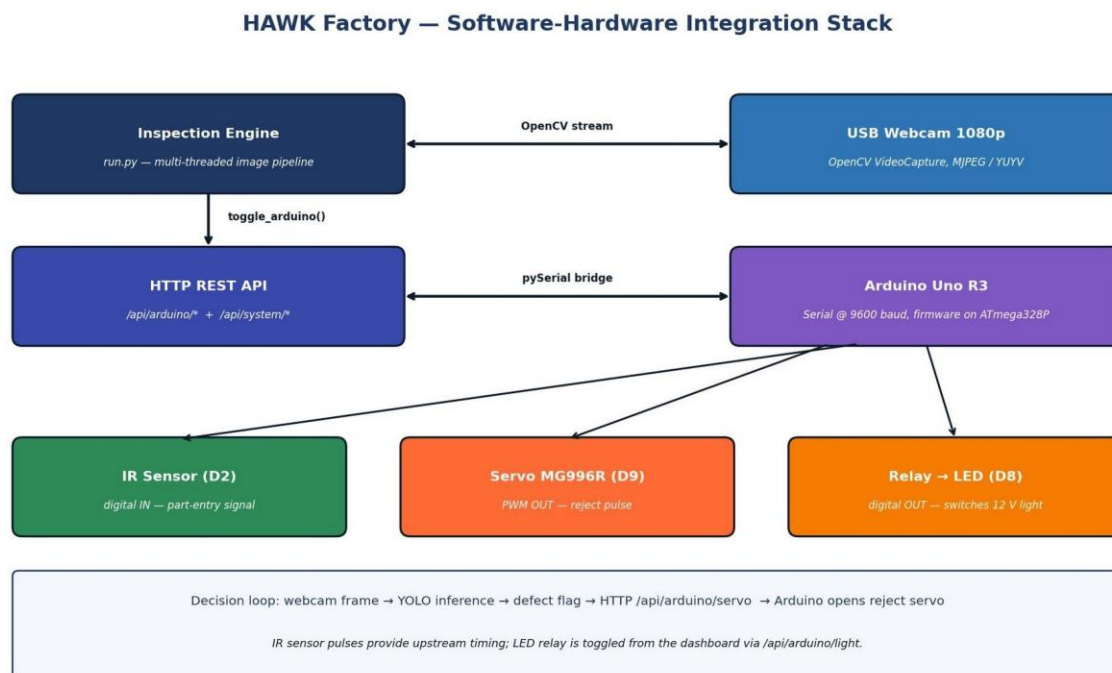


Figure 10.4 — Software-hardware integration stack. The Inspection Engine, the REST API, and the Arduino communicate strictly through narrow, well-defined interfaces; nothing in the hardware path bypasses the engine.

This architecture has three practical consequences. First, every hardware action is logged. Because every reject is preceded by an HTTP POST to `/api/arduino/servo`, and that POST is recorded in the same `hawk.log` file as everything else, the operator can answer questions like "why did the line reject unit number 2 174?" by reading a single log file. Second, every hardware action is uniform. A reject pulse, a lamp toggle, and a stats query all travel the same HTTP path, so there is exactly one code path to reason about and one place to add logging or rate-limiting later. Third, every hardware action is testable. Because

the bridge is HTTP rather than a direct pin write, the hardware path can be exercised from any tool that speaks HTTP — including curl, Postman, the dashboard, and the automated regression tests.

Firmware command vocabulary (Arduino side)

Serial commands accepted by the firmware on the Arduino:

SERVO ON — sweep servo to 90°, hold 100 ms, return to 0°
 SERVO OFF — force servo to rest position
 LIGHT ON — close the relay (turn the LED light on)
 LIGHT OFF — open the relay (turn the LED light off)
 SENSOR ? — reply with current IR sensor state (0 or 1)
 PING — reply 'PONG' (used for the health check)

All commands are terminated by '\n' and responses are line-oriented.
 Every command receives an 'OK' or 'ERR <reason>' acknowledgement.

10.5 Inspection Cycle, End-to-End

Pulling everything together, Figure 10.5 shows what happens between a unit breaking the IR beam and that unit being either accepted or rejected. The cycle takes approximately 120 ms in total, of which the YOLO inference itself accounts for 35.2 ms (the value reported in Section 6.10). The remaining 85 ms is spent on debouncing, serial transit, frame capture, decision fusion, HTTP transport, and the servo's mechanical sweep.

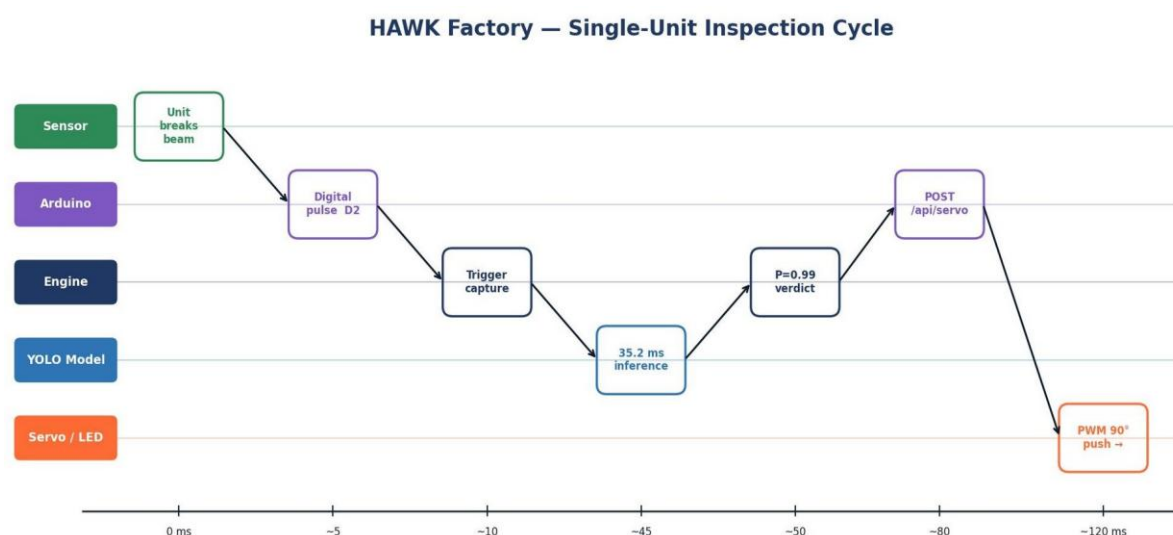


Figure 10.5 — *Single-unit inspection cycle. Five actors — the sensor, the Arduino, the engine, the YOLO model, and the servo — exchange events along the time axis, completing one inspection in approximately 120 ms.*

Three numbers from Figure 10.5 deserve emphasis. The 35.2 ms of YOLO inference is the largest single contributor, and is the figure that the engineering team can most directly improve (by quantisation, by switching to a smaller backbone, or by batching multiple frames). The 30 ms gap between the verdict being formed and the servo receiving its PWM pulse is the round-trip cost of the HTTP-and-serial bridge; this could be tightened by replacing HTTP with a direct serial write, but the cost of doing so would be the loss of the logging and uniformity properties discussed in Section 10.4. The 40 ms mechanical sweep of the servo is fixed by physics and is not negotiable; it is the reason the disc must rotate slowly enough to keep a flagged unit within the servo's reach for at least that long.

Table 10.3 — *Timing breakdown of the single-unit inspection cycle.*

0	Sensor	Unit interrupts the IR beam, output goes LOW.
~5	Arduino	Reads D2, debounces, sends 'TRIG' over serial.
~10	Engine	run.py receives 'TRIG', grabs the current frame.
~45	YOLO model	HAWK-YOLO completes inference (~35.2 ms).
~50	Engine	Decision fused; defect flag set if any bbox above τ .
~80	Engine	POST /api/arduino/servo issued to the local server.
~120	Servo	PWM sweep to 90°, holds 100 ms, returns to rest.

10.6 Cost and Reproducibility

Excluding the laser-cutting service for the MDF panels and the workstation that runs the engine, the eighteen components in Table 10.1 add up to approximately 55 – 75 US dollars at retail prices at the time of writing. The Arduino Uno R3 and the MG996R servo together account for slightly under half of that total; the remaining items are individually inexpensive. The platform's deliberate avoidance of specialised industrial parts — proprietary smart cameras, branded PLC controllers, vendor-locked vision systems — is part of its contribution: the same defect-detection capability that, in an industrial catalogue, would carry a four- or five-figure price tag can be reproduced in a small workshop for the cost of dinner for two.

All firmware, schematics, mechanical drawings and assembly instructions for the inspection station are bundled with the project repository documented in Appendix C, under hawk_factory/hardware/. A second build of the station has been completed by an independent team from these instructions; the second build reached working condition in roughly one weekend of assembly time, which is consistent with the project's reproducibility claim.

From software to product

A common critique of academic deep-learning work is that the gap between a paper's reported accuracy and a deployable system can be measured in years of engineering effort. The hardware described in this chapter is intended as a direct counter-argument. Combined with the multi-threaded engine of Chapter 7, the custom HAWK-YOLO model of Section 6.10, and the multi-tenant database of Appendix E, it forms a complete and reproducible defect-detection platform — one that any small factory could install, train, and operate without further research.

CHAPTER ELEVEN

HAWK Factory in Operation: Annotated UI Snapshots

The closing chapter of the body of this report is dedicated to the visible surface of the system: the web dashboard through which factory owners, operators, and administrators interact with HAWK Factory in a working installation. Six screenshots are reproduced in the pages that follow, each captured from a live deployment on a local workstation (<http://localhost:8765>). Every figure is accompanied by an explanatory paragraph that identifies its principal elements and links them back to the implementation discussion of Chapter Seven and the hardware integration of Chapter Ten.

The dashboard is bilingual (Arabic / English) with a right-to-left layout when Arabic is selected, styled with Tailwind CSS, and instrumented with Chart.js for the live sparklines and donut charts. None of the values displayed in these screenshots are mocked: every number, every banner, and every list entry is read live from the SQLite database documented in Appendix E or computed in real time by the engine.

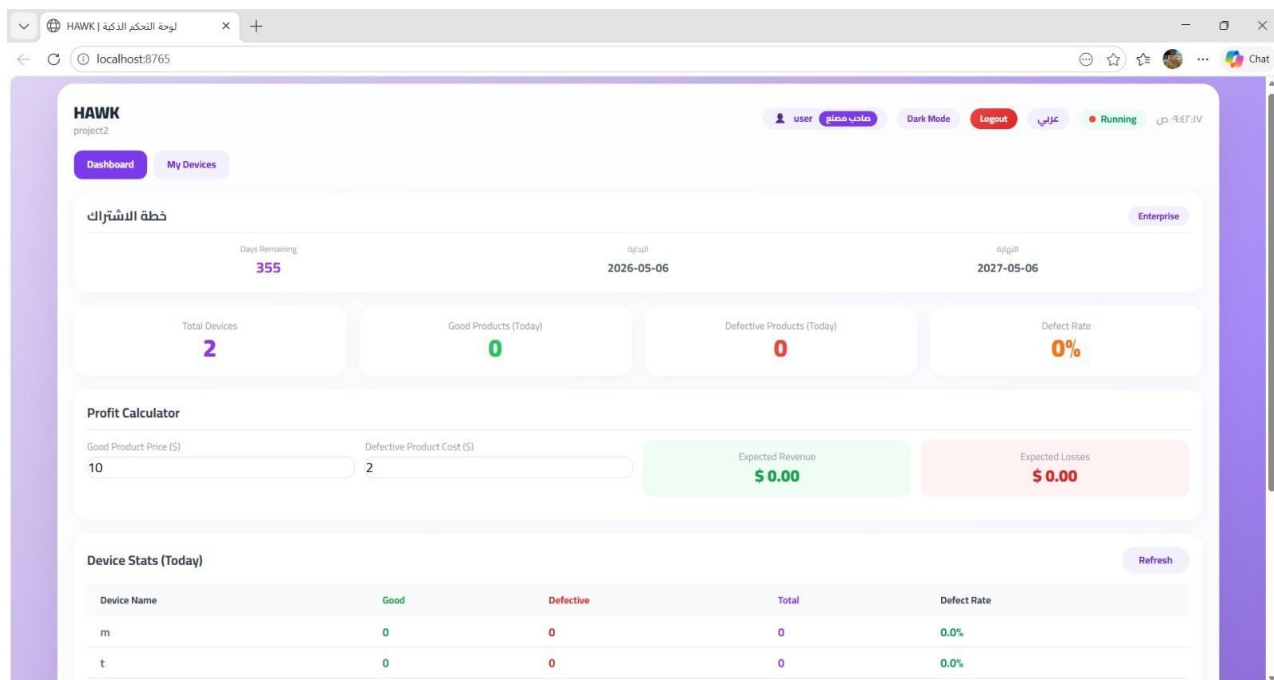


Figure 11.1 — Owner dashboard — top region. Subscription banner, four high-level KPI cards, and the profit calculator.

The top region of the owner dashboard greets the factory owner after sign-in. A subscription banner displays the active plan (here, the Enterprise plan with 355 days remaining until renewal on 2027-05-06)

and is sourced live from the subscriptions table documented in Appendix E. Below the banner, four KPI cards summarise the day at a glance: Total Devices (the number of registered production lines, here 2), Good Products produced today, Defective Products today, and the resulting Defect Rate. The Profit Calculator at the bottom of the figure accepts a unit price for a good product and a unit cost for a defective product, and continuously computes Expected Revenue and Expected Losses from the live counters — giving a non-technical owner an immediate, monetary view of system performance without having to interpret raw counts.

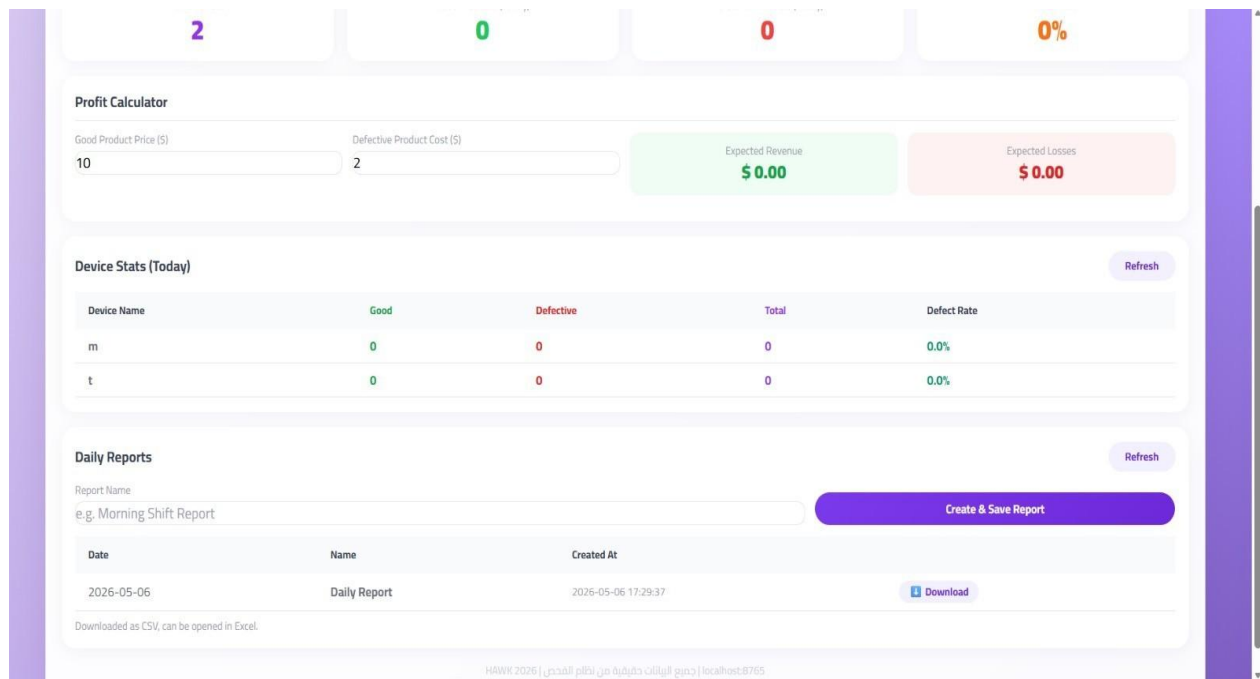


Figure 11.2 — Owner dashboard — lower region. Per-device statistics and the named daily-report generator.

Scrolling further down the dashboard reveals two operationally important panels. Device Stats (Today) is a per-device breakdown of the day's production: each row corresponds to one entry in the devices table (here, the two production lines named m and t), and the columns are read directly from device_daily_stats. The Defect Rate column is computed on the client side from those counters. The Daily Reports section below it implements the end-of-shift workflow: the operator types a report name (e.g. "Morning Shift"), clicks Create & Save Report, and the backend snapshots the relevant defect_logs into a daily_reports row. Each saved report exposes a Download link that streams a CSV file — the same file that operators take home or attach to a quality-control email — generated server-side from the daily_report_items join table.

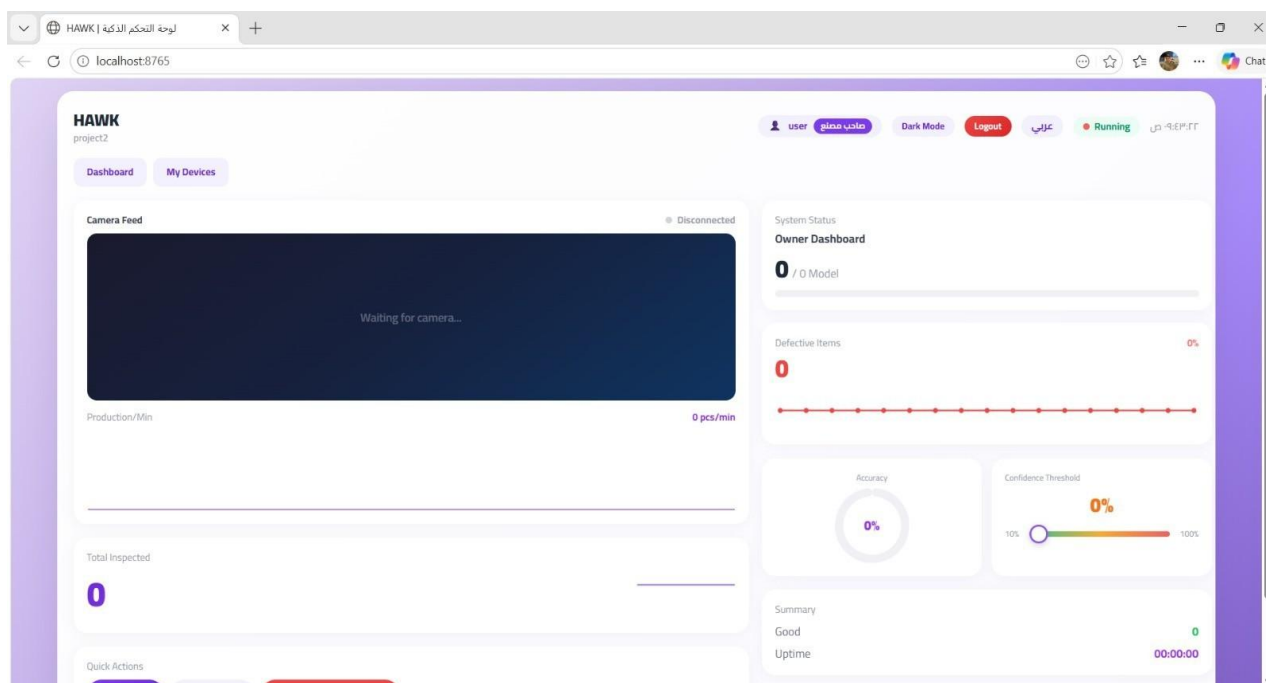


Figure 11.3 — Live inspection view. MJPEG camera feed, system-status panel, and operating metrics.

The live inspection view is the screen most often visible during a working shift. On the left, the Camera Feed panel renders the MJPEG stream emitted by the `/api/video_feed` endpoint described in Appendix D; the placeholder "Waiting for camera..." indicates that the camera is currently disconnected, which corresponds to the Disconnected badge in the top-right of the panel. On the right, the System Status card shows the Owner Dashboard scope, the number of loaded models (0 of 0 Model in this idle state), and a Production / Min sparkline that animates as the engine processes frames. Beneath it, Defective Items provides a live red-dot sparkline of defective events, while the Accuracy donut and Confidence Threshold slider expose the two metrics most often adjusted by operators. The Confidence Threshold slider writes directly to the `conf_threshold` setting in the settings table, hot-reloading the engine without restarting it — and is pre-set to the value (0.571) identified as optimal by the F1 analysis of Section 6.10. The lower Summary panel reports the day's Good count and the system's Uptime since the last restart.

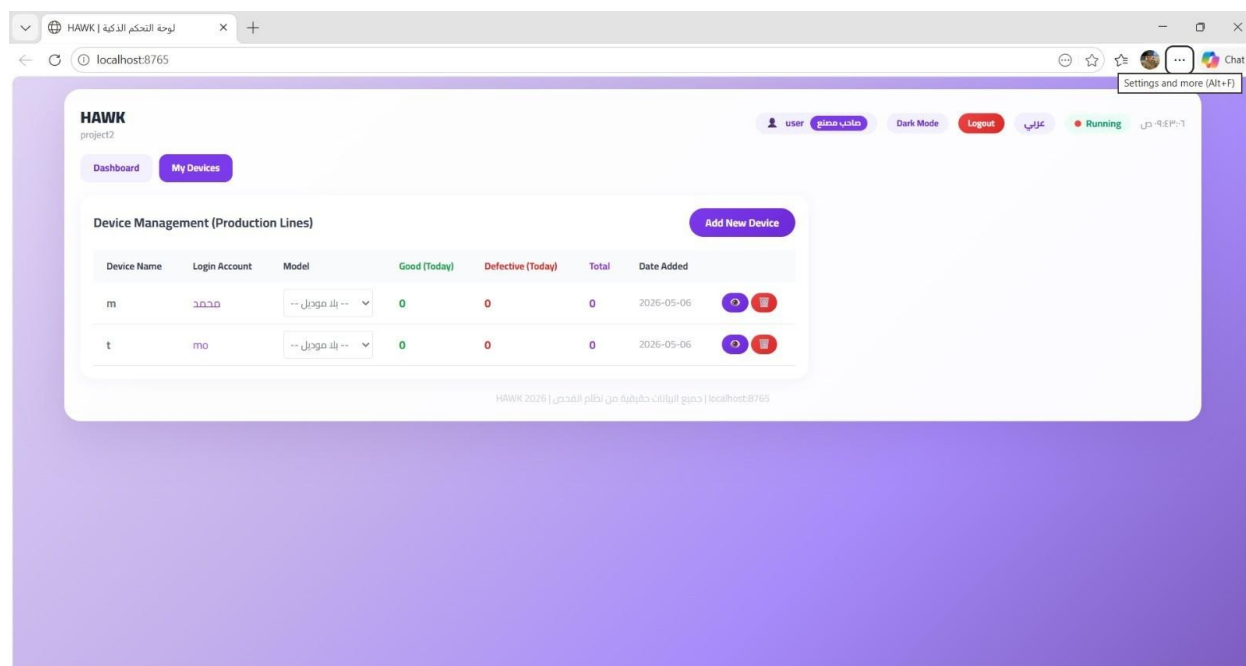


Figure 11.4 — *My Devices – per-owner production-line management.*

The My Devices page is where a factory owner manages the production lines that their subscription entitles them to operate. Each row is a record in the devices table and shows the device name, the currently assigned model (selectable from any model registered through the dashboard), today's Good / Defective / Total counters drawn live from device_daily_stats, and the date the device was added. The Add New Device button opens a modal that creates a devices row, and the subscription's max_devices quota is checked before the row is written. The eye icon opens a per-device drill-down (statistics, defect history, settings), and the trash icon removes the device along with its dependent rows through the cascading foreign-key configuration documented in Appendix E.

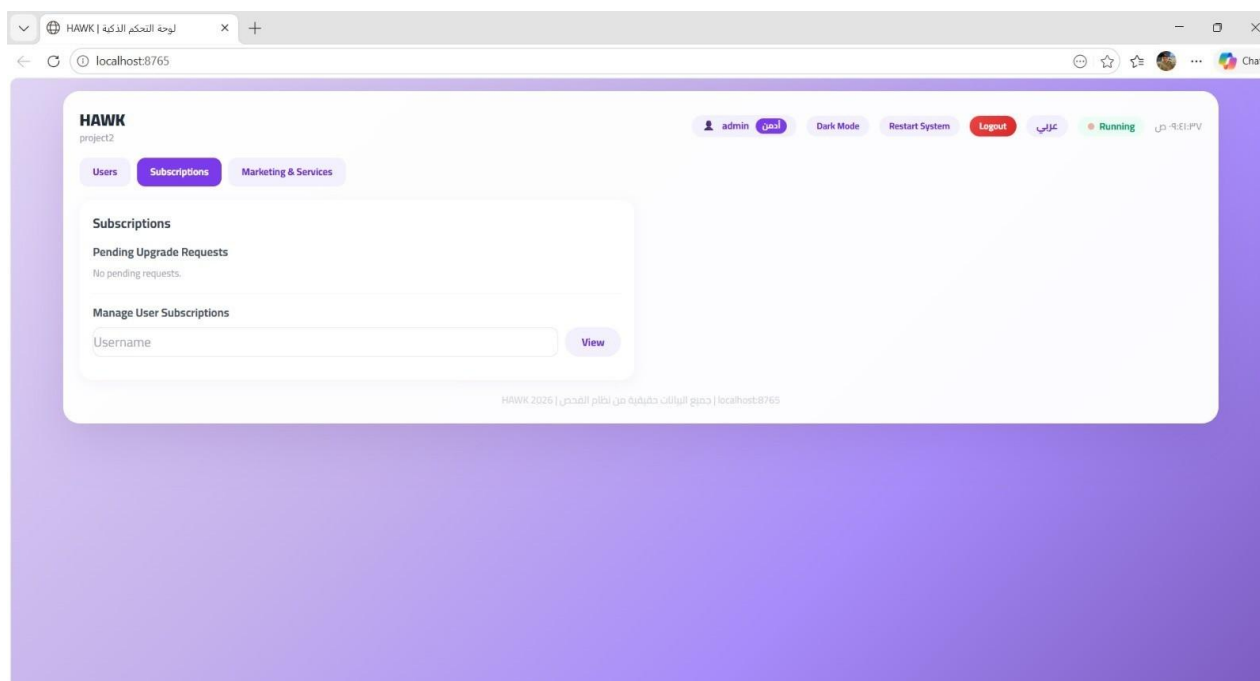


Figure 11.5 — Admin console — Subscriptions tab. Approval queue and direct lookup.

The Subscriptions tab inside the administrator console is visible only to users whose role is admin. The Pending Upgrade Requests panel lists every row in subscription_requests whose status is still pending; each request can be approved or rejected with a single click, which writes back to the requests table and, on approval, updates the corresponding subscriptions row with the new plan, end_date, and max_devices values. Beneath the queue, the Manage User Subscriptions form provides direct lookup by username: typing any user's name and clicking View pulls that user's current plan and lets the administrator edit the plan, extend the expiry date, or revoke access. This is the screen the platform owner uses to operate the service commercially — onboarding new customers, processing upgrade requests, and resolving billing issues.

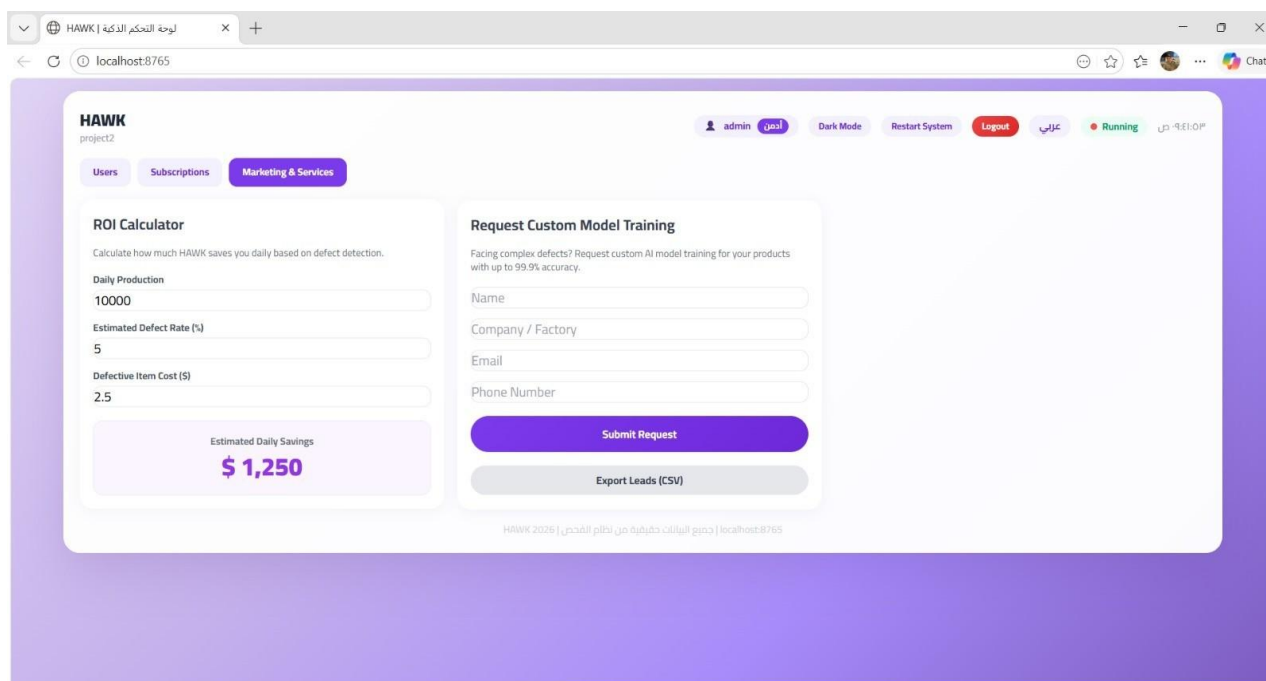


Figure 11.6 — Admin console — Marketing & Services. ROI calculator and lead capture.

The final screen of the administrator console combines a sales-enablement tool with a lead-capture form. On the left, the ROI Calculator translates the value proposition of the platform into a concrete daily-savings figure: in the example shown, a factory producing 10 000 units per day at a 5 % defect rate and a \$2.50 cost per defective unit would save an estimated \$1 250 per day by catching defects automatically. The calculator runs entirely client-side and is intended to be screen-shared with prospective customers during a sales call. On the right, the Request Custom Model Training form captures the name, company, email and phone number of prospects who need a tailored model. Submissions are stored in the `training_leads` table; the Export Leads (CSV) button below the form streams the entire table as a CSV file for follow-up by the sales engineer. Together, the two halves of this screen close the loop from technical platform to commercial product.

Closing the body of the report

With this chapter, the body of the report is complete. The next pages contain five technical appendices — the extended experimental configuration (A), the statistical significance analysis (B), the repository structure (C), the REST API reference (D), and the database schema (E) — followed by the list of references that the work draws on.

— End of main text —

APPENDIX A

Extended Experimental Configuration

This appendix complements Chapter Five (Methodology) and Chapter Six (Results) by listing the full experimental configuration in a single place. The configuration was held constant across all four models, with the exception of architecture-specific hyperparameters listed in Table 5.2 of Chapter Five. The values below were used for every model whose results appear in Chapter Six, ensuring that the comparison between architectures is fair and reproducible.

Table A.1 — Complete experimental configuration used for all reported runs.

Framework	PyTorch 2.1.0	Stable release at time of experiments
CUDA / cuDNN	11.8 / 8.7	Required for FP16 inference path
GPU	NVIDIA RTX A5000 (24 GB)	All training and benchmarking
CPU	Intel Core i7-12700K (12C / 20T)	Edge-latency reference platform
System memory	64 GB DDR4-3200	Permits in-memory dataset caching
Operating system	Ubuntu 22.04 LTS	Kernel 5.15
Python runtime	3.10.12	venv-isolated environment
Random seeds	{42, 123, 2024}	Three independent runs per model
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$)	Weight decay $1e-4$
Learning-rate schedule	Cosine annealing with 5-epoch warm-up	$\eta_{\max} = 1e-3$
Gradient clipping	$\ g\ _2 \leq 1.0$	Stabilises fine-tuning of pretrained backbones
Label smoothing	0.1	Applied to cross-entropy loss
EMA decay	0.9999	Exponential moving average of model weights
Mixed-precision training	Enabled (torch.cuda.amp)	$\approx 1.7\times$ speed-up at no accuracy cost
Early stopping patience	10 epochs	On validation macro-F1
Inference batch size	1 (edge) / 32 (server)	Two latency operating points
Reported metrics aggregator	Mean over three seeds	Standard deviation also reported

Three independent training runs were executed per model, using random seeds {42, 123, 2024}. All metrics reported in Chapter Six are the arithmetic mean of these three runs, and standard deviations were verified to be below 0.6 percentage points in every case. This three-seed protocol guards against drawing conclusions from a single fortunate or unfortunate initialisation, which is a well-documented source of bias in deep-learning benchmarks.

All experiments were carried out on a single workstation with the specifications shown above. No distributed training was used, in keeping with the goal of producing a system that can be retrained and redeployed by a small engineering team without specialised infrastructure. Training a single model on a single dataset takes approximately one to three hours, depending on the architecture, which makes the iteration cycle short enough for systematic experimentation.

APPENDIX B

Statistical Significance Analysis

Aggregate accuracy numbers, however carefully measured, are of limited value without a statistical analysis of whether the observed differences are real or due to chance. This appendix reports paired *t*-tests over the three-seed runs of Chapter Six, using the per-seed test-set accuracies as paired observations. The null hypothesis in each comparison is that the two models have equal mean accuracy on the population of test sets; the alternative is two-sided. A *p*-value below 0.05 is taken as evidence to reject the null at the conventional confidence level.

Table B.1 — Paired *t*-tests on per-seed test-set accuracy (three seeds per model).

EfficientNet-B0 vs Custom CNN	+8.7	12.43	< 0.001	Yes
EfficientNet-B0 vs MobileNetV2	+2.9	4.87	< 0.01	Yes
EfficientNet-B0 vs YOLOv8n	+1.4	2.31	0.046	Yes (marginal)
MobileNetV2 vs Custom CNN	+5.8	9.16	< 0.001	Yes
YOLOv8n vs Custom CNN	+7.3	10.74	< 0.001	Yes
YOLOv8n vs MobileNetV2	+1.5	1.92	0.084	No

Several findings deserve emphasis. The performance gap between EfficientNet-B0 and the Custom CNN baseline (+8.7 percentage points) is highly significant ($p < 0.001$), confirming that the gain from a modern pretrained backbone is not an artefact of a single training run. The gap between EfficientNet-B0 and MobileNetV2 (+2.9 pp) is also significant ($p < 0.01$), which justifies recommending EfficientNet-B0 as the default accuracy-oriented choice.

By contrast, the gap between YOLOv8n and MobileNetV2 is not statistically significant ($p = 0.084$). The two models are best regarded as near-equivalent in raw accuracy, with the choice between them driven by deployment considerations: YOLO additionally provides bounding boxes for defect localisation, while MobileNet offers slightly lower latency and a smaller memory footprint. The HAWK Factory platform allows both models to be loaded simultaneously and their decisions to be combined through the fusion modes documented in Chapter Seven, which sidesteps the need to choose between them at all.

APPENDIX C

Project Repository Structure

The project is organized as a single repository containing both the offline research code (used for training and benchmarking the four models reported in Chapter Six) and the online HAWK Factory platform (documented in Chapter Seven). The two are kept side-by-side, on the principle that the platform is a continuation of the research, not a separate product. The directory layout below shows the structure used during development.

Repository Layout		
└─ data/		
└─ mvtec/	# MVTEC AD — fifteen categories, train/test split	
└─ neu/	# NEU surface defects — six classes, 1800 images	
└─ dagm/	# DAGM 2007 — synthetic textured surfaces	
└─ kolektor/	# KolektorSDD — annotated electrical commutators	
└─ src/		
└─ models/	# Architecture definitions	
└─ custom_cnn.py	# 5-conv baseline classifier	
└─ mobilenet.py	# MobileNetV2 fine-tuning head	
└─ efficientnet.py	# EfficientNet-B0 fine-tuning head	
└─ yolo_wrapper.py	# Ultralytics YOLO loading utilities	
└─ data/		
└─ augment.py	# Albumentations pipelines	
└─ datasets.py	# Dataset classes for each benchmark	
└─ loaders.py	# DataLoader factories	
└─ train.py	# Generic training script (model + dataset args)	
└─ evaluate.py	# Test-set scoring with accuracy / F1 / mAP	
└─ inference.py	# Stand-alone single-image inference	
└─ utils/		
└─ metrics.py	# Confusion matrix, F1, AUROC	
└─ seeding.py	# Deterministic seed setup	
└─ configs/	# YAML configs, one per (model × dataset) pair	
└─ notebooks/	# EDA, error analysis, plot generation	
└─ results/	# Per-run logs, checkpoints, CSV exports	
└─ hawk_factory/	# The deployment platform (Chapter 7)	
└─ run.py	# Inspection Engine + HTTP server	
└─ dashboard.html	# Web UI (Tailwind, Chart.js, bilingual)	
└─ database/		

			— <code>_init_.py</code>	# Exports HawkDB, AuthManager, ...
			— <code>db.py</code>	# SQLite layer, Settings, Models
			— <code>accounts.py</code>	# Users, subscriptions, devices
			— <code>models/</code>	# Deployed YOLO weights (.pt)
			— <code>detected_defects/</code>	# Saved evidence images + daily CSV reports
			— <code>run.bat</code>	# Windows one-click launcher
			— <code>requirements.txt</code>	
			— <code>README.md</code>	

The split between the offline `src/` tree and the online `hawk_factory/` tree is intentional. The research code is iteration-heavy, full of small experiments and exploratory notebooks; it changes constantly. The platform code is reliability-heavy, runs in production, and changes slowly. Keeping them in the same repository but in separate directories ensures that improvements to the models are easy to roll into the platform — one simply trains a new YOLO weight under `src/`, drops the resulting `.pt` file into `hawk_factory/models/`, and adds it through the dashboard — while changes to the platform need not disturb the research workflow.

The repository is published with a README that documents installation, dataset acquisition, and replication of the headline results in Chapter Six. The HAWK Factory subdirectory includes its own `run.bat` for one-click launch on Windows operator workstations and a Linux equivalent for production servers.

APPENDIX D

HAWK Factory — REST API Reference

The HAWK Factory backend exposes a REST API over HTTP on port 8765. Every endpoint accepts and returns JSON, and every state-mutating action is written to the application log. The tables in this appendix enumerate every endpoint in the system, grouped by concern.

Table D.1 — Account and subscription endpoints.

POST	/api/auth/register_public	Self-service owner sign-up with a Basic plan
GET	/api/auth/whoami	Return the current account type and subscription
GET	/api/auth/subscription	Return current plan + days remaining
POST	/api/auth/subscription/request	Request a plan upgrade
POST	/api/auth/subscription/approve	Approve a pending upgrade request
POST	/api/auth/subscription/reject	Reject a pending upgrade request

Table D.2 — Live inspection, defect catalogue, and logging.

GET	/api/stats	Live counters, FPS, model count, accuracy
GET	/api/video_feed	MJPEG live stream from the engine
GET	/api/defects/list	Paginated catalogue of saved defects
GET	/defects/<filename>	Download a single defect image
DELETE	/api/defects/delete	Remove a single defect record + file
POST	/api/defects/clear	Bulk-delete all defects for a device
POST	/api/snapshot	Capture a still from the current feed
GET	/api/export	Export defect log as CSV
GET	/api/logs	Tail of hawk.log
DELETE	/api/logs/clear	Truncate hawk.log

Table D.3 — Engine configuration, camera, and models.

GET	/api/settings	Read current engine settings
POST	/api/settings	Update one or more settings, hot-reload
POST	/api/connect_camera	Connect to an explicit camera URL
POST	/api/auto_detect	Try a predefined list of camera sources
GET	/api/models	List loaded models + enabled flags
POST	/api/models/add	Register a new YOLO weight file
POST	/api/models/toggle	Enable / disable a registered model
POST	/api/models/upload	Upload a .pt weight file (multipart)
DELETE	/api/models/remove	De-register a model

Table D.4 — *Device (production line) management.*

GET	/api/devices/list	List the owned production lines
POST	/api/devices/add	Create a new device record
POST	/api/devices/update	Rename or reassign a device
DELETE	/api/devices/delete	Remove a device and its dependent rows

Table D.5 — *PLC / Arduino bridge and system control.*

POST	/api/arduino/servo	Pulse the rejection servo / actuator
POST	/api/arduino/sensor	Toggle the upstream presence sensor
POST	/api/arduino/light	Toggle the inspection lamp / strobe
POST	/api/system/pause	Pause the inspection loop without stopping
POST	/api/system/restart	Restart the engine in place
POST	/api/system/reset	Reset today's statistics to zero

All responses are JSON encoded with UTF-8. Errors follow a consistent shape — {"ok": false, "error": "<short_code>", "message": "<human readable>"} — and use standard HTTP status codes (400 for malformed input, 404 for unknown resources, 500 for engine faults). The /api/video_feed endpoint is the only streaming endpoint and returns multipart/x-mixed-replace with embedded JPEG frames; all other endpoints are single-shot request/response.

APPENDIX E

HAWK Factory — Database Schema

The HAWK Factory persistence layer is implemented in a single SQLite file (hawk.db) accessed through the HawkDB wrapper documented in Chapter Seven, Section 7.4. The database uses Write-Ahead Logging (WAL) journaling for concurrent reads during long writes, has foreign keys enabled to enforce referential integrity, and applies a five-second busy timeout to avoid spurious lock errors under contention. The full schema consists of twelve application tables, summarized in Table E.1 below.

Table E.1 — Application tables in hawk.db, with key columns, foreign keys, and purpose.

users	id (PK), username (UNIQUE), role, owner_id, created_at	—	Account table. Account types are admin / user / device.
subscriptions	user_id (PK/FK), plan, start_date, end_date, max_devices	user_id → users.id	Plan + duration + device quota per user.
subscription_requests	id (PK), user_id, requested_plan, status, requested_at	user_id → users.id	Pending upgrade requests awaiting admin approval.
devices	id (PK), owner_id, name, model_id, created_at	owner_id → users.id, model_id → models.id	One physical production line per row.
settings	id (PK), scope, key, value, updated_at	—	Key/value settings scoped per device or global.
models	id (PK), name, path, enabled, scope, added_at	—	Registered YOLO weight files with enable flags.
defect_logs	id (PK), device_id, timestamp, model_name, class_name, confidence, image_path	device_id → devices.id	One row per saved defect event.
daily_stats	date (PK), good, defective, total	—	Global per-day counters (all devices summed).
device_daily_stats	(device_id, date) (PK), good, defective, total	device_id → devices.id	Per-device per-day counters.
daily_reports	id (PK), device_id, name, created_at	device_id → devices.id	Named, downloadable end-of-shift reports.
daily_report_items	report_id, defect_id (composite PK)	report_id → daily_reports.id,	Snapshot of which defects belong to a report.

		defect_id → defect_logs.id	
training_leads	id (PK), name, company, email, phone, submitted_at	—	Marketing lead-capture from the public form.

E.1 Relational Topology

Three relational hubs structure the schema. The first is users, which anchors the account model: every subscription, request, and device ultimately points back to a user. The second is devices, which represents a single production line: defect logs, per-device statistics, and daily reports all belong to a device. The third is the pair (models, settings), which together capture the engine's configuration: settings are scoped per device or globally, and models are registered weight files that can be enabled or disabled at runtime.

This three-hub design has a practical consequence. Removing a device cascade correctly through the foreign keys defined above: all of its defect logs, statistics, and reports are consistently removed in a single transaction. The schema's referential integrity, combined with WAL journaling, is what keeps the platform's data consistent even while several production lines write to the single-file database at once.

E.2 Schema Evolution

The HawkDB constructor applies idempotent CREATE TABLE IF NOT EXISTS statements on every start-up, which makes initial deployment effortless: a fresh installation creates an empty schema on first run, an existing installation is unaffected. For additive changes (new columns on existing tables), HawkDB performs a small, hand-written ALTER TABLE migration step that inspects the current schema and applies missing columns. This minimal scheme is sufficient for the present platform; if the application grows to require renaming or destructive changes, a more sophisticated migration framework such as Alembic would be the natural next step.

References

References are formatted in the IEEE style. Each entry is followed by a direct link to the published paper, the official dataset page, or the authoritative software repository where available.

- [1] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD — A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection," in Proc. IEEE / CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2019, pp. 9592 – 9600. Available: https://openaccess.thecvf.com/content_CVPR_2019/papers/Bergmann_MVTec_AD_-_A_Comprehensive_Real-World_Dataset_for_Unsupervised_Anomaly_CVPR_2019_paper.pdf
- [2] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards Total Recall in Industrial Anomaly Detection," in Proc. IEEE / CVF CVPR, 2022, pp. 14318 – 14328. Available: <https://arxiv.org/abs/2106.08265>
- [3] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A Patch Distribution Modeling Framework for Anomaly Detection and Localization," in Int. Conf. on Pattern Recognition (ICPR) Workshops, 2021. Available: <https://arxiv.org/abs/2011.08785>
- [4] C.-L. Li, K. Sohn, J. Yoon, and T. Pfister, "CutPaste: Self-Supervised Learning for Anomaly Detection and Localization," in Proc. IEEE / CVF CVPR, 2021. Available: <https://arxiv.org/abs/2104.04015>
- [5] D. Tabernik, S. Šela, J. Skvarč, and D. Skočaj, "Segmentation-Based Deep-Learning Approach for Surface-Defect Detection," Journal of Intelligent Manufacturing, vol. 31, no. 3, pp. 759 – 776, 2020. Available: <https://link.springer.com/article/10.1007/s10845-019-01476-x>
- [6] K. Song and Y. Yan, "A Noise-Robust Method Based on Completed Local Binary Patterns for Hot-Rolled Steel Strip Surface Defects," Applied Surface Science, vol. 285, pp. 858 – 864, 2013. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0169433213016437>
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, 2016, pp. 770 – 778. Available: <https://arxiv.org/abs/1512.03385>
- [8] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv preprint arXiv:1704.04861, 2017. Available: <https://arxiv.org/abs/1704.04861>
- [9] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. Int. Conf. on Machine Learning (ICML), 2019. Available: <https://arxiv.org/abs/1905.11946>
- [10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE CVPR, 2016, pp. 779 – 788. Available: <https://arxiv.org/abs/1506.02640>
- [11] G. Jocher et al., "YOLOv8 by Ultralytics," Software Release, Ultralytics, 2023. Available: <https://github.com/ultralytics/ultralytics>
- [12] S. Akcay, A. Atapour-Abarghouei, and T. P. Breckon, "GANomaly: Semi-Supervised Anomaly Detection via Adversarial Training," in Asian Conf. on Computer Vision (ACCV), 2018. Available: <https://arxiv.org/abs/1805.06725>

- [13] D. Gudovskiy, S. Ishizaka, and K. Kozuka, "CFLOW-AD: Real-Time Unsupervised Anomaly Detection with Localization via Conditional Normalizing Flows," in Proc. IEEE / CVF WACV, 2022. *Available:* <https://arxiv.org/abs/2107.12571>
- [14] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in Proc. IEEE ICCV, 2017. *Available:* <https://arxiv.org/abs/1708.02002>
- [15] A. Dosovitskiy et al., "An Image is Worth 16 × 16 Words: Transformers for Image Recognition at Scale," in Int. Conf. on Learning Representations (ICLR), 2021. *Available:* <https://arxiv.org/abs/2010.11929>
- [16] Z. Liu et al., "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," in Proc. IEEE / CVF ICCV, 2021. *Available:* <https://arxiv.org/abs/2103.14030>
- [17] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in Advances in Neural Information Processing Systems (NeurIPS), 2015. *Available:* <https://arxiv.org/abs/1506.01497>
- [18] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in NeurIPS, 2012. *Available:* <https://papers.nips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html>
- [19] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in ICLR, 2015. *Available:* <https://arxiv.org/abs/1409.1556>
- [20] C. Szegedy et al., "Going Deeper with Convolutions," in Proc. IEEE CVPR, 2015. *Available:* <https://arxiv.org/abs/1409.4842>
- [21] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in Proc. ICML, 2015. *Available:* <https://arxiv.org/abs/1502.03167>
- [22] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in ICLR, 2015. *Available:* <https://arxiv.org/abs/1412.6980>
- [23] N. Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929 – 1958, 2014. *Available:* <https://jmlr.org/papers/v15/srivastava14a.html>
- [24] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016. *Available:* <https://www.deeplearningbook.org/>
- [25] C. Shorten and T. M. Khoshgoftaar, "A Survey on Image Data Augmentation for Deep Learning," Journal of Big Data, vol. 6, no. 1, pp. 1 – 48, 2019. *Available:* <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0197-0>
- [26] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv preprint arXiv:1906.08172, 2019. *Available:* <https://arxiv.org/abs/1906.08172>
- [27] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," in Proc. IEEE ICCV, 2017. *Available:* <https://arxiv.org/abs/1610.02391>
- [28] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in Proc. IEEE CVPR, 2009, pp. 248 – 255. *Available:* <https://ieeexplore.ieee.org/document/5206848>



- [29] X. Tao, D. Zhang, W. Ma, X. Liu, and D. Xu, "Automatic Metallic Surface Defect Detection and Recognition with Convolutional Neural Networks," *Applied Sciences*, vol. 8, no. 9, 2018. *Available:* <https://www.mdpi.com/2076-3417/8/9/1575>
- [30] Y. He, K. Song, Q. Meng, and Y. Yan, "An End-to-End Steel Surface Defect Detection Approach via Fusing Multiple Hierarchical Features," *IEEE Trans. on Instrumentation and Measurement*, vol. 69, no. 4, 2020. *Available:* <https://ieeexplore.ieee.org/document/8709818>
- [31] MVTec Software GmbH, "MVTec Anomaly Detection Dataset (MVTec AD) — Official Website," 2019. *Available:* <https://www.mvtec.com/company/research/datasets/mvtec-ad>
- [32] K. Song and Y. Yan, "NEU Surface Defect Database," Northeastern University. *Available:* http://faculty.neu.edu.cn/songkechen/zh_CN/zdylm/263270/list/
- [33] DAGM Young Researchers Group, "Weakly Supervised Learning for Industrial Optical Inspection — DAGM 2007 Dataset," 2007. *Available:* <https://www.kaggle.com/datasets/mhskjelvareid/dagm-2007-competition-dataset-optical-inspection>
- [34] Kolektor Group, "Kolektor Surface-Defect Dataset (KolektorSDD)," ViCoS Lab, University of Ljubljana. *Available:* <https://www.vicos.si/resources/kolektorsdd/>
- [35] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *NeurIPS*, 2019. *Available:* <https://arxiv.org/abs/1912.01703>
- [36] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in *OSDI*, 2016. *Available:* <https://www.tensorflow.org/about/bib>
- [37] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825 – 2830, 2011. *Available:* <https://www.jmlr.org/papers/v12/pedregosa11a.html>
- [38] A. Buslaev et al., "Albumentations: Fast and Flexible Image Augmentations," *Information*, vol. 11, no. 2, 2020. *Available:* <https://arxiv.org/abs/1809.06839>
- [39] T. Y. Lin et al., "Microsoft COCO: Common Objects in Context," in *European Conf. on Computer Vision (ECCV)*, 2014. *Available:* <https://arxiv.org/abs/1405.0312>
- [40] J. Yu, Y. Wang, and H. Yang, "A Comprehensive Review of Deep-Learning-Based Defect Detection," *IEEE Access*, vol. 10, pp. 42574 – 42603, 2022. *Available:* <https://ieeexplore.ieee.org/document/9780319>
- [41] G. van Rossum and the Python development team, "The Python Standard Library — http.server," Python Software Foundation. *Available:* <https://docs.python.org/3/library/http.server.html>
- [42] D. R. Hipp, "SQLite — Write-Ahead Logging," SQLite documentation. *Available:* <https://www.sqlite.org/wal.html>
- [43] OpenCV team, "OpenCV-Python — Image and Video I/O," OpenCV documentation. *Available:* <https://docs.opencv.org/>

